2025 edition with Angular 21

The Ultimate Guide to Angular Evolution

How Each Angular Version Impacts Efficiency, DX, UX and App Performance

HOUSE OF ANGULAR

created in cooperation with

angular.love

The Ultimate Guide to Angular Evolution

Author: Mateusz Dobrowolski, Mateusz Stefanczyk, Mitosz Rutkowski
Technical editor: Krzysztof Skorupka, Przemystaw Leczycki, Dominik Donoch, Dawid Kostka, Sebastian Smulski
Marketing project coordinator: Katarzyna Ambrozewicz, Lizaveta Kaliada
Partner Relations Marketing Executive: Natalia Dabrowska, Korneliusz Kopeć
Graphic designer: Izabela Rawa, Paulina Kozłowska

Published by House of Angular
str. Jana Kazimierza 5 (3rd floor)
01-248 Warszawa

www.houseofangular.io

Table of Contents

Preface

This ebook is a combination of Angular roadmap and changelog, supplemented with clear explanations and use cases of all relevant changes in recent years. It brings together all the new features in one place and puts them in a broader context of Angular's evolution. It also highlights how changes introduced between Angular 14 and 21 can influence business outcomes in areas such as development efficiency, long-term maintainability, and strategic decision-making. Regardless of whether you are interested in learning about a specific functionality or want to stay up to date with all the changes and their impact on the framework, this text has got you covered.

Introduction

Angular is gaining momentum we've never seen before. Since 2021, we've been getting a lot of new features on a regular basis, and the pace of change is not slowing down. This is a positive for Angular apps, but it also means you need to dedicate an amount of time every year to keep track of these changes. It's crucial to stay up to date with Angular's changes and new features in order to provide your users with a quality, high-performing application and in-house developers with a good developer experience.

Regularly updating the Angular version will make your application easier to maintain thanks to features and tools that make it easier to develop, test, and improve your application. This translates directly into several benefits for your business, including improved app performance, better security, increased developer efficiency, and better user experience.

This ebook will serve as your guide to previous Angular versions, the changes and new features they brought. As a bonus it provides expert insights and predictions of what the framework will look like in the future.

We will be focusing on versions 14 and up. This is because the earlier period of Angular development was marked by significant internal changes (related to the migration of the compiler and rendering engine to Ivy), which, at the same time, meant much less new functionality and changes to the framework's API. To visualize this, we can try to divide the lifespan of Angular so far into three phases:

	1st Phase	2nd Phase	3rd Phase
Angular major version	2–9	9–13	13+
Period	2016–2019	2019–2021	2021–now*
Summary	Framework foundation	Migration to IVY	Post migration-to-IVY boom

*Now - the time of publication, Q4 2025

Angular along with other frameworks like React and Vue, popularized single-page applications and built a strong community around them. The first versions of Angular were very thoroughly battle-tested. During that period the limitations of the legacy build and rendering pipeline called "View Engine" were learned. After that Angular core developers decided to completely redesign it and gave it a new code name called "Ivy". Complete migration took a few years to finally abandon the previous solution.

Ivy engine introduced a number of new opportunities and brought us into the 3rd Phase, a kind of "boom" of new functionalities and long-term shifts in Angular mental models. In the following pages, we will look at all of these in detail.

How this book is organized

The technical content is grouped by major release versions of Angular (starting with version 14, up to version 21, and beyond). Sometimes, however, functionalities were introduced between major releases or introduced gradually and improved over several versions. While we wanted to follow the chronological order as closely as possible, there may be some deviations that do not disrupt the broader context of changes to the framework.

Our approach to presenting this content is systematic and reader-friendly, ensuring that you can both grasp the technicalities and see the broader picture of each feature's impact. We structured all functionalities into three distinct sections:

Challenge - This section outlines the specific problem or need that the new feature addresses. Understanding the challenge provides context and highlights the significance of the feature in real-world scenarios.

Solution - Here, we delve into the technical details of the feature. We describe how it works, its implementation, and any variations or configurations that are relevant.

Benefits - This part showcases the advantages of using the feature. We focus on how it contributes to business values.

To further enhance your reading and learning experience, each feature is categorized using labels. That way, you have the option of reading this ebook cover-to-cover to get a full picture of Angular's evolution or jump to specific sections that are most relevant to your immediate needs. The labels and structured format make it easy to find and focus on the features that are most pertinent to your current projects or interests.

The four labels are:

Performance

Performance enhancements in frontend development are crucial for both the end-user experience and the efficiency of development processes. These enhancements focus on optimized JavaScript execution, efficient resource loading, and minimized browser reflow and repaint, resulting in quicker page loads and smoother web element interactions. Additionally, these improvements can streamline the development workflow and CI/CD pipeline, enabling quicker builds, more efficient testing, and faster deployment cycles. We use this label to mark such beneficial changes in performance.

Dev Experience

Improving developer experience can increase productivity and code quality. When developers have better tools, processes, and documentation, they can implement features more quickly, reliably and in a more maintainable way. They are more motivated to work on your project and stay in your team for a longer time.

This label marks changes that have a positive impact on the developer's experience.

UX

Focusing on user experience is crucial, because it leads to products that are more intuitive, accessible and enjoyable to use. This can increase user engagement, satisfaction and loyalty, which are important factors for the success of any software application. This label marks changes that improve user experience in Angular projects.

Efficiency

By increasing the speed at which new features and fixes are delivered, companies can respond more quickly to market changes and user feedback, leading to a competitive edge and improved customer satisfaction. This label marks changes that speed up the developer's work and increase his/her efficiency.

We hope this book serves as a valuable resource in your journey with Angular, whether you're a seasoned developer or just starting out. **Happy reading, and happy coding!**

Help us improve this ebook

As new Angular version is coming, we will be happy to prepare the next edition of the ebook and share insights about the latest version, but…
we need your help.

You, as a reader, would know much better how to make the Guide more useful, so it really serves your needs and solves your problems.

Please let us know how you we can improve the ebook:
Share your feedback in this short form

Spread the word

If you find the Guide valuable feel free to **share it with your friends, collegues or other Angular enthusiasts.**

We've created this ebook because we believe that Angular has a great potential and using the latest versions can boost efficiency, developer experience and performance of your applications.

Help us spread this idea!

Share the information about the ebook via **X, LinkedIn, Facebook** or **Reddit.**

Join Angular community and level up your skills

You can also find lots of useful information, tips, case studies at Angular.love community. Check the blog and take part in the Angular events to get knowledge from the most experienced experts and Angular Enthusiasts.

Angular.love blog https://angular.love/
Angular.love meetups and camps https://meetup.angular.love/

Partner conferences:
NG-DE https://ng-de.org/
NGRome: https://ngrome.io/
DevDays: https://devdays.lt/
WeAreDevelopers: https://www.wearedevelopers.com/world-congress/

Typed Forms

Standalone API

Angular v14

release date: **06.2022**

Before version 14, a significant goal was accomplished with the final deprecation of the legacy View Engine. This change, along with aligning all internal tools with Ivy, not only simplified but often enabled the introduction of innovations in Angular. It also led to easier framework maintenance, reduced codebase complexity, and smaller bundle sizes.

This was the first wave of improvements made possible by Ivy. The best examples are the Standalone API (in developer preview), Typed Forms and better developer experience related to debugging applications, both in the CLI and in the browser.

The introduction of standalone components streamlines the development process, reducing the need for additional configurations. Enhanced type safety in forms ensures error-resistant applications, which is especially beneficial for complex use cases. Features like streamlined page title accessibility improve user experience and SEO, while extended developer diagnostics offer actionable performance insights. With support for the latest TypeScript and ECMAScript standards, Angular 14 provides developers with a more powerful, efficient and flexible toolset for building advanced web applications.

Short list of features:	Dev Experience	Efficiency	Performance	UX
Standalone API (developer preview)	✅	☑️	☑️	
Typed forms	✅			
Inject function	✅	☑️		☑️
CDK Dialog and Menu	✅	☑️		☑️
Setting the page title	✅	☑️		
ENVIRONMENT_INITIALIZER Injection Token	✅	☑️		
Binding to protected component members	✅			
Angular extended diagnostics	✅	☑️		
ESM Application Build (experimental)	✅		☑️	
Typescript/Node.js support	✅			
Enhanced Awaited type (available in 4.5)	✅			
Template string types as discriminants (available in 4.5)	✅			
Control Flow Analysis for Destructured Discriminated Unions (Available in 4.6)	✅			
Allows code in Constructors before super() (Available in 4.6)	✅			

Standalone API (developer preview)

Performance | Dev Experience | Efficiency

Challenge:

Although ECMAScript has a native module system supported in all modern browsers, Angular delivers its own module system, which is based on NgModules.

NgModule is a structure that describes how to create an injector and how to compile a component's template at runtime. It includes definitions of components, directives, pipes, and service providers that will be added to the application dependency injectors. The goals of NgModules are to organize the application and extend it with capabilities from external libraries.

This solution was met with some criticism from the beginning, because it was considered too complicated and illegible. The complex connections between modules and their providers, unclear dependencies between components, and unclear NullInjectorErrors were some of the reasons a simplified alternative needed to be provided.

Solution:

In version 14 (developer preview), the Angular Team made NgModules optional with full backward compatibility. Creating components, directives and pipes as standalone is possible by setting the "standalone" flag to true in their decorators.

```
@Component({
  selector: 'footer',
  template: '<ng-content></ng-content>',
  standalone: true,
})
export class FooterComponent {}
```

Standalone components can be imported by another standalone component or module or used within a routing declaration.

```
  //in module
@NgModule({
 imports: [FooterComponent],
})
export class Module {}

//in component
@Component({
 selector: main,
 template: '<ng-content></ng-content>',
 standalone: true,
imports: [FooterComponent ]
})
export class Component {}

//routing
export const ADMIN_ROUTES: Route[] = [
 {path: 'footer', component: FooterComponent}
];
```

This transition makes components, directives and pipes self-contained. NgModule is no longer the smallest building reusable block. This results in many positive outcomes, including:

- a component no longer needs to be defined in its own NgModule and can be reused independently,
- reading components is much easier, as the reader doesn't have to track a component's module to understand its dependencies,
- tracking implicit dependencies on NgModules context is very costly for tools and makes it difficult to optimize generated code. With the standalone API the tools can be optimized,
- the API for dynamic loading and rendering components is simpler, since we no longer have to care about a component's module when using ViewContainerRef.createComponent(...).

Angular's main ApplicationModule is no exception. It also became optional, making it possible to bootstrap application directly with a standalone component using the bootstrapComponent function:

```
import { Component, bootstrapComponent } from '@angular/core';

@Component({
  selector: 'my-app',
  standalone: true,
  template: '<h1>Hi there!</h1>',
})
export class AppComponent {}

bootstrapComponent(AppComponent);
```

How does it affect the routing? The new API allows to lazy load standalone paths directly, with no need for NgModule:

```
RouterModule.forRoot([
 {
   path: '/path/to/standalone/component',
   loadComponent: () => import('./default-standalone.cmp')
 }
]);
```

There is also no need for NgModule while testing standalone components.
This is how the test case can look like

```
const fixture = TestBed.createStandaloneComponent(MyComponent, {...});
```

At this point, in developer preview mode, the change is not production-ready. It has some problems to overcome. (Spoiler: Angular overcomes them in Angular version 15). Still, it presents a major mental model change that turns standalone components into basic building blocks of an application.

The Angular Core Team also made sure that everything was compatible with the existing module-based ecosystem, meaning all existing libraries should work as-is.

Benefits:

This revolution **boosts the developer experience** by:

- reducing boilerplate,
- making reading component code (especially dependencies) easier,
- removing the whole concept of an extra module system based on NgModules,
- improving compilation time, and
- lowering the entry barrier for novice developers.

Reduced bundle size and Tree-Shaking (thanks to function-based API) also improves the performance. More details and benefits are described in chapter "Angular 15", where Standalone API becomes Stable.

Expert Opinion:

Standalone APIs will make the authoring and building of Angular apps much simpler, especially for beginners to the Angular framework. The concept of Angular modules had been complex to explain and many developers who wanted to learn Angular were confused. Thankfully Standalone APIs will change that.

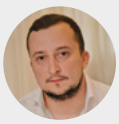
~ Aristeidis Bampakos
Google Developer Expert

From my perspective, the introduction of the Standalone API has significantly simplified the learning curve for Angular. In the past, learners had to grapple with understanding the complex NgModule relationships, including figuring out what to export, where to import, and dealing with the intricacies of eager and lazy-loaded routing. Standalone API, however, empowers developers to concentrate solely on the component they are working on, without the need to concern themselves with the intricacies of the Ng-Module. The extension of this approach to Pipes and Directives is like the icing on the cake.

Furthermore, Angular CLI provides support for creating projects with a focus on the standalone component-based approach, which is a valuable feature. There are also several open-source libraries available to facilitate the migration of applications from NgModule-based architecture to a standalone component-based one, making this transition smoother and more accessible.

~ Balram Chavan
Google Developer Expert

Standalone APIs introduced an important shift towards simplifying application development in Angular. They are the basis for many powerful features that have been introduced recently.



~ Marko Stanimirović
Google Developer Expert

Typed forms

Dev Experience

Challenge:

Before Angular version 14, many APIs related to Reactive Forms (FormGroup, FormControl, FormArray, etc.) included the usage of "any" type. This resulted in poor type safety, bad support for coding tools, codebase inconsistency and problematic refactoring.

Solution:

In response to the challenges mentioned above, the Angular Team introduced Typed Forms. Existing reactive forms were extended to include generic types to enforce type safety for form controls, groups, and arrays. By applying types, you can catch potential errors at compile time, making your codebase more robust and maintainable.

Example of reactive control with a generic type:

```
const nameFormControl = new FormControl<string>('');
// type of nameFormControl.value is string | null
```

The reason control values are nullable is because of the control.reset() method. If you don't pass an argument, it will change the value to null. However, it's possible to change this behavior by setting the new flag called 'nonNullable':

```
const nameFormControl = new FormControl<string>('', { nonNullable: true });
// type of nameFormControl.value is string

nameFormControl.reset();
// reset method will change value to empty string
```

Things get more interesting when we inspect the behavior of complex controls, like FormGroup:

```
const myForm = new FormGroup({
  name: new FormControl('', { nonNullable: true }),
});
// type of myForm.value.name is string | undefined
// type of myForm.getRawValue().name is string
```

The possibly undefined type comes from the fact that name control might be disabled (and not included in form value object).

For the use cases where we don't know all control keys beforehand, like when controls are added dynamically, the Angular Team added the new FormRecord class.

```
const myForm = new FormRecord<FormControl<string>>({ });
myForm.addControl('foo', new FormControl<string>('bar', { nonNullable: true
}));
// type of myForm.value is  Partial<{[key: string]: string}>
```

Migrating to Angular 14 will not break any existing untyped forms, because all occurrences of forms classes will be automatically replaced by their untyped versions:

```
const login = new UntypedFormGroup({
   email: new UntypedFormControl(''),
   password: new UntypedFormControl(''),
});
```

You can incrementally migrate to Typed Forms by removing the "Untyped" prefix in your application.

Benefits:

Leveraging TypedForms in Angular:

- **Leads to more readable, maintainable, and reliable code:** this feature enhances type safety and development efficiency by providing compile-time checking and improved IDE support for autocompletion and error detection. Discrepancies in data types or structures are caught early, and refactoring becomes safer and more straightforward.

- Facilitates cleaner, more concise code and streamline validation processes, aligning well with Angular's design philosophy for a more cohesive development experience.

Inject function

Dev Experience | Efficiency

Challenge:

So far, when it comes to dependency injection, we have been limited to injections through the constructor. This brings some limitations associated with reusability (as we could not reuse the constructor definition), testing, and a general separation of concerns. For example, classes required a knowledge of how to create their dependencies instead of focusing purely on their main responsibilities.

Solution:

The 'inject' utility function allows us to retrieve an instance of dependency from the Angular Dependency Injection System outside of a class constructor. This function has to be called in an injection context, that is one of the following:

- a constructor of the component, directive, pipe, injectable service or NgModule,
- an initializer for fields of such classes,
- a factory function (in a 'useFactory' object of a provider or an injectable),
- an InjectionToken's factory, or
- function within a stack frame that is run in an injection context.

Let's take a look at an example of usages inside a component:

```
@Component({...})
class HomeComponent  {
  private readonly dependencyA: DependencyA;
  private readonly dependencyB: DependencyB = inject(DependencyB);

  constructor() {
    this.dependencyA  =  inject(DependencyA);
  }
}
```

When you want to verify if you're in an injection context, you can use a helper function called assertInInjectionContext:

```
function getService(): FooService {
  assertInInjectionContext(getService);
  return inject(FooService);
}
```

This feature allows us to create many reusable DI-dependent functions like the following:

```
function getProductId(): Observable<string> {
  return inject(ActivatedRoute).paramMap.pipe(
    map((params) => params.get('productId'))
  );
}


@Component({...})
class ProductDetails  {
   private readonly productId$ = getProductId();
  ...
}
```

Benefits:

The inject function in Angular:

- **Provides flexibility:** allows developers to access service instances and other dependencies outside of class constructors, making code more modular and testable.
- **Streamlines the process of dependency retrieval:** promotes cleaner and more maintainable code by abstracting the complexity of dependency management and injection.

CDK Dialog and Menu

Dev Experience | Efficiency | UX

Challenge:

Before the introduction of the CDK, styling certain components available in Angular Material was difficult due to their ready-made design.In such a situation, it was necessary to overwrite the styles of a given component, which could be problematic.

Solution:

Thanks to the CDK, it is possible to create unstyled dialogs and menus and customize them by ourselves.

Open dialog is called by an open method with a component or with a TemplateRef representing the dialog content and returns a DialogRef instance.

```
const dialogRef = dialog.open(DialogComponent, {
  height: '300px',
  width: '500px',
  panelClass: 'empty-dialog',
});
```

cdkMenu from CdkMenuModule provides directives to create custom menu interactions based on the WAI ARIA specification. It's possible to create your own design or use ready-made classes from directives to make it easier to add custom styles.

A typical menu consists directives:

- cdkMenuTriggerFor - trigger element to open ng-template with menu
- cdkMenu - create menu content after click on the trigger
- cdkMenuItem - create and add item to menu

```
  <button [cdkMenuTriggerFor]="menu">Open menu</button>

<ng-template #menu>
  <div cdkMenu>
    <button cdkMenuItem>Item 1</button>
    <button cdkMenuItem>Item 2</button>
    <button cdkMenuItem>Item 3</button>
  </div>
</ng-template>
```

Benefits:

Angular CDK:

- **Provides a set of tools** to build feature-packed and high-quality Angular components.
- **Simplifies common pattern and behavior implementation.**

Setting the page title

Dev Experience | Efficiency

Challenge:

Angular provides a Title service that allows modifying the title of a current HTML document. To use it, you have to inject it. It is completely independent from routing, so any linking to the data in the routing configuration requires complex logic using Angular Router.

Solution:

Angular 14 offers a new feature - TitleStrategy - to set unique page titles using the new Route.title property in the Angular Router. The title property takes over the title of the page after routing navigation.

```
export const routes: Routes = [
  {
    path: 'home',
    title: 'Home Page',
    loadComponent: () =>
      import('./home/home.component').then((m) => m.HomeComponent),
  }
];
```

It is also possible to provide a custom TitleStrategy to apply more complex logic behind a page title.

```
const routes: Routes = [{
  path: 'home',
  component: HomeComponent
}, {
  path: 'about',
  component: AboutComponent,
  title: 'About Me'  // <-- Page title
}];

@Injectable()

export class TemplatePageTitleStrategy extends TitleStrategy {
  constructor(private readonly title: Title) {
    super();
  }
  override updateTitle(routerState: RouterStateSnapshot) {
    const title = this.buildTitle(routerState);
    if (title !== undefined) {
      this.title.setTitle(`My App - ${title}`);
    } else {
      this.title.setTitle(`My App - Home`);
    }
  }
}

@NgModule({
  ...
  providers: [{provide: TitleStrategy,  useClass: TemplatePageTitleStrategy}]

})
export class MainModule {}
```

Benefits:

This feature is:

- **A new convenient way for manipulating a page title:** with no dependency injection and reactivity overhead.

ENVIRONMENT_INITIALIZER Injection Token

Dev Experience | Efficiency

Challenge:

It was possible to use a class with NgModule decorator to run initialization logic, i.e.:

```
@NgModule(...)
export class LazyModule {
  constructor(configService: ConfigService) {
    configService.init();
  }
}
```

But in the absence of NgModules, some Standalone API counterpart was needed.

Solution:

The Environment injector is a more generalized version of the module injector, introduced together with Standalone APIs in Angular v14. ENVIRONMENT_INITIALIZER is a token for initialization functions that will run during the construction time of an environment injector.

When we navigate to a lazy loaded route, a new environment injector is also created for that route. Then we can provide ENVIRONMENT_INITIALIZER functions that will be executed upon such navigation. Inside the initialization function, you can use the inject(...) function and perform any logic you need when the application is bootstrapped or lazy loaded content is instantiated.

Initialization functions on a standalone application bootstrap:

```
bootstrapApplication(AppComponent, {
 providers: [
  {
   provide: ENVIRONMENT_INITIALIZER,
   multi: true,
   useValue: () => inject(ConfigurationService).init(),
  },
 ],
});
```

Initialization functions on lazy loaded routes:

```
export const lazyRoutes: Routes = [
 {
  path: '',
  component: FooComponent,
  providers: [
   {
    provide: ENVIRONMENT_INITIALIZER,
    multi: true,
    useValue: () => inject(BarService).activate(),
   },
  ],
 },
];
```

Benefits:

This Injection token:

- **Increases efficiency:** we can place our initialization logic for the entire application or a specific lazily loaded route in a way that is compatible with Standalone APIs and NgModules.
- **Improves developer experience:** this part of the process is now more clear and convenient to carry out.

Binding to protected component members

Dev Experience

Challenge:
Only public members of a component were accessible in its component template. It meant that every binding in the template was also a part of the component's public API. Component classes exposed too much and violated encapsulation.

Component's public API is relevant when we reference components programmatically, i.e. via ViewChild decorator:

```
@ViewChild(MyComponent) myComponent: MyComponent;
```

Solution:
In Angular v14, it is possible to use fields marked as protected in the template.

```
@Component({
  selector: 'my-component',
  template: '{{ message }}',  // Now compiles!
})
export class MyComponent {
  protected message: string = 'Hello world';
}
```

Benefits:
With this change, we get:

- **An improved developer experience:** the overall encapsulation is improved. We can now encapsulate (hide) methods and properties that we use inside a component's template but don't want to expose them anywhere else.

Angular extended diagnostics

Dev Experience | Efficiency

Challenge:

Sometimes in the Angular codebase, there are potential anomalies that are not straightforward bugs. For example, when they don't cause a compilation error and meet all syntax requirements. At the same time, objections might occur, and they might not necessarily reflect what the programmer had in mind.

Let's take a look at the following example:

```
<component ([foo])="bar"></component>
```

This is a valid Angular example, but it is not a standard two-way data binding. Instead, it is a one-way data binding for an event (output) called "[foo]". The valid two-way data binding looks like this:

```
<component [(foo)]="bar"></component>
```

Solution:

Angular 13.2.0 brought us a new functionality called Extended Diagnostics. It's a tool built into the compilation process of Angular view templates (it's part of the compiler itself), and it doesn't require any additional infrastructure or scripts. – It simply works out of the box, and with the ng serve during the transpilation process.

Its task is to detect potential anomalies just like the one mentioned above. It serves as a kind of additional linter for angular view template syntax. We enable it inside the tsconfig file in the angularCompilerOptions section.

```
{
  "angularCompilerOptions": {
    "strictTemplates": true,
    "extendedDiagnostics": {
      "checks": {
        "invalidBananaInBox": "error"
      },
      "defaultCategory": "error"
    }
  }
}
```

The list of currently available diagnostics is available in the official documentation: https://angular.io/extended-diagnostics. The Angular Team plans to add new diagnostics in minor releases of the framework. New diagnostics and new bugs may appear along with version upgrades, so by default, detected anomalies are returned as warnings. This can be controlled with the "angularCompilerOptions.extendedDiagnostics.defaultCategory" field in the tsconfig file.

Ideas for new diagnostics can be submitted via the github feature requests: https://github.com/angular/angular/issues/new?template=2-feature-request.yaml

Benefits:

These extended diagnostics result in:

- **Improved code security and reliability:** with no extra effort and/or cost.

ESM Application Build (experimental)

Dev Experience | Performance

Challenge:

The standard Angular bundler is considered quite slow by developers. The Angular Team tested various other approaches that can speed up the package-building process.

Solution:

Angular version 14 introduces an experimental feature that leverages the esbuild-based build system for the "ng build" command. This experimental build system compiles pure ECMAScript Modules (ESM) output.

To enable it, use the following snippet in the angular.json config file:

```
"builder": "@angular-devkit/build-angular:browser-esbuild"
```

Esbuild itself is an extremely fast JavaScript bundler and minifier written in the Go language that's designed for modern web development. It supports heavy parallel processing and might significantly outperform competitors such as Webpack. Its support in Angular framework will be developed in the upcoming releases.

Benefits:

Introducing a new esbuild-based build system is:

- **A step towards faster build-time performance:** including both initial builds and incremental builds.
- **An opening to new tools:** thanks to ESM, which includes dynamic import expressions for lazy module loading support.

TS Typescript/Node.js support

Support for Node.js v12 and Typescript older than 4.6 has been removed. Angular 14 supports TS v4.7 and targets ES2020 by default, meaning initial bundle size is reduced. Here are the new feature examples in the now-supported Typescript:

Enhanced Awaited type (available in 4.5)

Dev Experience

Challenge:

Developers who frequently work with Promises, especially with async/await syntax, sometimes want to explicitly describe the type of value returned by the resolved Promise. For regular synchronous functions, there is the ReturnType<FnType> utility, but before TypeScript 4.5, there was no counterpart for asynchronous functions.

Solution:

A new utility type called Awaited was introduced in TypeScript 4.5. It unwraps promise-like "thenables" without relying on PromiseLike, and it does it recursively.

```
// Name = string
type Name = Awaited<Promise<string>>;


// Age = number
type Age = Awaited<Promise<Promise<number>>>;


// Foo = boolean | number
type Foo = Awaited<boolean | Promise<number>>;
```

Benefits:

Extra explicit typing of async functions:

- **Positively affects the type-safeness**, and
- **Improves code readability.**

Template string types as discriminants (available in 4.5)

Dev Experience

Challenge:

TypeScript could not correctly use template string types to narrow the type in discriminated unions. In such cases, the exact type could not be inferred therefore, typing support was limited.

Solution:

With this feature, Typescript is now able to use template string literals as discriminants. The following example used to fail, but now successfully type-checks:

```
export interface Success {
  type: `${string}Success`;
  body: string;
}


export interface Error {
  type: `${string}Error`;
  message: string
}


export function handler(r: Success | Error) {
  if (r.type === “HttpSuccess”) {
     const token = r.body; // correct!
  }
}
```

Benefits:

This change brings:

- **More flexibility:** when defining types and more intelligent type inference by TypeScript transpiler.

Control Flow Analysis for Destructured Discriminated Unions (Available in 4.6)

Dev Experience

Challenge:

If we had a discriminated union and we tried to destructure it, we could no longer narrow its members using discriminator property.

Solution:

Since TypeScript version 4.6, it is possible to narrow destructed discriminated object properties. The following example explains such a case:

```
type Action =
  | { kind: “NumberContents”; payload: number }
  | { kind: “StringContents”; payload: string };
function processAction(action: Action) {
  const { kind, payload } = action;
  if (kind === “NumberContents”) {
  // payloadis narrowed to number
    let num = payload * 2;
    // …
  } else if (kind === “StringContents”) {
  // payload is narrowed to string
    const str = payload.trim();
    // …
  }
}
```

Benefits:

We gain more flexibility when using and mixing discriminated unions and object destruction.

Allows code in Constructors before super() (Available in 4.6)

Dev Experience

Challenge:

In JavaScript classes, it’s necessary to invoke super() before using “this” keyword. TypeScript also has this rule, although it used to be excessively strict in ensuring it. In TypeScript, it used to be considered an error to have any code at the start of a constructor if the class containing it had any property initializers.

Solution:

From now on we can place a code inside the constructor, before calling “super()”. Note that it is still mandatory to call super() before referring to the “this“ keyword.

```
  class Base {
 // ...
}
class Derived extends Base {
 someProperty = true;
 constructor() {
  doSomeStuff(); // do any logic, but don't refer 'this' yet
  super();
 }
}
```

Benefits:

The unnecessary limitation has been removed and the transpiler still validates the code correctly, giving more freedom to the programmer.

Directive Composition API

Image Directive

Standalone API

Angular v15

release date: **11.2022**

Angular v15 introduces significant improvements, phasing out legacy systems, enhancing developer experience and optimizing performance. Standalone APIs are now stable, supporting simpler development practices and ensuring compatibility with core libraries. The release includes more efficient bundling with tree-shakable standalone APIs for Router and HttpClient and an ngSrc image directive for smarter data fetching and improved performance.

Short list of features:	Dev Experience	Efficiency	Performance	UX
Standalone API (Stable)	✅	☑️	☑️	
Directive composition API	✅	☑️		
Image directive		☑️	☑️	☑️
MDC-based components	✅	☑️		☑️
CDK Listbox	✅	☑️		☑️
Improved stack traces	✅			
Auto-imports in language service	✅	☑️		
Typescript/Node.js support	✅			
The satisfies Operator (available in 4.9)	✅			
Checks For Equality on NaN (available in 4.9)	✅			

Standalone API (Stable)

Performance | Dev Experience | Efficiency

Challenge:

The problems with NgModules and the benefits of the Standalone API were presented in the Angular 14 chapter. The solution at that time had disadvantages related to not being in a stable version, as well as shortcomings regarding providers, adapting many basic modules and benefiting from abandoning modules.

Solution:

Starting with version 15, the standalone APIs drop the developer preview label and become stable. Thanks to this, we get the green light to safely utilize them in our applications, including production.

Let's explore the major changes around Angular Routing. First, we received a new type of guard – canMatch. So what's the difference between this new one, canLoad, which tells us whether we can load a route that references a lazily loaded module, and canActivate/canActivateChild, which tells us whether we can activate a child route/route? CanMatch works, in a sense, at a different, "earlier" stage and decides whether the current url can be matched against a given route. This means it can play a similar role to both canLoad (which it will eventually replace) and canActivate.* However when it returns false, subsequent routing configuration entries are processed.

This means we gain a new possibility – defining a route with the same path, but navigating to different places based on the logic implemented by the guard. This is useful, for example, when handling different user roles, or conditionally loading another version of the feature based on feature flags. Here's a usage example:

```
class CanMatchSettings implements CanMatch {
 constructor(private currentUser: User) {}

 canMatch(route: Route, segments: UrlSegment[]): boolean {
  return this.currentUser.isAdmin;
 }
}

const routes: Routes = [
 {
  path: 'settings',
  canMatch: [CanMatchSettings],
  loadComponent: () =>
   import('./admin-settings/admin-settings.component').then(
    (v) => v.AdminSettingsComponent
   ),
 },
 {
  path: 'settings',
  loadComponent: () =>
   import('./user-settings/user-settings.component').then(
    (v) => v.UserSettingsComponent
   ),
 },
];
```

The Router API has been fully adjusted to the standalone approach, so we no longer need to use the Router Module. Instead, we get a whole set of alternative APIs, which also have the advantage of being easily treeshakeable:

```
const routes: Routes = [...];

bootstrapApplication(AppComponent, {
 providers: [
  provideRouter(
   routes,
   withDebugTracing(),
   withPreloading(PreloadAllModules)
  ),
 ],
});
```

Another new feature is support for functional guards and resolvers. This means that it is possible to implement them in the form of plain functions, so we can say goodbye to the classes being the only option here. The introduction of this concept triggered many reactions among the community, both positive and negative. Some developers see this as a new direction of the framework. Personal preferences aside, one thing cannot be denied – it requires much less boilerplate. What is more, it is now very easy to create higher-order, parametrized functions returning a properly configured version of the guard based on that. A usage example, which you can easily compare with the earlier example, looks as follows:

```
const routes: Routes = [
{
   path: 'settings',
   canMatch: [() => inject(User).isAdmin],
   loadComponent: () =>
    import('./admin-settings/admin-settings.component').then(
      (v) => v.AdminSettingsComponent
    ),
 },
 {
   path: 'settings',
   loadComponent: () =>
    import('./user-settings/user-settings.component').then(
      (v) => v.UserSettingsComponent
    ),
 },
];
```

A small, but lovely, improvement also appeared in the syntax for importing lazy-loaded paths. It is possible to omit the ".then(...)" part if we use the default export in the target file.

```
@Component({
 standalone: true,
 ...
})
export default class MyComponent { ... }

{
 path: 'home',
 loadComponent: () => import('./my-component'),
}
```

Every lazy-loaded route creates a separate injector, just like every lazy loaded ngModule created its own injector. The counterpart of the "providers" array in the NgModule configuration is now moved to the route configuration:

```
{
  path: 'admin',
  providers: [AdminService],
  children: [
    {path: 'users', component: AdminUsersCmp},
    {path: 'teams', component: AdminTeamsCmp},
  ],
}
```

Providers declared in the route are available for the component declared at the same level and for all its descendants.

Similarly to Router, HttpClient has also been adapted to the module-less approach:

```
bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(
      withXsrfConfiguration({
        cookieName: 'MY-XSRF-TOKEN',
        headerName: 'X-MY-XSRF-TOKEN',
      })
    ),
  ],
});
```

The transition to Standalone API also affected Angular interceptors:

```
bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(
      withInterceptors([
        (request, next) => {
          console.log('Url: ', request.urlWithParams);
          return next(request);
        },
      ])
    ),
  ],
```

During migration to functional interceptors, you can still use your class-based interceptors configured using a multi-provider by adding withInterceptorsFromDi utility function as follows:

```
bootstrapApplication(AppComponent, {
  providers: [
    provideHttpClient(withInterceptorsFromDi()),
    {
      provide: HTTP_INTERCEPTORS,
      useClass: YourClassBasedHttpInterceptor,
      multi: true,
    },
  ],
});
```

If in your standalone app, you need any providers from third-party libraries that are available only inside NgModules, you can use the importProvidersFrom utility function:

```
bootstrapApplication(AppComponent, {
  providers: [importProvidersFrom(MatDialogModule)],
});
```

It is also worth mentioning that the shift towards a module-less approach simplifies the creation of dynamic components that are self-contained and no longer need their Ngmodules.

```
@Component({...})
export class MyComponent {
  constructor(private readonly viewContainerRef: ViewContainerRef) {}
  create(): void {
    this.viewContainerRef.createComponent(OtherComponent);
  }
}
```

Benefits:

The proposed module-less approach:

- **Simplifies the process of application development:** reduces the reliance on NgModules and minimizes boilerplate code.
- **Enables faster development cycles and a cleaner codebase:** provides a more direct and streamlined API for key aspects like bootstrapping, routing, and dynamic component instantiation.
- **Improves app performance:** thanks to the shift towards a providers-first approach, which enhances tree-shakability.

Expert Opinion:

The standalone API makes many things easier. Especially since you now have to provide everything you need in one component, and it does not magically come from somewhere. This makes the topic of DependencyInjection much easier to understand. This also made the lazy loading of components possible. My tip for this: Declare the component class as a default export so that you don't have to resolve the promise yourself during lazy loading. This makes the dynamic import shorter.

~ David Muellerchen
Google Developer Expert

Directive composition API

Efficiency | Dev Experience

Challenge:

One of the most wanted features in the framework was the ability to reuse directives and apply their behavior to other directives or components.

Up to this point, there have been few possibilities to partially achieve a similar result, such as the use of inheritance, where the main limitation is that only one base class can be used.

Another idea used, for example, by Angular Material, is the use of TypeScript mixins. But this forces a specific approach to the code shared this way, which heavily complicates implementation and doesn't allow for the use of Angular APIs in mixins.

Solution:

The Directive Composition API in Angular is a feature that allows directives to be applied to a different directive or a component's host element directly from within the component's TypeScript class. The only major restriction is that only standalone directives can be applied to our directives/components, which on the other hand, don't need to be standalone.

```
@Component({
 selector: 'my-component',
 templateUrl: './my-component.html',
 hostDirectives: [
  {
   directive: NgClass,
  },
  {
   directive: CdkDrag,
   inputs: ['data'],
   outputs: ['moved: dragged'],
  },
 ],
 standalone: true,
})
export class MyComponent {}
```

The above piece of code applies the NgClass and CdkDrag directives to our component. The first one doesn't expose any inputs or outputs, so you won't be able to use them in the template where our component is used. The second, on the other hand, exposes both input and output, with an alias defined for output. Therefore, the use of our component could look as follows:

```
<my-component [data]="myData" (dragged)=onDragged($event)>
</my-componenet>
```

But can we control the behavior of applied directives from inside the component? This is possible using the inject function, which allows us to inject the instance of the directive into the component and manipulate its properties. It looks like this:

```
@Component({
  selector: 'my-component',
  templateUrl: './my-component.html',
  hostDirectives: [
    {
      directive: NgClass,
    },
  ],
  standalone: true,
})
export class MyComponent {
  private ngClassDirective = inject(NgClass);
  someCallback(): void {
    this.ngClassDirective.ngClass = 'my-class';
  }
}
```

Benefits:

Directive Composition API:

- **Improves the developer experience:** enhances code modularity by allowing developers to encapsulate and reuse behaviors across different components and directives.
- **Leads to a cleaner and more organized codebase**
- **Makes the maintenance and updating of the application more efficient.**
- **Simplifies the implementation process:** it applies directives directly to the host element from within the TypeScript class, reducing the need for complex configurations and boilerplate code in the templates.

Image directive

Efficiency | Performance | UX

Challenge:

The app may take a long time to load in the browser due to the way images are loaded. This can be especially noticeable when the website contains a lot of multimedia.

Solution:

In collaboration with the Aurora team, the Angular team has introduced the NgOptimizedImage to enhance image optimization and incorporate best practices for image loading performance. It is a standalone directive designed to boost image loading efficiency. It became a stable feature in Angular version 15.

To activate iNgOptimizedImage, simply replace the image's src attribute with ngSrc.

```
<img ngSrc="cat.jpg">
```

If the LCP image is shown, a good way to prioritize its loading is to use property called "priority".

```
<img ngSrc="cat.jpg" priority>
```

Thank for that 'priority' applies three optimizations:

- Sets fetchpriority=high - gets resources in the optimal order and prioritizes the image
- Sets loading=eager - lazy-loading images
- Automatically generates a preload link element if it uses SSR.

NgOptimizedImage requires us to specify a height and width for the image or attribute 'fill' to prevent image-related layout changes. But if the 'fill' attribute is used, it's necessary to set the parent element 'position: relative/fixed/absolute'. When using CDN images, it is possible to compress images and convert them on demand to formats such as WebP or AVIF.

The NgOptimizedImage image directive also offers other solutions to improve application performance when loading images such as:

- supporting resource hints - preloading critical assets
- possibility to preload the resource for all routes instead of manual addition of preload resource hint.

Benefits:

Using the NgOptimizedImage directive and ngSrc attribute is very simple, almost cost-free, and can significantly improve an application's performance, SEO and core web vitals.

MDC-based components

Efficiency | Dev Experience | UX

Challenge:

The current version of the Angular Material library was loosely linked to the official Material Design specification. All the styles and behaviors were reinterpreted and reimplemented to Angular style. With the arrival of Material Design Components for Web (MDC), the library became outdated It became clear that new Angular Material should be strongly and directly based on official MDC design tokens. These include values like colors, fonts and measurements.

Solution:

In Angular Material version 15, a significant migration took place. Many components are now being refactored to be based on Material Design Components for Web (MDC). Various components have been refactored, leading to changes in styles, APIs and even complete rewrites for some. Components like form-field, chips, slider and list have significant changes in their APIs to integrate with MDC. There are library-wide changes affecting component size, color, spacing, shadows and animations to improve spec-compliance and accessibility. Theming changes include updates to default typography levels and themeable density. Each component has specific changes, including style updates, element structure and API modifications.

Benefits:

These changes offer several benefits, such as:

- **Improved accessibility**
- **Better adherence to the Material Design spec**
- **Faster adoption of future Material Design versions:** due to shared infrastructure

CDK Listbox

Efficiency | Dev Experience | UX

Challenge:

Creating a typical listbox with accessibility support, keyboard events support, multiselection and correct scroll behavior, as well as satisfying WAI ARIA listbox pattern requirements is a time-consuming task.

Solution:

The newly added component to the Angular CDK library meets all the above guidelines while being a fully customizable solution.

The code example:

```
<label class="example-listbox-label" id="example-appointment-label">
  Appointment Time
</label>
<ul cdkListbox
    [cdkListboxValue]="appointment"
    [cdkListboxCompareWith]="compareDate"
    (cdkListboxValueChange)="appointment = $event.value"
    aria-labelledby="example-appointment-label"
    class="example-listbox">
  <li *ngFor="let time of slots"
      [cdkOption]="time"
      class="example-option">
    {{formatTime(time)}}
  </li>
```

Benefits:

Using a ready-to-use, safe, tested and trustworthy solution with WAI ARIA standards directly from the creators of Angular saves developers a significant amount of time.

Improved stack traces

Dev Experience

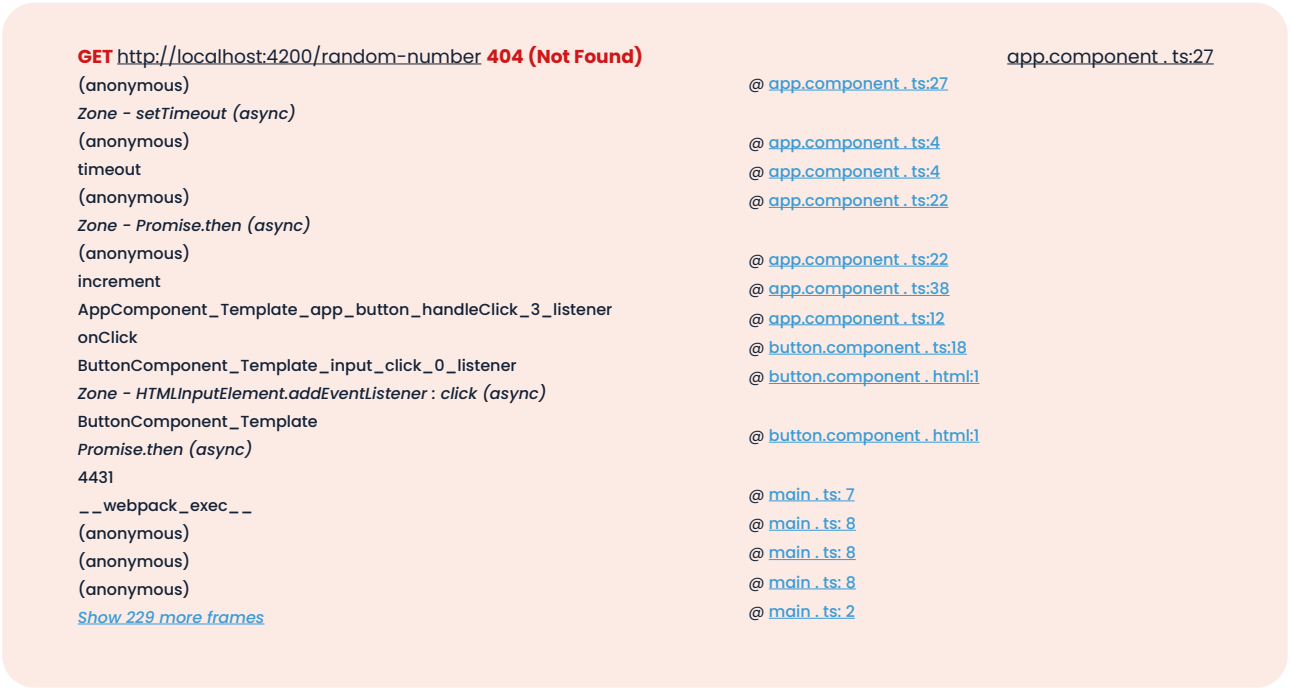
Challenge:

Up until now, the stack traces presented in CLI and browser devtools were quite obscured by external functions (i.e. from webpack or node_modules). It was hard to investigate the execution order when the trace included many lines from outside the code written by the programmer.

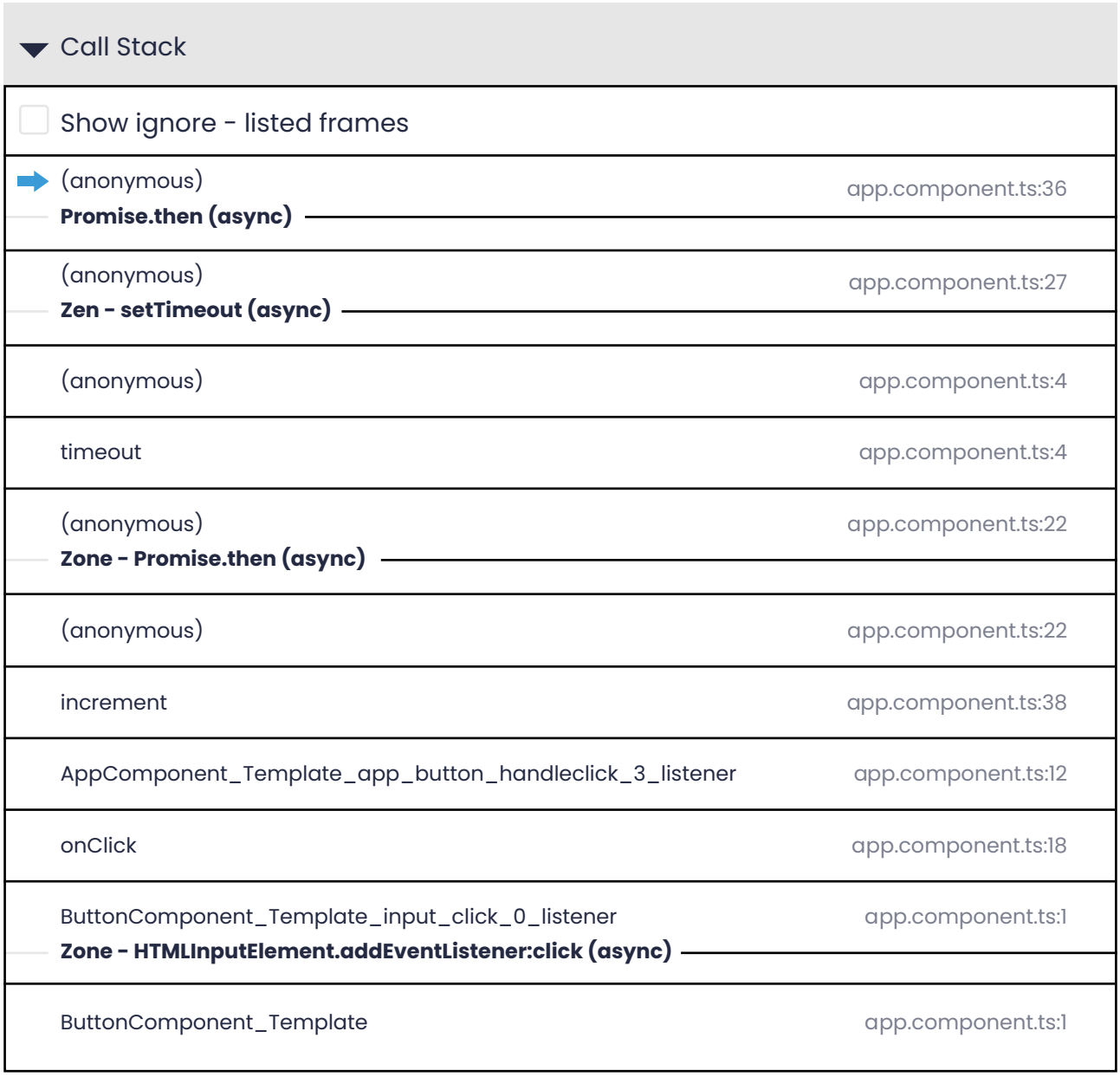
Solution:

Improvements in this area are possible thanks to the cooperation of the Angular Team and the Chrome team. The goal was to mark scripts as "external" and thus exclude them in the development tool functions(e.g. stack traces). Starting with Angular 14.1, the contents of the node_modules and webpack folders are marked in this way. It is worth mentioning that this mechanism is available to all developers, so authors of other frameworks can use it as well.

The result is a stack trace that omits scripts that are probably not in the developer's area of interest when an error is shown in the console. Also, the code that belongs to excluded scripts is skipped when you're debugging and iterating over subsequent instructions in the code.



The second new feature worth mentioning is stack trace linking for asynchronous operations. The code executed following the completion of an asynchronous action can now be properly linked with the code initiating this action (e.g. a click and call to the server). This is possible thanks to the introduction of the so-called Async Stack Tagging API mechanism in Chrome.



Benefits:

Both sync and async traces have readable form and provide more information to the programmer. Therefore these improvements:

- **Boost developer experience**
- **Ease the debugging process**

Auto-imports in language service

Efficiency | Dev Experience

Challenge:

With Standalone API, we frequently need to add extra imports to the Component metadata, because the only components, directives and pipes available in the template are ones we explicitly imported.

Solution:

The Angular Language Service is a tool utilized by all code editors to enhance errors, hints and navigation between Angular templates. The new DX-related improvement, a part of language service, allows us to automatically import components whose selectors were used in another component's template. This applies to both standalone and module-based components.

```
import { Component, NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

@Component ({
      selector: 'app-root'
      template: '<app-foo></app-foo>',
      styleUrls: ['./app
})
export class AppComp
      title = 'tour - of - h
}
```

Quick Fix...

Import FooModule from './foo/foo.module' on AppModule

Extract...

Extract to readonly field in class "AppComponent"

Benefits:

This tool is utilized by IDEs, and it significantly simplifies and speeds up the import of components, directives and pipes using keyboard shortcuts, enabling the programmer to focus on other tasks.

Typescript/Node.js support

Angular 14.2 added support for TypeSript 4.8, Node.js v18 and Angular 15.1 added support for Typescript 4.9. According to the official documentation, it includes, among other things, a number of performance improvements that can reduce build and hot reload times of TypeScript projects by up to 40%. Let's take a look at other new features worth mentioning.

The satisfies Operator (available in 4.9)

Dev Experience

Challenge:

When we define an expression, TypeScript infers the most general type for it:

```
const person = { name: 'John' };
// typeof person: { name: string }
```

In the example above, the typeof "name" property is not narrowed to const literal "John" but generalized to any string value. In some cases, we would like to ensure that the expression matches a certain type while also ensuring the most specific type at the same time.

Solution:

TypeScript version 4.9 introduced a new operator called "satisfies." It validates whether the type of an expression matches a type without changing the resulting type of that expression.

```
type Colors = 'red' | 'green' | 'blue';
// Ensure that we have exactly the keys from 'Colors'.
const myColors = {
  red: 'yes',
  green: false,
  blue: 'maybe',
  orange: false,
  // error - "orange" is not part of "Colors" union type
} satisfies Record<Colors, unknown>;
/**
 * typeof myColors is not Record<Colors, unknown>
 *  but {"red": string, "green": boolean, "blue": "string"}
 *  All the information about the 'red', 'green', and 'blue'
 *  properties are retained.
 */
const isGreen: boolean = myColors.green;
//typeof myColors.green is not unknown despite "satisfies" type check
```

This operator can be combined with "as const" to infer type from constant values and check them against a broader type at the same time:

```
  const friends = [
 { name: 'John' },
 { name: 'Paul' },
] as const satisfies readonly { name: string }[];
```

In the example above, expression is type-checked with the satisfies operator, but the inherited type for friends const comes directly from the value.

Satisfies is somewhat similar to "as" operator. The difference is that with "as," the programmer is responsible for the type safety and with "satisfies", TypeScript validates the type we assert automatically.

```
type Red = 'red';
const x: Red = 'blue' as Red;
// everything ok
const y: Red = 'blue' satisfies Red;
// error: Type '"blue"' does not satisfy the expected type '"red"'
```

Benefits:

The new operator improves the TypeScript developer experience as it enforces type safety without the loss of information (i.e. via type widening). By replacing the "as" operator with "satisfies," we also reduce the reliance on untrustworthy type assertions. The code readability is also improved in some cases.

Checks For Equality on NaN (available in 4.9)

Dev Experience

Challenge:

In IEEE-754 floats standard, supported by JavaScript and TypeScript, nothing is ever equal to NaN, the"not a number" value. However, it is a common mistake to check it with someValue === NaN, instead of using the built-in Number.isNaN function.

Solution:

Typescript no longer allows direct comparisons against NaN and suggests Number.isNaN instead:

```
function validate(someValue: number) {
  return someValue !== NaN;
  //     ~~~~~~~~~~~~~~~~~
  // error: This condition will always return 'true'.
  //        Did you mean '!Number.isNaN(someValue)'?
}
```

Benefits:

Thanks to the new restriction, we avoid an obvious error and are clearly informed about an error.

Esbuild

Non-destructive Hydration

Signals

Angular v16

release date: **05.2023**

Angular version 16 brings substantial advancements, focusing on creating a more modern and efficient framework. The introduction of signals marks the beginning of a refined change detection mechanism, while non-destructive hydration sets the stage for advanced hydration scenarios crucial for public web platforms.

Enhancements in development ease are achieved through mandatory inputs, routing parameter bindings and the new DestroyRef. Furthermore, the Angular CLI now supports creating applications with standalone components, and the experimental esbuild-based builder offers a substantial speed boost in application building.

Short list of features:	Dev Experience	Efficiency	Performance	UX
Signals library (developer preview)	✅	☑	☑	
SSR Hydration (developer preview)			☑	☑
Vite-powered dev server	✅		☑	
Vite-powered dev server	✅			
Input transform function	✅			
Router data input bindings	✅	☑		
Injectable DestroyRef and takeUntilDestroyed	✅			
Self-closing tags	✅			
runInInjectionContext	✅			
Standalone API CLI support	✅	☑		
Typescript/Node.js support	✅			
ECMAScript decorators (available in 5.0)	✅			
Extending multiple TS configuration files	✅			
All enums as Union enums (available in 5.0)	✅			
Unrelated Types for Getters and Setters	✅			

Signals library (developer preview)

Performance | Dev Experience | Efficiency

Challenge:

Angular applications can face performance issues due to inefficient change detection mechanisms, resulting in an excessive number of computations to determine what has changed in the application state. Additionally, developers might struggle with a more complex mental model for understanding reactivity, data flow, RxJS and dependencies within the application, making it harder to write, maintain, and optimize their code efficiently.

Solution:

In Angular 16, we received a developer preview of a brand new library containing a primitive that represents a reactive value–the Angular signals library.

Angular's signals library introduces a way to define reactive values and establish explicit dependencies between them within an application. Unlike RxJS, observables that push changes through a stream of values, signals allow for a more direct and straightforward way to declare reactive state, compute derived values and handle side-effects, offering a different approach to managing reactivity in Angular applications.

A signal in Angular is essentially a wrapper around a value, which notifies interested consumers (e.g., components, functions) when that value changes. Signals can hold any type of value, ranging from simple primitives like numbers and strings, to more complex data structures such as objects or arrays.

Here are a few reasons behind Angular signals implementation:

- Angular can keep track of which signals are read in the view. This lets you know which components need to be refreshed due to a change in state.
- The ability to synchronously read a value that is always available.
- Reading a value does not trigger side effects.
- No glitches that would cause inconsistencies in reading the state.
- Automatic and dynamic dependency tracking, and thus no explicit subscriptions. That frees us from having to manage them to avoid memory leaks.
- The possibility to use signals outside of components, which works well with the Dependency Injection.
- The ability to write code in a declarative way.

This is how we can define signals and express dependencies between them:

```
@Component({…})
export class App {
  firstName = signal('Ash');
  lastName = signal('Ketchum');
  fullName = computed(() => `${this.firstName()} ${this.lastName()}`);
  setName(newName: string): void {
    this.firstName.set(newName);
  }
}
```

'firstName' and 'lastName' are writable signals, meaning they provide an API for updating their values directly. 'fullName' is a readonly signal that depends on the other signals and is recalculated every time any dependent signal changes its value.

In many cases, it is useful to define a side effect. This is a call to code that changes state outside its local context, such as sending an http request or synchronizing two independent data models. To create an effect we can use "effect" function:

```
effect(() => {
  console.log(`The current value of my signal is: ${mySignal()}`);
});
```

The Angular Team also provides a bridge between signals and RxJS, placed in the @angular/core/rxjs-interop library. It is possible to convert a signal to observable and an observable to a signal.

```
import { toObservable, toSignal } from '@angular/core/rxjs-interop';
@Component({...})
export class App {
  firstName = signal('Ash');
  firstName$ = toObservable(this.firstName);

  lastName$ = of('Ketchum');
  lastName = toSignal(this.lastName$, '');
}
```

The real revolution will come when the Angular Team releases signal-components. These components will work without zoneJS and will have their own change detection strategy based solely on signals. It will be possible to update DOM elements with surgical precision, without unnecessary traversal of the component tree.

It's also worth mentioning that external libraries like ngrx also implement support for signals. The NgRx store will have its signal-based counterpart called SignalStore.

```
import { signalState } from '@ngrx/signals';
const state = signalState({
  user: {
    firstName: 'John',
    lastName: 'Smith',
  },
  foo: 'bar',
  numbers: [1, 2, 3],
});
console.log(state()); // { user: { firstName: 'John', lastName: 'Smith' }, foo: 'bar' }
console.log(state.user()); // { firstName: 'John', lastName: 'Smith' }
console.log(state.user.firstName()); // 'John'
```

Benefits:

The introduction of a new experimental reactive primitive is a promise of a simplified mental model for reactivity in Angular. Its goals are to lower the learning curve, reduce the number of errors and make the whole development process intuitive and straightforward. With the future arrival of signal-based components, significant performance improvements are expected.

Expert Opinion:

I think signals will have a dramatic impact on many sides of the framework from performance to bundle size improvements. This will be especially visible when signals are integrated into the change detection mechanism. It will make the process much more efficient and performant because, unlike the current zone.js-based implementation, signals will allow Angular to know in which exact component a change occurred and update only the affected components. Besides that, the change detection mechanism will become much simpler and unified, because we will have only one predictable change detection strategy, which will also have a positive impact on the learning curve. Finally, we will be able to reduce bundle size by removing the zone.js dependency, which will not be needed for the new change detection mechanism. And this is just one of the examples. I think that we are currently seeing only the tip of the iceberg.

~ Dmytro Mezhenskyi
Google Developer Expert

The Angular signal feature is currently in its Developer Preview stage. Once this feature becomes stable, it is poised to introduce an entirely new approach to writing Angular applications. The elimination of zone.js will free developers from concerns related to change detection cycles and performance issues, allowing them to concentrate more on their business logic. This is exemplified by the transition from familiar [(ngModel)] binding syntax to the new signal and effect syntax, which requires developers to adapt and update their projects. Despite these changes, I firmly believe that this innovation holds great promise in advancing the framework to the next level.

~ Balram Chavan
Google Developer Expert

SSR Hydration (developer preview)

Performance **UX**

Challenge:

By default, Angular renders applications only in a browser. If any web crawler tries to index the page, it receives an almost empty HTML file and is unable to navigate and find searchable content. This is one of the reasons why we might want to implement Server-Side Rendering, the process of rendering our site on the server side and returning the whole HTML structure.

A server-side rendered HTML is displayed in the browser, while the Angular app bootstraps in the background, reusing the information available in server-generated HTML. The HTML then destroys DOM and replaces it with with a newly rendered and fully interactive Angular app.

Destroying and replacing an application's DOM may result in a visible UI flicker and has a negative impact on Core Web Vitals, such as FID, LCP and CLS.

Solution:

Angular introduced hydration, a new feature currently available for developer preview. We can enable it as follows:

```
import { bootstrapApplication, provideClientHydration, } from '@angular/
platform-browser';
...

bootstrapApplication(RootCmp, {
  providers: [provideClientHydration()]
});
```

It enables reusing server-side rendered HTML in the browser without destroying it. Angular matches the existing DOM elements with application structure at runtime. This results in a performance improvement in Core Web Vitals and a better SEO performance.

There is also an option to skip hydration for some components, or rather component trees, if they're not compatible with hydration. For example, manipulating DOM directly with browser APIs. Use one of the following options to enable hydration:

#1

```
<test-component ngSkipHydration />
```

#2

```
@Component({
 ...
 host: {ngSkipHydration: 'true'},
})
class TestComponent {}
```

There are also some other improvements added for SSR. These include new standalone-friendly utility functions to provide SSR capabilities to an application (provideServerRendering()) and HTTP transfer state (withTransferCache()).

If your Angular application uses SSR, you should definitely check this feature out. More details are available in the official documentation: https://angular.io/guide/hydration

Benefits:

Full hydration brings:

- **Better UX:** faster loading and interactivity, no more flickering
- **Enhanced SEO:** in comparison to destructive hydration

Expert Opinion:

The Angular team has been deeply committed to the ongoing refinement of Server-Side Rendering (SSR) capabilities. Over time, they've introduced a series of enhancements to SSR applications, aiming to provide a smoother and more efficient experience for both developers and end-users. The team's focus has been on achieving the goal of a fully hydrated application, which means that the application is not just server-rendered but also preloaded with the necessary data and components, offering a seamless and responsive user experience.

This concerted effort to improve SSR reflects the Angular team's dedication to staying at the forefront of web development technology. It's an exciting journey of innovation, and it's intriguing to anticipate how these ongoing efforts will shape the landscape of Angular in the future. The advancements in SSR hold the promise of further enhancing the performance, SEO-friendliness, and overall user experience of Angular applications, making them even more competitive and appealing in the dynamic world of web development.

~ Balram Chavan
Google Developer Expert

Vite-powered dev server

Performance **Dev Experience**

Challenge:

Previous experimental features related to the Angular building systems affected only the building process and not the development process, especially in the case of the development server with hot-reload.

Solution:

The esbuild-based build system for Angular CLI entered developer preview in version 16, with the goal of significantly speeding up the build process. Early tests showed improvements of over 72% in cold production builds. With this update, Vite is utilized as the development server, while esbuild enhances both development and production builds for faster performance. However, it's important to note that Angular CLI exclusively uses Vite as a development server. This is due to the Angular compiler's need to maintain a dependency graph between components, requiring a different compilation model.

You can enable esbuild and Vite by updating your angular.json file:

```
...
"architect": {
  "build": {
    "builder": "@angular-devkit/build-angular:browser-esbuild",
     ...
```

Benefits:

Improvements in the Angular building system bring:

- **An improved developer experience**
- Reduced time and costin heavy CI/CD processes

Please note that these changes are still in experimental mode.

Required inputs

Dev Experience

Challenge:

Until Angular 16, it was not possible to mark inputs as required, but there was a common workaround for this using a component selector:

```
@Component({
  selector: 'app-test-component[title]', // note attribute selector here
  template: '{{ title }}',
})
export class TestComponent {
  @Input()
  title!: string;
}
```

Unfortunately, this was far from ideal. The first thing was that we polluted the component selector. We always had to put all required input names to the selector, which was especially problematic during refactors. It also resulted in a malfunction of the auto-importing mechanisms in IDEs. And second, forgetting to provide the value for an input marked this way meant the error was not very accurate, as there was no match at all for such an "incomplete" selector. You can see this here:

ERROR

src/app/app.component.html:1:1 - error NG8001: 'app-test-component' is not a known element:

1.If 'app-test-component' is an Angular component, then verify that it is part of this module.

2. If 'app-test-component' is a Web Component then add 'CUSTOM_ELEMENTS_SCHEMA" to the @NgModule.schemas' of this component to suppress this message.

1.<app-test-component></app-test-component>

~~~~~~~~~~~~~~~~

src/app/app.component.ts:5:16

5 templateURL: './app.component.html',

~~~~~~~~~~~~~~~~

Error occurs in the template of component AppComponent.

Solution:

The new feature allows us to explicitly mark the input as required, either in the @Input decorator:

```
@Input({required: true}) title!: string;
```

or the @Component decorator inputs array:

```
@Component({
 ...
 inputs: [
  {name: 'title', required: true}
 ]
})
```

However, the new solution has two serious drawbacks. One of them is that it only works in AOT compilation and not in JIT. The other drawback is that this feature still suffers from the strictPropertyInitialization TypeScript compilation flag, which is enabled by default in Angular. TypeScript will raise an error for this property since it was automatically declared as non-nullable but not initialized in the constructor or inline.

ERROR

src/app/test-component/test-component.component.ts:9:3 - error TS2564: Property 'title' has no initializer and is not definitely assigned in the constructor.

9 title: string;
~ ~ ~

This means that you still need to disable this check here e.g. by explicitly marking this property with a non-null assertion operator, even though it has to be provided in the consumer template:

```
@Input({required: true}) title!: string;
```

Benefits:

The existing workarounds are replaced by a new solution with a very simple and straightforward syntax.

Input transform function

Dev Experience

Challenge:

Angular interprets all static HTML attributes as strings. For example, if boolean attributes are considered true when they are present on a DOM node,i.e. “disabled,” Angular, by default, interprets them as strings.

Solution:

Since the release of Angular 16.1, we can provide an optional transform function for the @Input decorator. This allows us to simplify the process of transforming values and discontinue the use of setters and getters that have been used for this purpose so far.

Transform function:

```
function toNumber(value: string | number): number {
  return isNumber(value) ? value : parseInt(value);
}


@Component({
  selector: ‘app-foo’,
  template: ``,
  standalone: true,
})
export class FooComponent {
  @Input({ transform: toNumber }) width: number;
}
```

Usage in template:

```
<app-foo width=”100” />
<app-foo [width]=”100” />
```

Additionally, Angular offers us two built-in transform functions that we can use:

```
import { booleanAttribute, numberAttribute } from '@angular/core';

// Transforms a value (typically a string) to a boolean.
@Input({ transform: booleanAttribute }) status!: boolean;

// Transforms a value (typically a string) to a number
@Input({ transform: numberAttribute }) id!: number;
```

Benefits:

Transform functions are pure functions, so they improve readability, reusability and testability.

Router data input bindings

Dev Experience **Efficiency**

Challenge:

Obtaining router-related data inside components is not very comfortable. We have to inject an ActivatedRoute token, subscribe to its properties, manage these subscriptions to ensure no memory leaks, and so on.

Solution:

Angular version 16 brings another interesting feature related to component inputs: the ability to bind them directly to the current route variables, such as path params and query params. This eliminates the need to inject ActivatedRoute into the component in order to use router data.

With this feature, data is being bound only to the routable components present in the routing configuration, and inputs of children components used in the templates of routable routes are not affected. Route data is matched with inputs by name,or input alias name if present. This is done so there is more than one piece of data that can potentially be put as the input value.

The list below shows the precedence of data being bound to input property if the names are the same:

1. Resolved route data
2. Static data
3. Optional/matrix params
4. Path params
5. Query params

There is much more complexity if we consider “inheriting” data from places like the parent route configuration and the parent component presence.

To enable this feature, we have to use a dedicated function called with ComponentInputBinding.

Example route definition:

```
bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(
      [
        {
          path: 'example/:id',
          data: {
            bar: true,
          },
          resolve: { baz: () => 'example string' },
          loadComponent: () => import('./app/todo/todo.component'),
        },
      ],
      withComponentInputBinding()
    ),
  ],
});
```

Exinput binding (input names must match route param names or their aliases):

```
@Component({
  selector: 'example-cmp',
  standalone: true,
  template: '',
})
export default class ExampleComponent {
  @Input() id!: string;
  @Input() bar!: boolean;
  @Input() baz!: string;
}
```

Benefits:

Router data input bindings simplifies the task of transmitting router data to the routed components, consequently diminishing the requirement for repetitive code and dealing with extra RxJS subscriptions.

Injectable DestroyRef and takeUntilDestroyed

Dev Experience

Challenge:

The cleanup process was a common problem Angular programmers had to deal with, especially when we wanted to implement a component/directive lifecycle with long RxJS subscriptions. The standard way to do this was to use an OnDestroy lifecycle hook to complete all relevant streams and execute other cleanup actions. However, this caused some overhead and unwanted boilerplate code. Another way to achieve this implementation was to use third-party solutions like @ngneat/until-destroy, but as opposed to built-in ones, they can introduce compatibility issues and less seamless integration.

Solution:

In the new Angular version, we receive a brand new injectable for registering cleanup callback functions that will be executed when a corresponding injector is destroyed.

```
const destroyRef = inject(DestroyRef);


// register a destroy callback
const unregisterFn = destroyRef.onDestroy(() => doSomethingOnDestroy());


// stop the destroy callback from executing if needed
unregisterFn();
```

Such an injectable is allowed to introduce yet another feature: the takeUntilDestroyed operator. This operator uses the DestroyRef injectable underneath and is a safe way to avoid memory leaks in Angular apps.

```
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';


@Component({
  ...
})
export default class MyComponent {
  constructor() {
    interval(3000).pipe(takeUntilDestroyed()).subscribe(
      value => console.log(value)
    );
  }
}
```

Benefits:

The new operator is a convenient way to avoid memory leaks without dealing with OnDestroy lifecycle hooks and subscription references. DestroyRef also allows you to deal with other cleanup tasks in any way that fits your needs.

Self-closing tags

Dev Experience

Challenge:

When we don't use a content projection mechanism, we are not placing any content between tags of the Angular components we use in templates. HTML specification contains many self-closing tags for nodes without inner HTML (i.e. <img />, <input />,
), but in Angular, we always had to repeat the same name in opening and closing tags, as follows:

```
<app-your-component-name [foo]="bar">
</app-your-component-name>
```

Solution:

Self-closing tags is a highly requested feature that arrived in Angular version 16. You can use self-closing tags for your components.

This feature is optional and backward compatible, so you can continue to use the standard approach, especially when using content projection.

```
<app-your-component-name [foo]="bar"/>
```

Benefits:

Angular template syntax gets easier and more readable.

runInInjectionContext

Dev Experience

Challenge:

There was no convenient way to inject dependencies outside of construction time, like after user interaction or conditionally after resolving some asynchronous value.

Solution:

An “inject” function is a function that Injects a token from a current injection context. We mostly use it during the construction time of classes being instantiated by the DI system. Examples of classes include components, pipes and injectable services.

There is, however, a possibility to call “inject” at any time of the life cycle if we use the “runInInjectionContext” function. This allows us to instantiate dependency on demand after a user interaction.

```
@Injectable({
  providedIn: 'root',
})
export class MyService {
  private injector = inject(EnvironmentInjector);
  someMethod(): void {
    runInInjectionContext(this.injector, () => {
      const otherService = inject(OtherService);
    });
  }
}
```

Benefits:

We get a possibility to inject dependencies at any time, as long as we have a reference to the injector. This approach can improve application performance if we combine it with lazy loading.

Standalone API CLI support

Dev Experience | Efficiency

Challenge:

There was no way to generate a new project as a standalone from the beginning. We had to manually remove AppModule and rewrite the bootstrapping process to a standalone version with a bootstrapApplication function call. All components, directives and pipes in such a project were generated using built-in code generators in a non-standalone version by default.

Solution:

Angular version 16 introduced a new flag for CLI command with which we can generate a new Angular project.

```
ng new --standalone [name]
```

The project output with such a flag is simpler compared to the old ngModule-based version. Generators for components, directives and pipes in this project will also create standalone versions by default.

Benefits:

Kickoff of a new standalone project is made very easy, increasing overall project efficiency.

TS Typescript/Node.js support

Angular v16 brings support for TypeScript 5.0, and Angular v16.1 for Typescript 5.1. This results in further computation time, memory usage and package size optimizations, as well as many feature enhancements.

ECMAScript decorators (available in 5.0)

Dev Experience

Challenge:

Typescript supported custom experimental decorators for years. They're familiar to every Angular Developer, as we use them to define things like modules, components, injectable services, pipes, directives, inputs, outputs and injections in constructor arguments. They were implemented long before TC39 decided on a decorator's standard, and now there is a mismatch between the old TypeScript and the new ECMAScript decorator. Such mismatch can cause compatibility and integration issues for developers.

Solution:

TypeScript 5.0 supports the new official ECMAScript decorators. This results in better typing, like the ClassMethodDecoratorContext type which impacts meta information like method name, access modifier and the extra function addInitializer

This is an example of a fully typed ECMAScript decorator in Typescript 5.0:

```
function loggedMethod<This, Args extends any[], Return>(
  target: (this: This, ...args: Args) => Return,
  context: ClassMethodDecoratorContext<
    This,
    (this: This, ...args: Args) => Return
  >
) {
  const methodName = String(context.name);
  function replacementMethod(this: This, ...args: Args): Return {
    console.log(`LOG: Entering method '${methodName}'.`);
    const result = target.call(this, ...args);
    console.log(`LOG: Exiting method '${methodName}'.`);
    return result;
  }
  return replacementMethod;
}
```

There is one important difference between the old and new implementation - decorators used in the constructor parameters will not work, as the standard doesn't support the following code:

```
constructor(@Optional() public myService: MyService) {}
```

So we need to use the inject function instead:

```
myService = inject(MyService, { optional: true });
```

Benefits:

We're up to date with the official ECMAScript standard with improved typings. We no longer need to rely on the TypeScript custom implementation without its JavaScript counterpart. Instead, we can use a solution that will be directly supported by all modern browsers.

Extending multiple TS configuration files

Dev Experience

Challenge:

Typescript supported custom experimental decorators for years. They're familiar to every It is uncomfortable to compose TypeScript configuration files with a single file extension, especially in a mono-repository where multiple applications or libraries are involved. This process requires us to create a chains of extensions as follows:

```
  // tsconfig1.json
{
 “compilerOptions”: {
    “strictNullChecks”: true
 }
}
// tsconfig2.json
{
“extends”: “./tsconfig1.json”,
 “compilerOptions”: {
    “noImplicitAny”: true
 }
}
// tsconfig.json
{
 “extends”:”./tsconfig2.json”,
 “files”: [“./index.ts”]
}
```

Solution:

Typescript 5.0 allows us to extend multiple tsconfig files.

```
// tsconfig1.json
{
 “compilerOptions”: {
    “strictNullChecks”: true
 }
}
// tsconfig2.json
{
 “compilerOptions”: {
    “noImplicitAny”: true
 }
}
// tsconfig.json
{
 "extends": ["./tsconfig1.json", "./tsconfig2.json"],
 "files": ["./index.ts"]
}
```

Benefits:

Extending multiple TS configuration files can simplify the configuration of your Angular workspace by streamlining the management of compiler options across multiple projects.

All enums as Union enums (available in 5.0)

Dev Experience

Challenge:

Before Typescript 5.0, the language required all constant enum members to be string/numeric literals, like 1,2, 300, “foo” or “bar,” for an enum to make it a union enum. In such a case, union enum members become types as well. That means that certain members can only have the value of an enum member. It also means that enum types themselves effectively become a union of each enum member.

Solution:

In TypeScript 5.0, all enums, even the ones with computed members, are converted into union enums. All members of all enums can be referenced as types and all enums can be successfully narrowed.

The following example with computed enum members is correctly interpreted in TypeScript 5.0 and above:

```
const prefix = 'data';

const enum Routes {
  Parts = `${prefix}/parts`,
  Invoices = `${prefix}/invoices`,
}
```

Benefits:

This feature brings more flexibility when defining types and more intelligent type inference by the TypeScript transpiler.

Unrelated Types for Getters and Setters

Dev Experience

Challenge:

In a getter/setter pair, the get type had to be a subtype of the set type, which potentially limited the ability to return more specific or transformed data types through getters.

Solution:

Since the release of TypeScript version 5.1, we can specify two completely unrelated types for a getter/setter pair.

```
interface CSSStyleRule {
  // ...
  /** Always reads as a `CSSStyleDeclaration` */
  get style(): CSSStyleDeclaration;
  /** Can only write a `string` here. */
  set style(newValue: string);
  // ...
}
```

Benefits:

The developer gains more flexibility by being able to cover edge cases that require different types.

Esbuild

Signals

New Control Flow

Deferred Loading

Angular v17

release date: **11.2023**

Angular 17 marks a pivotal update in the framework's evolution, introducing streamlined features and setting a robust foundation for future advancements in signals and template syntax. In this version, Angular also continues its journey towards enhancing performance and improving SSR.

Short list of features:	Dev Experience	Efficiency	Performance	UX
Signals library (stable)	✅	✅	✅	
Signal inputs	✅	✅	✅	
New control flow (Developer preview)	✅	✅	✅	
Deferred loading (developer preview)	✅		✅	✅
Inputs Binding with NgComponentOutlet	✅	✅		
Animation lazy loading			✅	✅
View Transitions				✅
Esbuild + Vite (stable)	✅		✅	
SSR Hydration (stable)			✅	✅
CLI improvements	✅	✅		
Devtools Dependency Graph	✅			
Rebranding and introduction of angular.dev	✅			
Typescript/Node.js support	✅			
Explicit Resource Management (available in 5.2)	✅			
Named and Anonymous Tuple Elements (available in 5.2)	✅			
Copying Array Methods (available in 5.2)	✅			

Signals library (stable)

Performance | Dev Experience | Efficiency

Note:

Signals are introduced and described in the “Angular 16” chapter. Here we will focus on the changes that have occurred since then.

Challenge:

The Angular signals library, in its developer preview phase, encountered some modifications and adjustments due to feedback from the Angular community. Various Angular developers raised doubts related to mutability and the change notification system in the initial proposal.

Solution:

In Angular 17, signals became stable, so we could confidently use them in commercial applications. At the end of the developer preview, we received two significant changes.

The first change was the removal of the ‘mutate’ function that was previously used to mutate the value stored by the signal. The mutate function skipped comparing values because its purpose was to modify the value of the signal by design. Now, it is recommended to use only the update function.

The second change had to do with the implementation of the default function for comparing signal values.

The defaultEquals function implementation in v16 considered any two objects to be different. so that even if we returned references to the same object, all dependent signals were notified of the change.

```
// Angular 16 version
export function defaultEquals<T>(a: T, b: T) {
  return (a === null || typeof a !== 'object') && Object.is(a, b);
}
// Angular 17 version
export function defaultEquals<T>(a: T, b: T) {
  return Object.is(a, b);
}
```

New implementation of the defaultEquals function relies solely on Object.is(). As a result, if an object is mutated by using the update() function without changing the reference, other dependent signals will not be notified of the change. To get a signal to notify you about the change, create a new reference of the object with the updated properties (e.g., by using the spread operator) or provide your own implementation of the equals function and then specify it in the signal options.

The upgrade of the API to the stable version does not include the "effect" function, as there is still ongoing discussion about its semantics and potential improvements.

A significant change is that if all components in the application use the OnPush change detection strategy and a new signal value triggers Change Detection in a specific component template, only this component is marked as dirty and its ancestors are notified about the change. This “local” change detection can majorly affect application performance.

It is important to note that Angular 17 still does not include signal-based component implementation, even in the developer preview mode. When this implementation arrives, it is supposed to revolutionize the change detection system, because components will not use zoneJS and DOM recalculation will only happen after the values related to the signal view are changed.

Benefits:

The stabilization of the Angular signals library in Angular 17 lays solid groundwork for future advancements, particularly in moving away from zone.js and towards signal-based components. This shift is a step towards change detection system transformation, which promises enhanced performance and efficiency in application development.

Expert Opinion:

At this juncture, I hold no particularly strong opinion on the signals library. My perpetual quest revolves around enhancing both user experience (UX) and developer experience (DX). Nonetheless, I avoid excessive "hype" concerning the introduction of signals at the framework's core, opting instead for a rational assessment of the potential opportunities signals may herald in the future. A substantial benefit would arise if we could seamlessly transition away from zone.js, aligning with our overarching objective of establishing a viable route toward creating fully zoneless applications (as one of the goals).

~ Artur Androsovych
Angular Expert

Signals are becoming a mainstream technique, being introduced in the most popular frontend frameworks, the reason being that it is the perfect technique for fine-grained synchronous reactivity. For a long time, we used to handle all states in an asynchronous reactive approach using RxJS, whereas state management itself is a synchronous task, such that RxJS is not the right tool by default. This overcomplicated a few use cases and is the reason for unexpected glitching. This is a lot simpler with signals. Additionally, this fine-grained reactive primitive can, and will, be used for fine-grained and zoneless change detection in the future, when signal-based components land in Angular. This is no small change, and will rather change the way we develop reactive Angular applications completely.

~ Stefan Haas
Nx Champion

Signal inputs

Performance **Dev Experience** **Efficiency**

Challenge:

The current implementation of component inputs is not very reactive. In order to listen for changes, we use component lifecycle hooks. That means that if we want the input values to interact with the Signal API, we need to handle the logic manually.

Solution:

Angular version 17.1 enables signal inputs for components. This means we no longer use the decorator. Instead, a dedicated function defines the input:

```
  @Component({
  selector: 'my-cmp',
  standalone: true,
  template: `{{ name() }}`,
 })
export class MyComponent {
  name = input<string>();
}
```

This mechanism allows you to simplify the reactivity of the component, as it allows you to define computed signals and side effects for input value changes. There is no longer a need to use ngOnChanges lifecycle hook.

```
export class MyComponent {
  name = input<string>();
  helloMessage = computed(() => 'Hello ' + name());

  constructor() {
   effect(() => {
    console.log('logging: ' + this.name());
   })
  }
 }
```

The Signal input API has extra capabilities compared to its standard counterpart, such as providing default values, marking input as required, setting aliases, and various transform functions.

```
export class MyComponent {
  // required input
  name = input.required<string>();

  // input with default value
  color = input('red');

  // input with alias
  disabled = input<boolean>(false, { alias: 'isDisabled' });

  // input with transform function
 age = input.required({ transform: numberAttribute });
}
```

Benefits:

Signal inputs mark a significant step towards the deeper integration of signals into the ecosystem and transforms the way we deal with component input changes.

This feature significantly enhances the developer experience and code clarity by streamlining the management of component inputs and facilitating more intuitive coding practices.

New control flow (Developer preview)

Performance **Dev Experience** **Efficiency**

Challenge:

The standard control flow (if/else, switch/case, loop in component templates), based on structural directives, has a few disadvantages. For example, the boilerplate, which is large compared to other frontend frameworks, is necessary even for a single if-else statement.

```
<div *ngIf="condition as value; else elseBlock">{{value}}</div>
<ng-template #elseBlock>Content to render when value is null.</ng-template>
```

The second issue with the standard control flow is the current directives implementation, which is strongly based on the current change detection system. Support for both zone-based and signal-based approaches in directives would greatly complicate the code with no possibility of tree shaking it.

Solution:

The introduction of a new control flow to templates is one of the biggest changes in Angular v17. It is the first step to moving away from built-in structured directives, whose current design would not work in zoneless signal-based applications.

The new syntax is based on a structure called a block. Its appearance significantly differs from what we have seen in templates. We start each block with an "@" prefix and then use a syntax very similar to the one we know from JavaScript.

The change affects the three most commonly used structured directives: ngIf, ngSwitch and ngFor. At this point, there is no planned implementation of custom blocks.

An if/else statement:

```
@if (time < 12) {
  Good morning!
} @else if (time < 17) {
  Good afternoon!
} @else {
  Good evening!
}
```

A switch/case statement:

```
@switch (fruit) {
  @case 'apple' {
    <apple-cmp />
  }
  @case 'banana' {
    <banana-cmp />
  }
  @default {
    <unknown-fruit-cmp />
  }
}
```

A loop statement:

```
<ul>
  @for (item of items; track item.id) {
    <li>{{ item.name }}</li>
  } @empty {
    <li>No items...</li>
  }
</ul>
```

The loop has received the most valuable improvements out of all the structures:

- The @empty block is available, which allows us to display the content when the list we iterate over is empty.
- It is no longer necessary to create a special trackBy function to pass it as an argument. Instead, we only need to specify the unique key of the object to be tracked.
- The @for block requires the use of the track function, which significantly optimizes the process of rendering the list and managing changes without the need for developer interference.

Since we have two ways of using the control flow, you may ask : what will happen to the directives we are familiar with? In version 17, they remain unchanged. But with the arrival of future versions and the exit of the new control flow from the developer preview, they will go into a deprecated state.

However, there is no need to worry about refactoring. The Angular Team has created a scheme that automatically migrates the control flow to the new syntax. In most cases, the scheme should handle the transition to the new syntax without a programmer's interference. To switch to the new syntax, all you need to do is execute the following command:

```
ng generate @angular/core:control-flow
```

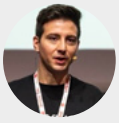
Benefits:

The new control flow in Angular offers a more succinct and readable syntax, significantly reducing boilerplate in templates. It ensures smoother integration with future signal-based change detection systems, enhancing application performance. Additionally, it lays the groundwork for advanced features such as deferred loading, contributing to optimized load times and a better overall user experience.

According to public benchmarks, operations on loops using the new built-in control flow are up to 90% faster.

Expert Opinion:

The new control flow is fabulous. The syntax is quite similar to the JavaScript syntax, and the readability has increased. This new syntax is what we are waiting for. It's also more similar to JSX (not equal), but this can reduce the gap with React, and that can help React developers jump into the Angular environment. With this new approach, we can focus on the view's logic and not on what Angular needs to show our logic correctly; that's a fantastic improvement for the developers and reduces the entry-level in the framework.



~ Luca Del Puppo
Google Developer Expert

The built-in new control flow brings several key improvements:
• No need to import NgIf, NgFor, and more in standalone components.
• Automatic enclosure of code blocks with {}. So no need for <ng-container>.
• Ergonomic and type-checking enhancements
• Improved repeater performance
• A step toward zoneless apps
These changes simplify development, reduce code verbosity, and enhance performance, making Angular even more developer-friendly.

~ Fatima Amzil
Google Developer Expert

Deferred loading (developer preview)

Performance | Dev Experience | UX

Challenge:

A lack of lazy loading in a web app can lead to slower page load times and increased bandwidth usage, as all content is loaded at once regardless of its immediate necessity. As long as Angular has lazy-loading modules in routing, there is no simple and convenient way to lazy load individual components, even though it's achievable with dynamic import() and ngComponentOutlet.

Solution:

Angular version 17 introduces the new lazy-loading primitive called "defer," based on the syntax introduced in the new control flow. This extremely useful mechanism allows a controlled delay in the loading of a page's selected elements. That is par-

ticularly important because by using "defer," we can significantly reduce the initial bundle size, which improves the application's loading speed, especially for users with a slower internet connection.

To control when to defer a block's content, use predefined conditions: when and on. You can use them individually or in any combination, depending on when you want to load the content.

"When": defines a logical condition that will load the block's content when it receives a true value.

```
@defer (when condition) {
  <deferred-cmp />
}
```

An important note is that once the block's content has been asynchronously loaded, there is no way to undo that loading. If we want to hide the block's contents, we need to combine the **@defer** block with the **@if** block.

"On": allows us to use predefined triggers which initiate loading.

These predefined triggers are:

- **Idle** - This is the default trigger. The element will be loaded when the browser enters the idle state. The requestIdleCallback method is used to determine when the content will be loaded.
- **Interaction** - Content will be loaded upon user interaction, click, focus, touch, and upon input events, keydown, blur, etc.
- **Immediate** - Loading will occur immediately after the page has been rendered.
- **Timer(x)** - Content will be loaded after X amount of time.
- **Hover** - Content will be loaded when the user hovers the mouse over the area covered by the functionality, which could be the placeholder content or a passed-in-element reference.
- **Viewport** - Content will be loaded when the indicated element appears in the user's view. The Intersection Observer API is used to detect an element's visibility.

```
@defer (on interaction) {
  <deferred-cmp />
}
```

We also have the ability to combine conditions and triggers:

```
@defer (when cond; on interaction, timer(5s)) {
 <deferred-cmp />
}
```

It is worth noting that loading content, as in the case of **when**, is a one-time operation.

"Prefetch": There may be situations where we want to separate the process of fetching content from rendering it on the page. In such a case, we need the prefetch condition. It allows us to specify the moment, using the previously mentioned triggers, when the necessary dependencies will be downloaded. As a result, interaction with this content becomes much faster, resulting in a better UX.

```
@defer (on interaction; prefetch on idle) {
 <deferred-cmp />
}
```

We also have three very useful, optional blocks we can use inside the @defer block:

@placeholder - used to specify the content visible by default until the asynchronously loaded content is activated. Example:

```
  @defer (when condition) {
 <deferred-cmp />
}
@placeholder (minimum 2s) {
 <span>There will be deferred content.</span>
 }
```

The **"minimum"** condition allows you to specify the minimum time after which the delayed content can be loaded. In the example used above, this means that even if the condition is met immediately, the content will be swapped after 2 seconds.

@loading - the content of this block is displayed when the dependencies are being loaded. Example:

```
@defer {
  <deferred-cmp />
}
@loading (after 100ms; minimum 1s) {
  <span>Content is loading...</span>
}
```

Within this block, we can also use the minimum condition, which works the same way as within the **@placeholder**. It indicates the minimum time for which the block's content will be visible. We can also use the **"after"** condition, which indicates the amount of time it will take for the block's content to appear. If it takes less than 100ms to load, the loader will not appear. Instead, `<deferred-cmp />` will replace it immediately.

@error - represents the content rendered when the deferred loading failed for some reason. Example:

```
   @defer (timeout 1s) {
  <deferred-cmp />
}
@error {
  <p>Failed to load the deferred component</p>
  <p>Error: {{ $error.message }}</p>
}
```

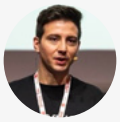
When using the defer block together with the **@error** block, it is possible to use a special **timeout** condition. This condition allows you to set a maximum loading time. If dependencies take longer than the specified time, the contents of the **@error** block will be displayed. Inside this block, the user has access to the **$error** variable, which contains information about the error that occurred during the loading process.

Benefits:

Deferred loading in Angular enhances performance by reducing initial bundle size, speeding up load times, and enabling controlled, asynchronous loading of individual components based on user interaction or other predefined conditions. This leads to a more efficient and user-friendly experience.

Expert Opinion:

Deferred loading opens a new way to reduce the bundle size of every component and introduces a new way to load only essential code needed for the current view. This feature will probably be one of the leading solutions to reduce the page size in a Server Side Render application; it helps to render only the needed parts and skip the dynamic content that isn't required on the page start-up or for the SEO.



~ Luca Del Puppo
Google Developer Expert

*As for the **deferred loading** feature, it's truly incredible! With this feature, we can lazily-load content within an Angular template. It's much more than just simple lazy loading; we can load chunks based on conditions and triggers and even prefetch in advance. This is bound to revolutionize the way we construct our Angular templates.*

~ Fatima Amzil
Google Developer Expert

Inputs Binding with NgComponentOutlet

Dev Experience **Efficiency**

Challenge:

When we used the NgComponentOutlet structural directive to create dynamic components, there was no convenient way to pass the data to them. The usual solution was to create a new injector and transfer data by a custom injection token.

Solution:

In Angular 16.2, we received a possibility to bind data to inputs of a component created with NgComponentOutlet.

```
Component({
 selector: 'my-component',
 imports: [NgComponentOutlet],
 template: `
  <ng-template
*ngComponentOutlet="dynamicComponent; inputs: dynamicComponentIn-
puts;"  />
 `
 standalone: true,
})
class FooComponent {
 readonly dynamicComponent = FooComponent;
 readonly dynamicComponentInputs  = { foo: 'Bar' }
}
```

In the child component from the example above, we have to create @Input with a 'foo' name, or alias and the data will be bound from the parent.

Benefits:

The new function simplifies the data passing process, making it more declarative. This improves code readability and developer experience.

Animation lazy loading

Performance **UX**

Challenge:

Animations are always downloaded while the application is bootstrapped, even though in most cases, animations occur while the user is interacting with an element on the page. This results in increased bundle size.

Solution:

Animation lazy loading is a new functionality in version 17 that solves this problem by introducing the ability to asynchronously load code that is associated with animations.

To start using lazy loading of animations, we need to add provideAnimationsAsync() instead of provideAnimations() to the providers of our application:

```
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
bootstrapApplication(AppComponent, {
  providers: [provideAnimationsAsync(), provideRouter(routes)],
});
```

And that's it! Now we just need to make sure that all functions from the @angular/animations module are only imported into dynamically loaded components.

Controlling imports in our application is easy, but the process can become problematic when we do it with libraries. An example of such a library is **@angular/material**, which relies heavily on **@angular/animations**, making it very likely that the module responsible for animations will be pulled into the initial bundle.

If we want to check if the animations were downloaded asynchronously, we can build our application with the **--named-chunks** flag. Then we should see **@angular/animations** and **@angular/animations/browser** in separate bundles in the **Lazy Chunk Files** section.

build with asynchronously loaded animations

Initial Chunk Files	Names	Raw Size
chunk-IETGG730.js	-	98.56 kB
main-S3BE5YUD.js	main	77.80 kB
polyfills-W2VU2Y2Q.js	polyfills	33.23 kB
styles-YD2MB7C6.css	styles	43 bytes
	Initial Total	209.63 kB

Lazy Chunk Files	Names	Raw Size
chunk-FBVTU5M2.js	browser	62.20 kB
chunk-D75ILFX5.js	-	3.53 kB
chunk-FW06XWLR.js	animated-component	1.25 kB

build with standard loaded animations

Initial Chunk Files	Names	Raw Size
main-C6GNJMLR.js	main	133.40 kB
chunk-5AHZPM5V.js	-	102.12 kB
polyfills-W2VU2Y2Q.js	polyfills	33.23 kB
styles-YD2MB7C6.css	styles	43 bytes
	Initial Total	268.79 kB

Lazy Chunk Files	Names	Raw Size
chunk-0CFIALD3.js	animated-component	1.23 kB

Benefits:

Lazy loading of any kind of resources enhances user experience by speeding up initial page load times, saving bandwidth, prioritizing critical resources, and potentially improving website performance and SEO.

View Transitions

UX

Challenge:

There is no convenient way to implement smooth animated transitions between pages in Angular.

Solution:

Support for the View Transition API has been introduced. This is a relatively new mechanism that allows us to create smooth and interactive transition effects between different views of a web page. Thanks to this API, we can make changes to the DOM tree while an animation is running between two states.

Adding the View Transition API to our project is very simple.

First, we need to import the 'withViewTransitions' function at the bootstrap point of our application.

```
import { provideRouter, withViewTransitions } from '@angular/router';
bootstrapApplication(AppComponent, {
  providers: [provideRouter(routes, withViewTransitions())],
});
```

Now, the application immediately gains a subtle input and output effect when changing the URL. Of course, we have the ability to create custom animations. In the example below, the transition effect was extended to 2 seconds by using prepared pseudo-elements in the `styles.css` file.

```
/* Screenshot of the view of the page we are leaving */
::view-transition-old(root),
/* Representation of the new page view */
::view-transition-new(root) {
   animation-duration: 2s;
}
```

This example is a very small sample of what we can achieve with this API. If you are interested in the practical use of this mechanism, We encourage you to read the material available in the Chrome browser documentation.

However, it is important to remember that this is a relatively new and experimental feature. This means that it may not be fully supported in some browsers. You can check the level of support for individual browsers on caniuse.

How does this feature work internally? When navigation starts, the API takes a screenshot of the current page and puts it in ::view-transition-old pseudoelement. It also renders a new page, puts it in view-transition-new and plays the animation between them.

Benefits:

The View Transitions API simplifies the creation of smooth and cohesive navigation animations, enhancing user experience by providing a seamless visual transition between different states or views of a web application.

Esbuild + Vite (stable)

Dev Experience | Performance

Challenge:

Until now, Angular's build system primarily relied on Webpack for compiling applications, requiring separate builders for different purposes like production builds, development server, server-side rendering, and prerendering. This led to more complex configurations in angular.json, slower build processes due to repetition of common steps, and less efficient module loading as it did not fully utilize ECMAScript Modules (ESM).

Solution:

In Angular v17, we received the new 'application' builder, which streamlines the build process by using @angular-devkit/build-angular:browser-esbuild as its core. This builder simplifies configurations by consolidating various tasks such as production builds, server-side rendering (SSR), and prerendering into a single, more efficient process. It enhances performance by executing common build steps only once and fully supports ECMAScript Modules (ESM), leading to faster build times and more efficient, modern module loading.

To apply these changes to an existing Angular project, change the builder from:

```
@angular-devkit/build-angular:browser
```

to:

```
@angular-devkit/build-angular:application
```

If your application supports SRR capabilities, you might need a few extra configuration changes (in such case check the official documentation for details).

The new application builder provides the functionality for all of the preexisting builders:

- browser
- prerender
- server
- ssr-dev-server

The usage of Vite build pipeline is encapsulated within the 'dev-server' builder, with no changes necessary to use new build system.

Benefits:

Thanks to the unified solution, faster build-time performance is also brought to SSR apps. Using the same builder for both types of applications reduces the risk of potential differences resulting from using different bundlers. For new build systems with SSR & SSG, 'ng build' gets its speed increased by up to 87%, and 'ng serve' gets an 80% increase in speed.

SSR Hydration (stable)

Performance **UX**

Challenge:

Full Hydration presented in Angular version 16 was in developer preview mode, which could have stopped us from using it in applications since it was not yet stable.

Solution:

From version 17, the solution is marked as stable, which means it is considered safe for use in a production environment.

To enable it, use the following utility function in your project:

```
import { bootstrapApplication, provideClientHydration, } from '@angular/plat-
form-browser';

...

bootstrapApplication(RootCmp, {
  providers: [provideClientHydration()]
});
```

Benefits:

We can safely enjoy the benefits of hydration such as better UX (faster loading and interactivity, no more flickering) and enhanced SEO in a production environment.

CLI improvements

Dev Experience | Efficiency

Challenge:

The CLI does not take several new features from recent releases into account and uses older solutions by default.

Solution:

In Angular 17, applications generated by 'ng new' will be standalone by default and will use Vite as the default build system. All relevant code generators (i.e. for components, directives, pipes) will produce standalone output as well.

You can use the following migration to convert existing projects to the standalone APIs:

```
ng generate @angular/core:standalone
```

'ng-new' also supports the '--ssr' flag or alternatively includes a prompt asking if we wan't to enable SSR or SSG by default, including hydration.

```
Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? (y/N) y
```

To try out the new built-in control flow syntax in your existing project, you can use the following migration:

```
ng generate @angular/core:control-flow
```

Benefits:

New features in the framework are becoming more accessible, and we get ready-made configurations out-of-the-box right from the start, which speeds up the project launch.

Devtools Dependency Graph

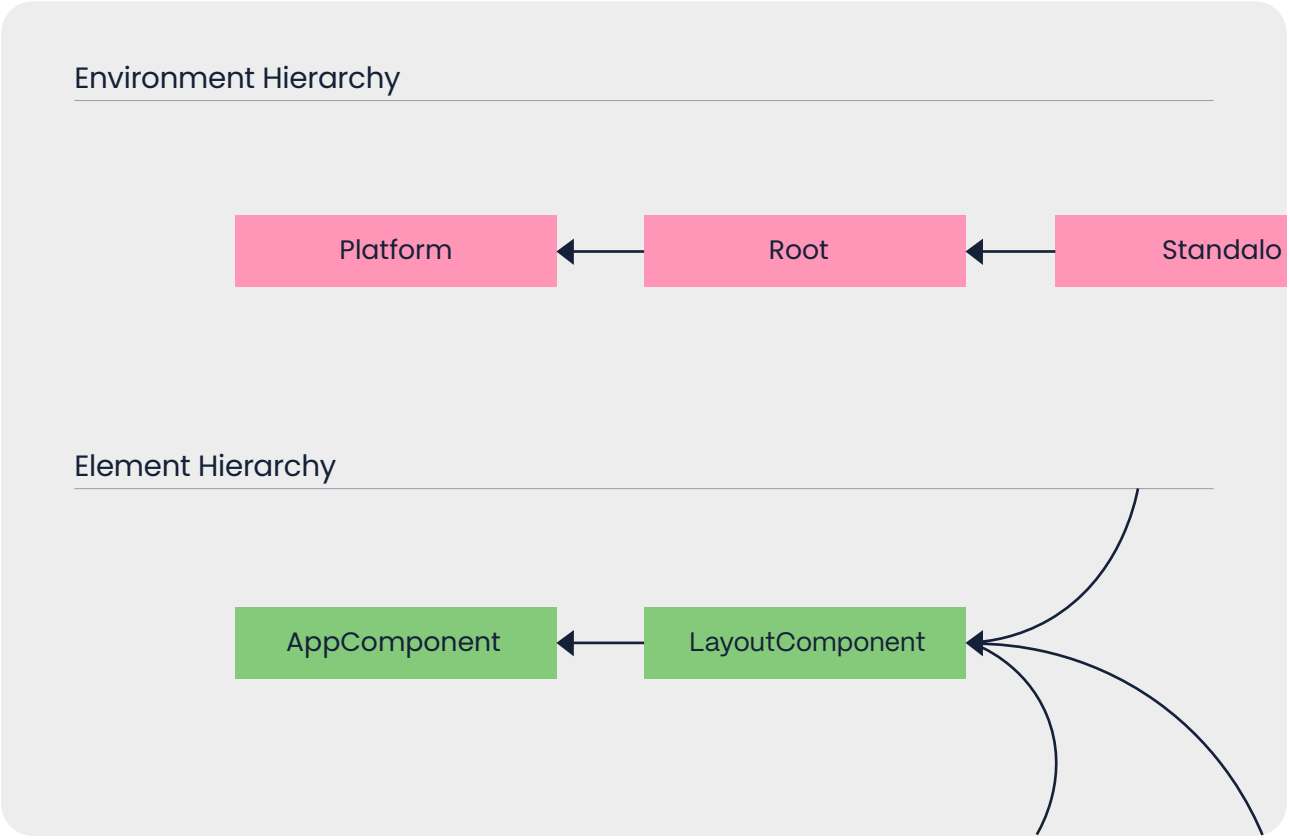
Dev Experience

Challenge:

Debugging dependency injection (DI) has always been problematic. If there were errors, they were always caused by the inability to inject a specific token, and if a token was successfully injected, there was no information about what injection context it came from.

Solution:

The Angular Team implemented brand new debugging APIs related to dependency injection. On top of that, they extended the capabilities of Angular Devtools and made it possible to inspect DI in the runtime. It is possible to verify a component's dependencies in a component inspector, injection tree with a dependency resolution path and providers declared in individual injectors.



Benefits:

The new debugging APIs for dependency injection in Angular helps developers to build more reliable and efficient applications.

Rebranding and introduction of angular.dev

Dev Experience

Challenge:

The branding of Angular has been almost identical since the beginning of AngularJS, which was released over 10 years ago. The framework documentation is also somewhat outdated, often based on ngModules.

Solution:

With the premiere of Angular version 17, we also received a new graphic identification of the framework and a new angular.dev - a home for Angular's new documentation.

Old logo:

New logo:

Angular.dev is a revamp with cutting-edge, interactive documentation. All examples have been reimagined with the standalone API and the latest Angular magic. Plus, we've got guides, tutorials and shiny Angular Playgrounds to code right in our browsers.

Benefits:

The new Angular.dev documentation makes it easier for developers to learn and use Angular.

Typescript/Node.js support

Angular v17 requires Typescript 5.2+ and Node.js v18+. Since 17.1, Angular also supports Typescript 5.3. New versions bring a handful of interesting changes that may be useful in Angular projects.

Explicit Resource Management (available in 5.2)

Dev Experience

Challenge:

There is no convenient way to explicitly manage resources with a specific life-time. For example, run some cleanup code when a given object reaches the end of its life.

Solution:

The system for managing objects with a specific lifetime will be part of the ECMAScript standard. For Typescript, it was already delivered in version 5.2. This system is inspired by solutions from other programming languages, for example, the "using" syntax from C# or the "try-with-resource" syntax from Java. It allows a programmer to manage the resource declaratively, meaning an object is disposed when execution leaves the scope where object has been defined, or imperatively by calling a Symbol.dispose method manually.

You can create a manageable resource by implementing a new Disposable interface in your class or by creating a function that returns an object implementing Disposable interface. Usually we use such a feature when we need some sort of "clean-up" after creating an object. Common cases include deleting temporary files and closing internet connections.

In the example below, we define a manageable object inside the function scope. When execution leaves the function context, the Symbol.dispose method is automatically called. This happens even if there was an exception during processing. We no longer have to wrap logic into the try/catch/finally block to make sure that clean-up logic is executed in all cases.

```
class Foo implements Disposable {
   [Symbol.dispose]() {
      // “clean-up” code goes here
   }
}

function doSomething() {
  using foo = new Foo();

  // do something
 if(someCondition()) {
    // do another thing
   return;
 }
}
```

This feature also contains support for async disposable resources, Symbol.asyncDispose, and extra methods for dealing with stacks like DisposableStack and AsyncDisposableStack.

Note that this feature is only available if you set the compiler target to es2022 and added “esnext” or “esnext.disposable” to the “lib” array.

```
{
 "compilerOptions": {
    "target": "es2022",
    "lib": ["es2022", "esnext.disposable", "dom"]
 }
}
```

Benefits:

Explicit Resource Management enhances code cleanliness and safety by providing a straightforward and efficient way to manage the lifecycle of resources. This ensures that clean-up or disposal code is automatically executed when the object goes out of scope or encounters an exception, without the need for extensive try/catch/finally blocks.

Named and Anonymous Tuple Elements (available in 5.2)

Dev Experience

Challenge:

There is no convenient way to explicitly manage resources with a specific lifetime. For example, run some cleanup code when a given object reaches the end of its life.

Solution:

Typescript gained support for optional names or labels for each element of the tuple. The all-or-nothing restriction has been removed and labels go as far as preserving the merging with other tuples.

```
// fully labeled tuple
type Coordinates = [latitude: number, longitude: number];

//partially labeled tuple
type RGB = [number, green: number, number];

// all labels are still preserved
type Combined = [...Coordinates, ...RGB];
```

Benefits:

The improvement enables greater flexibility and readability in TypeScript code.

Copying Array Methods (available in 5.2)

Dev Experience

Challenge:

Old and well-known array methods like sort, splice or reverse, all update an array in-place, meaning they mutate the original array. This can lead to unintended side effects and make code harder to understand and debug, especially in complex applications with shared state.

Solution:

TypeScript 5.2 adds definitions of ECMAScript array methods for modifications by copy. All methods that mutate the original array have their non-mutating variants available and ready to use.

- array.reverse() => array.toReversed()
- array.sort(...) => array.toSorted(...)
- array.splice(...) => array.toSpliced(...)
- array[i] = foo => array.with(i, foo)

Benefits:

Using non-mutable array methods over methods that will mutate arrays in place in the JavaScript language have multiple benefits. These include enhancing code predictability, maintainability, and readability by avoiding unintended side effects and ensuring consistent data state across different parts of an application.

Signal inputs

Hybrid Change Detection

Ng-content fallback

Angular v18

release date: **05.2024**

Angular 18 introduces significant advancements that enhance both performance and developer experience. This version replaces Zone.js with Hybrid Change Detection, which improves application speed and simplifies change tracking using signals. It also brings Signal Inputs and Model Inputs for more reactive components, along with Signal Queries for improved DOM handling. The new Output Syntax aligns with the signal-based approach, and function-based route redirects offer dynamic navigation options. Additionally, the introduction of ng-content fallback simplifies component development by allowing default content when no projection is provided.

Short list of features:	Dev Experience	Efficiency	Performance	UX
Hybrid Change Detection	✅		✅	
Signal inputs	✅	✅	✅	
Model inputs	✅	✅	✅	
Signal queries	✅		✅	
Output syntax	✅	✅		
Collaboration Between Angular and Wiz	✅		✅	
Ng-content fallback	✅	✅		
Route Redirects as Functions	✅			
New RedirectCommand	✅			
Improved Hydration Debugging Experience	✅			
New Observables in Forms	✅			
New Documentation	✅			
Hydration Support in CDK and Material	✅	✅	✅	
Material 3	✅	✅		
Typescript/Node.js support	✅			
instanceof Narrowing Through Symbol. hasInstance (available in 5.3)	✅			
Checks for super Property Accesses on Instance Fields(available in 5.3)	✅			
Narrowing on Comparisons to Booleans (available in 5.3)	✅			
Smarter Narrowing in Non-hoisted Functions (available in 5.4)	✅			
Expanded Template Literal Types (available in 5.4)	✅			
Enhanced readonly Arrays and Tuples (available in 5.4)	✅			
New Type Checking Compiler Options (available in 5.4)	✅			

Hybrid Change Detection

Performance | Dev Experience

Challenge:

Angular and Zone.js have been tightly connected from the beginning. While Zone.js helps Angular track asynchronous operations and automatically manage change detection, it can also slow down the application.Developers are often faced with the problem of the extra load that Zone.js adds and the difficulty of manually managing the detection of changes. Main Functions of Zone.js in Angular include:

- Tracking Asynchronous Context: Zone.js creates "zones" that track asynchronous actions like setTimeout, Promise, DOM events, and HTTP requests. This helps Angular to know when these actions have been done and to react properly.
- Decorating Asynchronous APIs: Zone.js intercepts and wraps standard browser APIs responsible for asynchronous tasks (setTimeout, setInterval, addEventListener, Promises). This allows Angular to monitor when these tasks start and finish.
- Change Detection: After an asynchronous task ends, Zone.js tells Angular that something might have changed in the application data. Angular then runs its change detection to update the view if there are any changes.

These functions are key to Angular's ability to manage asynchronous actions and update the view automatically. However, Zone.js adds performance overhead because it has to track and wrap all these asynchronous tasks. In large apps with many asynchronous calls, this can slow down performance. Techniques like running code outside the zone can help, but they are tricky and can negatively affect the developer experience.

Solution:

Hybrid Change Detection is an experimental solution introduced in version 18 of Angular. It allows us to get rid of the zone.js completely and track our changes using a new change detection mechanism. It's worth noting that this change is strongly related to the signals which simplify the mental model of detecting the changes. We can enable the new change detection using the dedicated function **provideExperimentalZonelessChangeDetection.** After adding the provider, remove zone.js from your polyfills in angular.json.

```
bootstrapApplication(RootCmp, {
 providers: [provideExperimentalZonelessChangeDetection()]
});
```

The new change detection mechanism should only respond to data changes that are extremely easy to detect from the signals. The new change detection cycle will be scheduled when:

- A Signal updated
- Calls to changeDetectorRef.markForCheck()
- A subscribed Observable with the AsyncPipe receives a new value
- A component gets attached/detached
- Setting an input

Benefits:

By eliminating Zone.js, the application experiences less overhead, resulting in faster performance, especially in applications with many asynchronous operations. We should expect the code to become simpler.

Some combinations related to running code outside of the zone or just manually triggering change detection should be rare cases which in effect will enhance developer experience. Excluding the scenario of manually invoking change detection, it will actually react to data changes, since data is what drives our applications, we will avoid unnecessary checks, improving overall performance and application responsiveness.

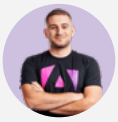
Additionally it should reduce application bundle size, improve core web vitals, speed uppage loading time and simplify stack trace and debugging.

Expert Opinion:

Zone.js affects Angular applications in terms of loading and runtime performance. The library size and the monkey patching of browser events that might trigger change detection more often than necessary can degrade Core Web Vitals metrics. Zoneless Change Detection will remove the dependency and improve the runtime performance of most existing Angular applications.

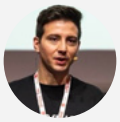
~ Aristeidis Bampakos
Google Developer Expert

If your app is already optimized for performance, zoneless will improve the performance even more, but if the performance is already good enough for you, it would be a nice-to-have. If you're using OnPush Change Detection, it means that your app is already zoneless compatible, so you can enable it without having any issues.



~ Enea Jahollari
Google Developer Expert

Removing Zone JS, it's one of the biggest changes for the Angular team in the end. I think it's one of the ideas from one of the first releases of Angular. Now, after the release of Ivy, I saw a lot of improvement in the framework at the end. With Ivy under the hood, the framework is starting to be more community-driven, first of all, because Angular team is able to release a lot of elements that are requested by the community. They're also faster in their release process.



~ Luca Del Puppo
Google Developer Expert

Signal inputs

Performance **Efficiency** **Dev Experience**

Challenge:

The current implementation of component inputs is not very reactive. In order to look for the changes, we use component lifecycle hooks. That means that if we want the input values to interact with the Signal API, we have to handle the logic manually.

Solution:

Angular version 17.1 enables signal inputs for components. This means we no longer use the decorator. Instead, a dedicated function defines the input:

```
@Component({
 selector: 'my-cmp',
 standalone: true,
 template: `{{ name() }}`,
})
export class MyComponent {
 name = input<string>();
}
```

This mechanism allows you to simplify the responsiveness of the component by letting you define computed signals and side effects for input value changes. It is no longer necessary to use the ngOnChanges lifecycle hook.

```
export class MyComponent {
 name = input<string>();
 helloMessage = computed(() => 'Hello' + name());

constructor() {
 effect(() => {
   console.log('logging: ' + this.name());
   })
 }
}
```

The Signal input API has extra capabilities compared to its standard counterpart, such as providing default values, marking inputs as required, setting aliases, and various transformingfunctions.

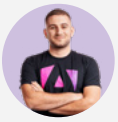
```
export class MyComponent {
// required input
name = input.required<string>();

// input with default value
color = input('red');

// input with alias
disabled = input<boolean>(false, { alias: 'isDisabled' });

// input with transform function
age = input.required({ transform: numberAttribute });
}
```

Benefits:

Signal inputs mark a significant step towards the deeper integration of signals into the ecosystem and transform the way we deal with component input changes. This feature significantly enhances the developer experience and code clarity by streamlining the management of component inputs and facilitating more intuitive coding practices.

Expert Opinion:

Don't sleep on signals! While there may be some rough edges to migrating if you're using RxJS for everything, signal will give you an even better developer experience and performance!



~ Enea Jahollari
Google Developer Expert

Model inputs

Performance | Efficiency | Dev Experience

Challenge:

Standard inputs in Angular are read-only and do not allow components to send new values back to the parent components. This limitation restricts the creation of interactive components that need to update the parent component's state based on user actions.

Solution:

Angular introduces model inputs, a special type of input that allows components to propagate new values back to the parent component. Currently in developer preview, this feature facilitates two-way binding, allowing for more dynamic interactions.

Model inputs are defined in much the same way as standard inputs, but with added capabilities for two-way data binding. Here's how to define and use model inputs in Angular:

1. **Defining Model Inputs:**
   Model inputs can be created using the model() function in Angular.
   Unlike standard inputs, model inputs can update their values and propagate these changes back to the parent component.
1. **Using Model Inputs in Parent Components:**

```
import { Component, model, input } from '@angular/core';

@Component({
  selector: 'custom-toggle',
  template: '<button (click)="toggle()">Toggle</button>'
})
export class CustomToggle {
 active = model(false); // This is a model input
 disabled = input(false); // This is a standard input

toggle() {
  this.active.set(!this.active());
  }
}
```

Parent components can bind properties to model inputs using two-way binding syntax.

In this example, the isActive signal in the parent component (AppComponent) stays

```
import { Component } from '@angular/core';
import { signal } from '@angular/core';

@Component({
 selector: 'app-root',
 template: '<custom-toggle [(active)]="isActive" />',
})
export class AppComponent {
 isActive = signal(false); // Writable signal
}
```

in sync with the active model input in the child component (CustomToggle).

1. **Two-Way Binding with Plain Properties:**
   You can also bind plain JavaScript properties to model inputs.
1. **Implicit Change Events:**

```
import { Component } from '@angular/core';

@Component({
selector: 'app-root',
template: '<custom-toggle [(active)]="isActive" />'
})
export class AppComponent {
isActive = false; // Plain property
}
```

Declaring a model input automatically creates a corresponding output event suffixed with "Change". This event is triggered whenever the value of the model input changes.

```
import { Directive, model } from '@angular/core';

@Directive({
selector: 'custom-toggle',
})
export class CustomToggle {
active = model(false); // Creates an output event "activeChange"
}
```

Benefits:

Model inputs have several advantages. They enable dynamic interactions by allowing child components to update the state of parent components through two-way data binding. This results in more interactive and responsive applications. Additionally, model inputs simplify code by reducing the need for additional properties and boilerplate code, leading to cleaner and more maintainable codebases. They also enhance flexibility, making it easier to create custom form controls and other interactive components that need to modify values based on user actions.

By improving the flexibility and interactivity of Angular components, the model inputs make it easier to build dynamic and user-friendly applications. This leads to better user experiences and more efficient management of complex state changes within the Angular components.

Signal queries

Performance **Dev Experience**

Challenge:

Similar to the old inputs, the current implementation of view queries is not very reactive. We need to use setters or observables to track when the queried dom changes.

Solution:

New functions have been introduced as an alternative to decorators. Each of them produces as a result a signal value of the queried DOM.

ViewChild:

The first of these is ViewChild().It's a signal based replacement for @ViewChild(). We use it when we are looking for a single result in our component. Similarly to input() or model(), we can also use the required option here.

```
@Component({
…,
template: `
<div #el></div>
<div #requiredDiv></div>
<my-child />
`
})
export class MyComponent {
divEl = viewChild<ElementRef>('el'); // Signal<ElementRef|undefined>
requiredDivEl = viewChild.required<ElementRef>('requiredDiv');
// Signal<ElementRef>
cmp = viewChild(ChildComponent); // Signal<ChildComponent|undefined>
}
```

ViewChildren:

Similarly, the ViewChildren function is the signal-based counterpart to @ViewChildren(). It is used for querying multiple elements within the component.

```
@Component({
template: `
<div #el></div>
<div #el></div>
<div #el></div>
`,
})
export class MyComponent {
firstSelector = viewChildren<ElementRef>('el');
// Signal<readonly ElementRef<any>[]>
secondSelector = viewChildren<ElementRef<HTMLDivElement>>('el');
// Signal<readonly ElementRef<HTMLDivElement>[]>
}
```

ContentChild and ContentChildren:

The contentChild() and contentChildren() functions replace @ContentChild and @ContentChildren, respectively, and are used for querying projected content within a component.

```
// parent.component.ts
@Component({
template: `
  <ng-content></ng-content>
`, // Notice the ng-content tag ng-content
standalone: true,
selector: 'app-parent'
})
export class ParentComponent {
 content = contentChild(ChildComponent);
 // Signal<ChildComponent | undefined>

 contentElements = contentChildren(ChildComponent);
 // Signal<readonly ChildComponent[]>
}
```

Benefits:

The new signal-based approach simplifies the codebase, making it easier to read and maintain. Signals provide a more reactive way of handling DOM queries, automatically updating as the queried elements change. This results in a better developer experience with less need for manual settings and immediate improvements in responsiveness. This leads to a smoother development process and less boilerplate code.

Output syntax

Efficiency **Dev Experience**

Challenge:

Inputs have been changed to be signal based. The output so far was still in the old syntax. This meant we needed decorators to declare them. What's more, there was inconsistency in terms of the type safety, as the emit function was accepting following type: T | undefined which was allowing us to emit an event without value even though we had typed it to be specific type.

```
@Component(...)
export class MyComponent {
  @Output() oldOutput = new EventEmitter<string>();

onEvent(): void {
  this.oldOutput.emit(); // It's ok even though we typed it as string
 }
}
```

Solution:

A new output function has been introduced. A new output function has been introduced. Mostly it's just a syntax adjustment, it has nothing to do with signals. For example, it doesn't mean that a sent value is immediately a signal, as it is with input, where sent values are always read as signals.

```
@Component(...)
export class MyComponent {
 valueChanged = output<string>();


 onValueChanged(msg: string): void {
   this.valueChanged.emit(msg);
 }
}
```

Changes have also been made to the return type of the output function. It now returns OutputEmitterRef<T>, which prevents the output of a value inconsistent with the declared one. The example we provided in the challenge section will generate an error when using the new syntax.

```
@Component(...)
export class MyComponent {
 newOutput = output<string>();

onEvent(): void {
 this.newOutput.emit(); // ERROR: Expected 1 arguments, but got 0.
 }
}
```

Benefits:

The new syntax alignment with signal-based DOM queries and inputs offers several benefits. It prevents the emission of undefined values, improving the reliability and accuracy of our code. Better typing makes our code less error-prone, reducing the likelihood of runtime errors and ensuring a smoother user experience.

Collaboration Between Angular and Wiz

Performance Dev Experience

Challenge:

Angular, despite its extensive feature set and developer-friendly tools, has faced significant competition from other frameworks like React and Vue. A major challenge has been balancing the need for performance optimization with maintaining good developer experience. In addition, Angular relies on zone.js for reactive programming, which has introduced overhead and complexity that affects optimal performance, especially in performance-critical applications used by millions of users, such as those developed by Google.

Solution:

To address these challenges, the Angular team has begun a collaboration with Wiz, an internal web framework developed by Google and known for its performance optimisations in applications such as Google Search and Photos. This collaboration aims to integrate Wiz's performance-related features into Angular's developer tools, starting with the introduction of Angular Signals.

Angular Signals, previewed at NG Conf 2024, represents a significant change to theAngular's reactive programming model. Designed to replace zone.js-based changedetection, Signals provides a new reactive foundation that improves both performance and developer experience. This integration has already been put into practice, with Signals being used in production for YouTube Mobile, demonstrating its effectiveness and potential.

Benefits:

The collaboration between Angular and Wiz brings several significant benefits. Integrating Wiz's performance optimizations with Angular improves the overall efficiency and speed of applications, addressing one of the keychallenges faced by Angular developers.

This not only reduces the overhead mostly associated with zone.js but also simplifies reactive programming, making it more intuitive and less prone to errors. As a result, developers can enjoy a smoother, more productive coding experience.

Moreover, the synergy between the two teams is good for continuous innovation, ensuring that both frameworks benefit from shared insights and technological advancements.

This partnership has the potential to enhance Angular's position in the web development community, attracting more developers due to its improved performance and user-friendly features. This ongoing collaboration signifies a future where Angular not only retains its solidity but also evolves to meet the high performance demands of modern web applications, making it an attractive choice for developers.

Expert Opinion:

The Angular and Wiz collaboration will try to blend together the best parts of both frameworks. In Angular, we will see improvements in hydration through Wiz's event dispatch library, which can record and replay events on the client while it is not yet interactive. In Wiz, we will see improvements in the state management with the extended usage of signals.

We are not going to see Wiz switching to a full open-source model. Rather, we will see some of its parts becoming open source through the Angular framework.

~ Aristeidis Bampakos
Google Developer Expert

What are the core aspects and benefits of merging Angular and Wiz? When can we expect Wiz to switch to an open-source model?

Maybe Google has a secret sauce that makes things super fast. How fast? Think how performant Gmail and Google search are. That's WIZ in action!

On the other hand, Angular is great for complex interactive UIs.
Combining the performance of WIZ with the awesomeness of angular will make it an even more popular framework.

Some of WIZ's features may become open source and others will find their way to Angular. We have already seen how great the deferrable views are, which is WIZ's influence on Angular.

~ Fanis Prodromou
Google Developer Expert

Ng-content fallback

Efficiency | Dev Experience

Challenge:

One of the well-known challenges in Angular development has been handling scenarios where a component may not receive any projected content. In such cases, developers traditionally had to implement complex workarounds to ensure that component still displays useful default content. This lack of native support for fallback content in ng-content has been a limitation, causing additional overhead and complexity to component design and development.

Solution:

Angular v18 introduces a significant enhancement with built-in support for fallback content in ng-content slots. This new feature simplifies the process of defining default content that will be displayed if no content is projected into the component. By allowing developers to specify fallback content directly within the ng-content tag, Angular v18 improves component development, making it more intuitive and efficient.

The implementation is straightforward. Developers can now include default content within the ng-content tag itself. For instance:

```
import { Component } from '@angular/core';

@Component({
standalone: true,
selector: 'my-component',
template: `
  <ng-content select="header"> Default header </ng-content>
  <ng-content> Default main content </ng-content>
  <ng-content select="footer"> Default footer </ng-content>
`,
})
export class MyComponent {}
```

In this example, if no content is provided for the header, main content, or footer, the default values ("Default header", "Default main content", "Default footer") will be displayed. This feature significantly reduces the need for conditional logic and additional checks within the component's logic to ensure that fallback content is shown.

Benefits:

The introduction of fallback content in ng-content slots brings multiple benefits to Angular developers. Firstly, it gives us the way to ensure that components always display meaningful content, even when no projected content is provided. This improvement reduces the risk of blank or incomplete UI elements in applications.

Additionally, the new feature simplifies the component development process. By embedding fallback content directly into the component's template, developers can avoid complex workarounds. This approach improves the overall developer experience.

Route Redirects as Functions

Dev Experience

Challenge:

In previous versions of Angular, route redirects were limited to static strings. This limitation made it difficult to handle redirects that depend on runtime conditions or complex logic. Developers needed a more flexible way to define redirects based on dynamic data, such as query parameters or application state.

Solution:

Angular version 18 introduces a new feature that allows redirectTo to accept a function that returns a string. This function can implement complex logic, enabling more flexible and dynamic redirects. It essentially receives the current route's context, including query parameters, allowing developers to tailor the redirection logic to specific runtime conditions.

Consider a scenario where you need to redirect users based on the presence of a userId query parameter. If the userId is provided, users should be redirected to their profile page. If not, an error should be logged, and users should be redirected to a "not found" page.

Here's how this can be achieved using the new redirectTo function feature:

```
const routes: Routes = [
 { path: "first-component", component: FirstComponent },
 {
  path: "old-user-page",
  redirectTo: ({ queryParams }) => {
   const errorHandler = inject(ErrorHandler);
   const userIdParam = queryParams['userId'];
   if (userIdParam !== undefined) {
    return `/user/${userIdParam}`;
   } else {
    errorHandler.handleError(new Error('Attempted navigation to user page
    without user ID.'));
    return `/not-found`;
   }
  },
},

 { path: "user/:userId", component: OtherComponent },
];
```

In this example:

- The redirectTo function checks the queryParams for a userId.
- If userId is present, the function returns the URL tothe user's profile.
- If userId is missing, the function logs an error using an ErrorHandler service and redirects to a "not found" page

Benefits:

The introduction of function-based route redirects in Angular greatly enhances the flexibility of routing in applications. It allows developers to define dynamic redirection logic based on the application's current state or runtime data. This means that developers can easily implement complex routing requirements without resorting to workarounds or additional code structures.

This feature not only improves the developer experience by simplifying the implementation of dynamic routing but also enhances the maintainability and readability of the routing code. It streamlines navigation logic, making it more intuitive and reducing the potential for errors, thus improving overall application reliability.

New RedirectCommand

Dev Experience

Challenge:

In previous Angular versions, managing complex navigation patterns and using NavigationExtras with Guards and Resolvers could be cumbersome and less maintainable. Developers struggled to handle advanced navigation scenarios, which required a more streamlined and flexible approach to manage redirections and navigation states.

Solution:

Angular version 18 introduces the RedirectCommand class, designed to manage NavigationExtras more effectively. This class allows for improved redirection capabilities within Angular applications, particularly when used in conjunction with Guards and Resolvers. The RedirectCommand class makes it easier to implement and maintain complex navigation logic, enhancing both flexibility and developer experience.

Consider a scenario where a route guard needs to redirect users based on certainconditions, such as checking user permissions or application state. The new RedirectCommand class simplifies this process by allowing developers to create a UrlTree and manage NavigationExtras in a concise manner.

Here's an example that demonstrates the use of RedirectCommand within a route guard:

```
const route: Route = {
path: 'page1',
component: PageComponent,
canActivate: [
  () => {
    const router: Router = inject(Router);
    const urlTree: UrlTree = router.parseUrl('./page2');
    return new RedirectCommand(urlTree, { skipLocationChange: true });
  },
 ],
};
```

In this example:

- The guard function uses Angular's dependency injection to get an instance of the Router.
- It then creates a UrlTree for the desired redirect URL ('./page2').
- The guard returns a new RedirectCommand, that includes the UrlTree and additional NavigationExtras options such as skipLocationChange.

Benefits:

The introduction of the RedirectCommand class in Angular 18 provides several advantages. It increases the flexibility and maintainability of routing logic, particularly in the complex navigation scenarios involving Guards and Resolvers. This feature streamlines the management of redirections and NavigationExtras, reducing boilerplate code and making it easier to implement robust navigation patterns. By simplifying the handling of challenging navigation tasks, the RedirectCommand class improves developer productivity and the overall developer experience in Angular applications.

This feature makes it more straightforward for developers to manage advanced redirection scenarios, improving the reliability and readability of navigation-related code. This leads to a better-maintained applications and a smoother development process.

Improved Hydration Debugging Experience

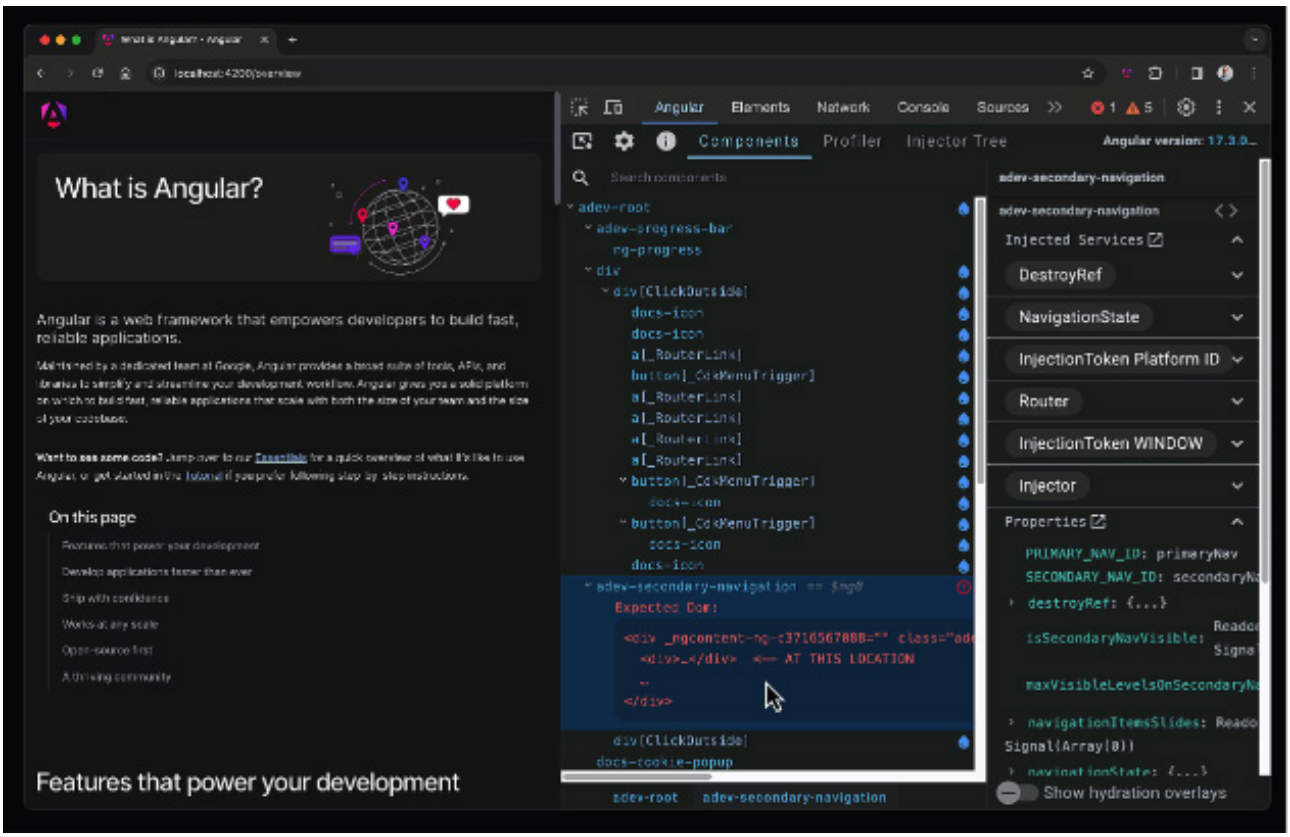
Dev Experience

Challenge:

Debugging hydration issues in Angular applications used to be challenging. Developers lacked visibility into the hydration status of components, making it difficult to identify and resolve hydration errors effectively. This limited insight often led to extended debugging sessions and reduced productivity.

Solution:

Angular has updated Angular DevTools to enhance the debugging experience by visualizing Angular's hydration process. This update includes new icons next to each component indicating its hydration status. Additionally, developers can enable an overlay mode to see which components have been hydrated on the page. If there are any hydration errors, Angular DevTools will visualize them directly in the component explorer, providing immediate feedback and detailed information about the problem.



Benefits:

The improved debugging experience with Angular DevTools provides several key benefits. It significantly enhances developers' ability to diagnose and fix hydration issues by providing clear visual indicators of each component's hydration status. The overlay mode provides a comprehensive view of the hydration process across the entire application, making it easier to understand and debug complex hydration scenarios. Additionally, visualizing hydration errors directly in the Component Explorer helps developers quickly pinpoint and resolve problems, improving productivity and reducing the time spent on debugging.

New Observables in Forms

Dev Experience

Challenge:

Managing state changes in form controls has been a recurring challenge. Traditionally, we have had to subscribe to multiple observables or use different methods to track changes in form control values, status, and other states such as pristine or touched. This approach can lead to increased complexity of the code, making it harder to maintain and understand, especially in large applications with many form controls. There were events we couldn't subscribe to at all, such as touched and untouched.

Solution:

Angular v18 introduces a unified approach to handling control state changes with a new feature added to the AbstractControl class. This feature exposes an event observable that tracks various state changes in form controls, such as value changes, pristine state, touch status, and overall control status. By combining these state changes into a single observable stream, Angular simplifies the management of form control states.

Here's how it works:

```
import { Component, OnInit } from '@angular/core';
import { FormControl } from '@angular/forms';

@Component({
 selector: 'app-my-form',
 template: `
 <form>
  <input [formControl]="control">
 </form>
`
})
export class MyFormComponent implements OnInit {
 control = new FormControl('');

ngOnInit() {
 this.control.events.subscribe(event => {
  console.log(event);
  });
 }
}
```

In this example, the control.events observable emits events whenever the value, status or state of the form control changes. This unified event stream allows developers to handle all state changes in a single subscription, making the code easier to maintain.

Benefits:

The unified control state change events make it easier to track and manage form control states by combining all state changes into one observable. This simplifies the code, making it clearer and easier to maintain. It also improves the reliability of handling user interactions and form submissions. Overall, this feature boosts developer productivity and simplifies the development process.

New Documentation

Dev Experience

Challenge:

The previous Angular documentation site, angular.io, had several areas that could be improved to enhance the learning experience for developers. Developers needed more intuitive navigation, better search functionality, and interactive learning tools to get started with Angular in a more effective way. What's more, some of the examples were already out of date (e.g. syntax) and updating them to the latest standards was quite difficult.

Solution:

Angular has launched a new documentation site, angular.dev, which is now the official documentation website for Angular. This new site has a modern look and feel and includes several enhancements aimed at improving the developer experience.

Features of Angular.dev:

1. Interactive Hands-On Tutorial:
   Angular.dev features an interactive, hands-on tutorial based on WebContainers, allowing developers to try out Angular concepts and code directly in the browser.

2. Interactive Playground:
   The site includes an interactive playground with examples, enabling developers to experiment with Angular code snippets and see the results in real time.

3. Improved Search:
   Powered by Algolia, the new search functionality offers faster and more accurate results, helping developers find the information they need more efficiently.

4. Refreshed Guides:
   The guides on angular.dev have been updated and improved to provide clearer, more detailed explanations and instructions.

5. Simplified Navigation:
   Navigation has been streamlined to make it easier for developers to find their way around the site and access the resources they need.

6. Modern Look and Feel:
   The site has been redesigned with a modern aesthetic, making it more visually appealing and easier to use.

Benefits:

The new angular.dev site significantly enriches the developer experience by providing a more intuitive and engaging learning environment. The interactive tutorials and playground allow developers to learn by doing, which can be more effective than just reading documentation. Improved search and simplified navigation help developers find the information they need quickly, reducing the time spent searching for answers and increasing productivity. Additionally, the refreshed guides and modern design make the documentation more accessible and enjoyable to use.

By redirecting all requests from angular.io to angular.dev and ensuring that existing links continue to work by forwarding to v17.angular.io, Angular has made the transition smooth for developers. This ensures that the community can continue to access valuable resources without disruption.

Visit angular.dev to explore the new site and take advantage of these improvements!

Hydration Support in CDK and Material

Performance | Efficiency | Dev Experience

Challenge:

In Angular version 17, some Angular material and CDK components were removed from hydration. This meant that these components would not retain their server-rendered state and would be rerendered on the client side. This re-rendering could impact performance and user experience, as the components had to be reinitialized and redrawn after the initial load.

Solution:

Starting with Angular version 18, all Angular Material and CDK components are fully hydration compatible. This means that these components will now retain their server-rendered state and do not require rerendering on the client side. This change improves performance by reducing the need for additional rendering work once the application loads in the browser.

Benefits:

The update to make all Angular Material and CDK components fully hydration compatible brings several benefits. It enhances the performance of Angular applications by reducing unnecessary rerendering of components. This leads to faster load times and a smoother user experience, as the components are ready to interact with immediately after the initial server-side rendering. It also simplifies the development process by ensuring consistent behaviour across all Material and CDK components, without the need for special handling or workarounds for hydration issues.

Material 3

Efficiency Dev Experience

Challenge:

A few months ago, Angular introduced experimental support for Material 3, the latest version of Google's Material Design guidelines. During this experimental phase, developers provided feedback highlighting areas for improvement. The goal was to refine these components to ensure they met the high standards expected by the developer community before releasing them as stable. At the same time, Angular Material was still based on Material 2, which made customization difficult. This lack of flexibility posed challenges for developers looking to implement the latest design trends and tailor components to their specific needs.

Solution:

After listening to feedback and making the necessary adjustments, Angular has now graduated the Material 3 components to stable status. This means that Material 3 is fully supported and ready for production use, providing developers with a polished set of UI components that adhere to the latest Material Design guidelines. Additionally, Angular Material 3 introduces tokens for theming, which greatly enhance customization possibilities. These tokens allow for more granular control over component styling, giving developers greater flexibility to tailor their applications to specific design requirements.

Benefits:

The transition of Material 3 to stable status provides several key benefits. First, it provides reliability, as developers can now use Material 3 components in production environments with confidence, knowing that they have been thoroughly tested and refined. Second, the components adhere to the latest Material Design guidelines, ensuring a modern and consistent look and feel across applications. Third, the updated material.angular.io site provides comprehensive documentation and resources, making it easier for developers to implement and customize Material 3 components. Together, these enhancements simplify the development process and ensure that applications built with Angular and Material 3 are visually appealing, consistent, and maintain high standards of user interface design.

Typescript/Node.js support

instanceof Narrowing Through Symbol.hasInstance (available in 5.3)

Dev Experience

Challenge:

Prior to TypeScript 5.3, the instanceof operator did not always work well with custom type checks. This made it hard to correctly check the type of some objects, which could lead to errors in the code.

Solution:

TypeScript 5.3 adds a new feature that allows the instanceof operator to use custom type checks. If a class has a [Symbol.hasInstance] method, TypeScript will use it to check the type of an object. This makes type checking more accurate.

```
interface PointLike {
 x: number;
 y: number;
}

class Point implements PointLike {
 x: number;
 y: number;

constructor(x: number, y: number) {
 this.x = x;
 this.y = y;
}

static [Symbol.hasInstance](val: unknown): val is PointLike {
  return (
   !!val &&
   typeof val === 'object' &&
   'x' in val &&
   'y' in val &&
   typeof val.x === 'number' &&
   typeof val.y === 'number'
  );
  }
}
```

```
function checkValue(value: unknown) {
if (value instanceof Point) {
  console.log(value.x); // Safe to access x and y
  console.log(value.y);
 }
}
```

Summary:

Before TypeScript 5.3, developers had to create manual type guard functions to handle custom instance checks, which made the code less clean and increased the potential for errors. This manual process involved writing functions to check whether an object met certain criteria to be considered of a particular type, such as checking properties and their types individually.

With the introduction of TypeScript 5.3, the instanceof operator can now leverage custom type guards defined through the Symbol.hasInstance method. This allows developers to embed custom type-checking logic directly within classes, making the type-checking process more accurate and the code cleaner. The instanceof operator now automatically uses this custom logic, simplifying the code and enhancing type safety.

In summary, TypeScript 5.3 enhances the instanceof operator by allowing it to use custom type guards via **Symbol.hasInstance**, resulting in more accurate type checking and cleaner code. This improvement reduces the need for manual type guard functions and makes the code more reliable and easier to maintain.

Checks for super Property Accesses on Instance Fields (available in 5.3)

Dev Experience

Challenge:

Using super incorrectly on instance fields could cause runtime errors because super only works on prototype methods, not instance properties. This led to confusing bugs that were difficult to diagnose and fix.

Solution:

TypeScript 5.3 adds checks to ensure that super is used correctly, preventing runtime errors by issuing a type error when super is used on instance fields.

```
class Base {
 someMethod = () => {
   console.log("Base method called!");
 }
}
class Derived extends Base {
 someOtherMethod() {
   super.someMethod(); // Error: 'super.someMethod' is 'undefined'
 }
}

new Derived().someOtherMethod();
```

This improvement prevents runtime errors, making the code safer and easier to debug.

Benefits:

Checks for super property accesses on instance fields prevent runtime errors, making code safer and easier to debug.

Narrowing on Comparisons to Booleans (available in 5.3)

Dev Experience

Challenge:

Boolean value comparisons were not well understood by TypeScript, leading to inaccurate type narrowing and potential type errors.

Solution:

TypeScript 5.3 improves the handling of boolean comparisons, enabling for accurate type narrowing and more secure code.

```
type MyType = { a: string } | { b: string };

function isA(x: MyType): x is { a: string } {
return "a" in x;
}

function someFn(x: MyType) {
  if (isA(x) === true) {
    console.log(x.a); // Works correctly
  }
}
```

Benefits:

Narrowing on comparisons to booleans improves type safety, reduces errors, and enhances code clarity.

Smarter Narrowing in Non-hoisted Functions (available in 5.4)

Dev Experience

Challenge:

Previously, type escaping in non-hoisted functions was inaccurate, leading to potential type errors and less reliable code. This was particularly problematic in functions where variables could change type or value multiple times, making it difficult for TypeScript to keep track of the correct type.

Solution:

TypeScript 5.4 refines type narrowing in non-hoisted functions by considering the last assignment point for type checking. This improvement ensures that TypeScript accurately tracks the type of variables based on their most recent value, resulting in more reliable and accurate type checks.

```
function example(flag: boolean) {
let value: string | number;

if (flag) {
   value = "string";
 } else {
  value = 42;
 }

// TypeScript now correctly infers the type of 'value' based on the last assign-
ment
if (typeof value === "string") {
console.log(value.toUpperCase()); // Works because 'value' is correctly in-
ferred as string
 } else {
console.log(value.toFixed(2)); // Works because 'value' is correctly inferred as
number
 }
}
```

Benefits:

Smarter narrowing in non-hoisted functions improves type accuracy, reduces type errors, and improves the overall clarity and reliability of the code. This makes it easier for developers to write safe and precise code without worrying about incorrect type inferences.

Expanded Template Literal Types (available in 5.4)

Dev Experience

Challenge:

Template literal types in previous versions of TypeScript were limited in their expressiveness and flexibility. Developers faced challenges when trying to define complex types that involved string manipulation and pattern matching, reducing the usefulness of template literal types in certain scenarios.

Solution:

TypeScript 5.4 extends the capabilities of template literal types, making type definitions more expressive and flexible. This enhancement makes it possible to create more complex and precise types that better meet the needs of various coding scenarios, especially those involving dynamic string patterns.

```
type OrderStatus = "pending" | "shipped" | "delivered";
type UppercaseOrderStatus = Uppercase<OrderStatus>; // "PENDING" |
"SHIPPED" | "DELIVERED"
type StatusMessage = `Order is ${OrderStatus}`; // "Order is pending" | "Order is
shipped" | "Order is delivered"

function getStatusMessage(status: OrderStatus): StatusMessage {
  return `Order is ${status}`;
}

let message: StatusMessage = getStatusMessage("pending");
console.log(message); // Output: "Order is pending"
```

Benefits:

Expanded template literal types improve the flexibility and expressiveness of type definitions, making it easier for developers to work with complex string patterns and dynamic types. This enhancement leads to clearer and more maintainable code, as types can now more accurately reflect the intended data structures.

Enhanced readonly Arrays and Tuples (available in 5.4)

Dev Experience

Challenge:

Read-only arrays and tuples in previous versions of TypeScript had limited type inference and immutability guarantees. This made it challenging to work with immutable data structures, as the type system did not always provide accurate type information or enforce immutability effectively.

Solution:

TypeScript 5.4 enhances the handling of read-only arrays and tuples by improving type inference and immutability guarantees. This ensures that the type system accurately reflects the immutability of these data structures, provides better type information and enforces immutability more effectively.

```
const numbers: readonly number[] = [1, 2, 3];
numbers.push(4); // Error: Property 'push' does not exist on type 'readonly
number[]'
type Point = readonly [number, number];
const point: Point = [1, 2];
point[0] = 3; // Error: Index signature in type 'readonly [number, number]' only
permits reading
```

Benefits:

Enhanced read-only arrays and tuples improve type inference, guarantee stronger immutability guarantees, and make it easier for developers to work with immutable data structures. This leads to more reliable and maintainable code, as developers can be confident that read-only data structures will not be inadvertently modified.

New Type Checking Compiler Options (available in 5.4)

Dev Experience

Challenge:

Developers previously had limited control over type checking settings in TypeScript, making it difficult to customize type checks to meet specific project needs. This lack of flexibility could result in type checking that was either too strict or too lenient, reducing the effectiveness of the type system.

Solution:

TypeScript 5.4 introduces new compiler options that give developers more control over type checking settings. These options allow for greater customization of type checks, so developers can tailor the type system to the specific needs of their projects.

```
{
  "compilerOptions": {
    "strictNullChecks": true,
    "noImplicitAny": true,
    "noImplicitThis": true,
    "strictPropertyInitialization": true
  }
}
```

Benefits:

New type checking options enhance customization and give developers more control over how TypeScript enforces type checks. This flexibility leads to a more tailored and effective type system, resulting in higher quality code and a better development experience.

Resource API

Server Route Configuration

Incremental Hydration

Angular v19

release date: **11.2024**

Angular v19 introduces a wealth of features designed to increase developer productivity, enhance application performance, and enrich the user experience.

With updates such as linkedSignal for reactive state management, hot module replacement for faster development cycles, and the experimental resource API for streamlined async operations, Angular continues to evolve as a modern framework.

From simplifying initializer setups to advancing language service capabilities, this version release makes it easy for developers to build efficient, maintainable, and responsive applications.

Short list of features:	Dev Experience	Efficiency	Performance	UX
New reactive primitive - linkedSignal (experimental)	✅	✅		
Resource API (experimental)	✅	✅		
afterRenderEffect Function (experimental)	✅		✅	
Minor signal improvements	✅		✅	
@let template variable syntax	✅			
Incremental Hydration (experimental)	✅		✅	
Server Route Configuration (experimental)	✅		✅	✅
RouterOutlet data input	✅			
RouterLink directive enhancements	✅			
Default query params handling strategy	✅	✅		
Components become standalone by default	✅	✅	✅	
New useful migrations (injections, standalone API)	✅	✅		
Strict standalone flag	✅	✅		
Initializer provider functions	✅			
New angular diagnostics	✅	✅		
Hot module replacement for ng serve	✅	✅		
New features in Angular Language Service	✅	✅		
Typescript/Node.js support	✅			
Inferred Type Predicates (available in 5.5)	✅			
Control Flow Narrowing for Constant Indexed Accesses (available in 5.5)	✅			
Disallowed Nullish and Truthy Checks (available in 5.6)	✅			
Isolated modules	✅			

New reactive primitive - linkedSignal (experimental)

Dev Experience | Efficiency

Challenge:

Managing the dependencies and changes between reactive signals in Angular can be a challenging task in complex applications.

Although the existing reactive primitives, such as signal and computed, offer robust solutions for handling reactivity, they lack mechanisms for directly coupling reactive state changes with derived calculations that automatically reset when their source changes.

Developers are often required to add extra boilerplate to achieve this functionality, which increases the complexity of the code and reduces its maintainability.

Solution:

Angular 19 introduces linkedSignal, a new experimental reactive primitive designed to fill this gap.

linkedSignal is a writable signal that links its value to a source signal and recalculates itself based on a given computation function whenever the source signal changes.

Key features of linkedSignal:

- when the source signal changes, the value of linkedSignal automatically resets,
- the value of linkedSignal can be set manually using the set method, but it will revert to the computed value when the source signal updates,
- the computation logic is located inside linkedSignal, keeping dependencies clear and declarative while preserving all reactive programming features.

As shown in the example below, the set method of favoriteColorId allows the user to specify a chosen color ID (see onFavoriteColorChange). When the available colors are updated (see changeColorOptions), the value of favoriteColorId is recalculated. If the selected color exists in the new list, the signal value remains unchanged; otherwise, it defaults to null.

```
protected readonly colorOptions = signal<Color[]>([
  { id: 1, name: 'Red',},
  { id: 2, name: 'Green'},
  { id: 3, name: 'Blue'}
]);

protected favoriteColorId = linkedSignal<Color[], number | null>({
  source: this.colorOptions,
  computation: (source, previous) => {
    if(previous?.value) {
      return source.some(color => color.id === previous.value) ? previous.value : null;
    }
    return null;
  }
});


protected onFavoriteColorChange(colorId: number): void {
  this.favoriteColorId.set(colorId);
}


protected changeColorOptions(): void {
  this.colorOptions.set([
    { id: 1, name: 'Red' },
    { id: 4, name: 'Yellow' },
    { id: 5, name: 'Orange' }
  ])
}}
```

Benefits:

linkedSignal makes state management easier by linking updates directly to source signals, ensuring that dependencies remain up-to-date without explicit subscriptions and/or side effects. With the flexibility to override values as needed, it reduces boiler-plate, improves maintainability, and optimizes responsiveness.

Expert Opinion:

The Signal APIs have significantly changed how Angular applications are built, making them easier for new developers to learn, especially as the need for the RxJS library will become optional.

However, in some situations, there were limitations that required developers to find new patterns. For example, changing the value of signals that were not designed to be changed (such as signal inputs) presented challenges.

The introduction of linkedSignal addresses this issue. It allows you to create a new signal that reflects the state of another signal while still enabling you to modify the new signal's value independently. It ensures data consistency while keeping the original source signal unchanged.

To further enhance linkedSignal, the Angular team added the ability to derive the signal's state conditionally using a computation function. It is particularly useful for scenarios where you need to reset the signal's value or provide a value based on certain conditions.

~ Fanis Prodromou
Google Developer Expert

I believe the main idea is to gradually make Angular more aware of the state of our data, thereby eliminating the need for state management libraries to handle data updates. This will simplify Angular applications a lot, and I already use such patterns in different projects with great success so far.

~ Alain Chautard
Google Developer Expert

Resource API (experimental)

Dev Experience | Efficiency

Challenge:

Managing asynchronous operations in Angular applications often involves complex patterns such as subscriptions, manual state tracking, and additional boilerplate. Current practices, such as combining RxJS for data fetching with signals for state management, can result in fragmented and difficult-to-maintain code. Developers need a simpler and more integrated approach to handle async data reactively.

Solution:

Angular 19 introduces the experimental resource API, which integrates asynchronous operations into the signal graph. A resource is a declarative way to define, load, and manage async dependencies, providing both the value and status of an operation as signals.

This API simplifies the management of asynchronous workflows by coupling data fetching, state, and reactivity into a single coherent abstraction.

```
fruitId = signal<string>('apple-id-1');
fruitDetails = resource({
 request: this.fruitId,
 loader: async (params) => {
  const fruitId = params.request;
  const response = await fetch(`https://api.example.com/fruit/${fruitId}`,
 {signal: params.abortSignal});
  return await response.json() as Fruit;
 }
});
protected isFruitLoading = this.fruitDetails.isLoading;
protected fruit = this.fruitDetails.value;
protected error = this.fruitDetails.error;
protected updateFruit(name: string): void {
 this.fruitDetails.update((fruit) => (fruit ? {
  ...fruit,
  name,
 } : undefined))
}
protected reloadFruit(): void {
 this.fruitDetails.reload();
}
protected onFruitIdChange(fruitId: string): void {
 this.fruitId.set(fruitId);
}
```

The optional request parameter accepts the input signal that the asynchronous resource is associated with (in our example, it is fruitId, but it could just easily be a computed signal consisting of multiple values).

We also define the loader function, with which we asynchronously download the data (the function should return a promise). The created resource named fruitDetails allows us to do the following, among other things:

- access the current value signal (which also returns undefined if the resource is not available at the moment),
- access the status signal (one of: idle, error, loading, reloading, resolved, local),
- access extra signals like 'isLoading' or 'error',
- trigger the 'loader' function again (using the 'reload' method),
- update the local state of the resource (using the 'update' method).

The resource is automatically reloaded when the 'request' signal (in our case fruitId) changes. The loader is also triggered when the resource is first created.

What about RxJS Interop? Angular also provides an RxJS counterpart to the resource method called rxResource. In this case, the loader method returns Observable, but all other properties remain signals.

```
fruitDetails = rxResource({
  request: this.fruitId,
  loader: (params) => this.httpClient.get<Fruit>(`https://api.example.com/fruit/
  ${params.request}`)
})
```

Benefits:

The resource API streamlines the management of async operations in Angular by using an integrated, declarative approach. It reduces boilerplate, boosts maintainability, and keeps async data seamlessly reactive with the app state. By exposing signals for both value and status, resources make it easier to consistently handle loading, success, and error states.

Expert Opinion:

Before the introduction of the Resource API, developers often struggled to find effective patterns to make HTTP requests based on changes to signal values.

Using effect: While useful in many situations, the effect was not designed to handle HTTP requests. It could lead to issues like race conditions, where multiple requests might be made simultaneously, and unpredictable application states.

Using RxJS subjects: Another approach involved using RxJS subjects to manage the signal values. However, this combined RxJS and Signals, increasing complexity and making debugging more challenging.

The Resource API simplifies making HTTP requests based on signal changes.

Declarative requests: Define HTTP requests that automatically trigger when the source signal updates.

Improved reliability: The API is designed to handle race conditions and prevent unpredictable application states.

Built-in loading state: The Resource API provides a built-in loading state, making it easy to display loading indicators to users without needing custom logic.

~ Fanis Prodromou
Google Developer Expert

Both linkedSignal and the resource API are the cherry on top of all the features the Angular team shipped in the last releases! linkedSignal enables the possibility of always having a valid state based on another signal, and the resource API is truly impressive! Using it, we can handle async requests exquisitely, giving the user the best feedback possible and keeping the code base clean and easy to develop and maintain.

~ Luca Del Puppo
Google Developer Expert

Angular is building and extending its reactive primitive API for a good reason. The bet on signals and therefore embedding reactivity and state management into the framework will be essential for the later plan to change the underlying rendering strategy to fine-grained reactivity. So the goalpost is to enable zoneless and fine-grained reac-

tivity, but that requires developers to integrate reactive primitives into their code. As Angular is introducing more of these primitives, the developer experience for signal-based Angular applications becomes better with every major release.

~ Stefan Haas
Nx Champion

*In my opinion, the Angular team added linkedSignal and the resource API so that developers avoid using the **effect** function to update signals and Observables. Both Ben and Alex Rickabaugh visited Tech Stack Nation and said developers were using the **effect** function incorrectly. Ben advised against using the **effect** to perform asynchronous tasks, such as making HTTP requests. Moreover, developers who subscribe to the Observable in the **effect** often forget to use the cleanUp callback to unsubscribe from subscriptions, which leads to a memory leak.*

*The **linkedSignal** is the alternative to updating the signal in the effect callback. When the source receives a new value, it either returns a previous value or computes a new one. The result of the **linkedSignal** is a writable signal; therefore, developers can also directly set the signal with new values.*

*On the other hand, the **resource** API solves the scenario where developers perform asynchronous tasks in the effect callback. Both **resource** and **rxResource** enable developers to make HTTP requests when the parameters are updated. The result of a resource includes error, status, and values that can programmatically decide what to render in the HTML template. The resource is not limited to data retrieval; it can be used in other use cases, such as opening a WebSocket connection and streaming. After the **resource** API was introduced, there was httpResource that could query data from the server. However, I advise against using the API in production until it is stabilized.*

In the long run, they will help Angular developers write correct reactive codes in Angular applications.

~ Connie Leung
Google Developer Expert

AfterRenderEffect Function (experimental)

Performance | Dev Experience

Challenge:

Managing side effects that depend on the DOM state in Angular can be challenging, especially when those effects need to take place only after the DOM has been updated. While Angular provides afterRender and afterNextRender to schedule post-render callbacks, these APIs do not track signal dependencies, making them less suitable for scenarios where side effects need to be re-executed based on changes in reactive data. Developers must often resort to manual tracking, which produces complex and less maintainable code.

Solution:

Angular's afterRenderEffect is an experimental function designed to handle side effects that should only occur after the component has finished rendering and specific dependencies have changed. Unlike afterRender and afterNextRender, which always schedule post-render callbacks without dependency tracking, afterRenderEffect ties callback execution to specific reactive dependencies. This makes it ideal for ongoing post-render tasks tied to dynamic application state.

```
counter = signal(0);
constructor() {
  afterRenderEffect(() => {
    console.log('after render effect', this.counter());
  })
  afterRender(() => {
    console.log('after render', this.counter())
  })
}
```

In this example, afterRender schedules its callback to run after every render cycle regardless of any state changes. In contrast, afterRenderEffect runs its callback only when the counter signal changes. This ensures that the effect is selectively executed, based on relevant updates, cutting down on unnecessary operations and improving application efficiency.

Benefits:

The afterRenderEffect function offers a powerful tool for managing post-render side effects tied to reactive dependencies. By tracking dependencies and executing only on relevant changes, it simplifies code, reduces boilerplate, and avoids unnecessary executions.

This makes applications more efficient and easier to maintain, especially in scenarios with frequent state updates and DOM interactions. As an experimental feature, afterRenderEffect lays the groundwork for more sophisticated reactive workflows in the evolving Angular's ecosystem.

Minor signal improvements

Performance Dev Experience

Challenge:

Reactive programming in Angular previously faced limitations in handling effects and signal updates. The effect() function restricted signal writes, adding additional complexity in certain scenarios. At the same time its execution timing often caused issues with premature or delayed updates. Similarly, the toSignal function lacked flexibility in value comparison, forcing unnecessary updates due to a lack of custom equality logic.

Solution:

Angular 19 tackles these challenges with major updates to effect() and improvements to toSignal.

Removing the allowSignalWrites flag in effect() simplifies usage, letting developers set signals directly without extra restrictions. In addition, effect() execution is now integrated into Angular's change detection cycle, ensuring logical alignment with the component hierarchy. This eliminates timing issues and makes effect execution more reliable.

```
effect(
  () => {
    console.log(this.users());
  },
  //This flag is removed in the new version
  { allowSignalWrites: true }
);
```

Angular now supports custom equality functions in toSignal. Developers can set custom equality logic to trigger updates only when meaningful changes are detected, improving performance.

```
// Create a Subject to emit array values
const arraySubject$ = new Subject<number[]>();
// Define a custom equality function to compare arrays based on their content
const arraysAreEqual = (a: number[], b: number[]): boolean => {
  return a.length === b.length && a.every((value, index) => value === b[index]);
};
// Convert the Subject to a signal with a custom equality function
const arraySignal = toSignal(arraySubject$, {
  initialValue: [1, 2, 3],
  equals: arraysAreEqual, // Custom equality function for arrays
});
```

Benefits:

These updates improve reactive programming in Angular. The revamped effect() function simplifies workflows by removing unnecessary restrictions and ensuring correct execution timing.
In turn, the custom equality function in toSignal gives more control over updates, reducing unnecessary re-renders and boosting performance. Together, these changes simplify the code, make it easier to maintain, and improve the overall developer experience in Angular apps.

@let template variable syntax

Dev Experience

Challenge:

In previous versions of Angular, declaring variables within templates often meant using the ngIf directive and @if with the as keyword. This method had its limitations, especially when dealing with falsy values (such as 0, empty strings, null, undefined, and false), which would prevent content from rendering. For example:

```
<div *ngIf="userName$ | async as userName">
 <h1>Welcome, {{ userName }}</h1>
</div>
```

If userName were an empty string, nothing would be displayed.

Solution:

Angular introduces the @let block to simplify template logic by enabling variable declarations directly in the template. This prevents issues with falsy values and improves readability. The @let block lets developers declare variables directly in the template, making it easier to handle complex conditions and asynchronous data.

With @let:

```
<div>
 @let userName = (userName$ | async) ?? 'Guest';
 <h1>Welcome, {{ userName }}</h1>
</div>
```

This code handles falsy values like empty strings by providing a default value ('Guest').

Benefits:

With the @let block, developers can declare variables within the template, simplifying the overall logic. This improvement makes the code cleaner and more readable, especially when dealing with complex conditions and dynamic data. It also prevents falsy value issues in ngIf, making sure content appears as expected.

Incremental Hydration (experimental)

Performance UX

Challenge:

Modern web applications demand both high performance and interactivity, especially for server-rendered content. Hydrating an entire application on the client side can be resource-intensive, leading to slower load times and delayed interactivity. Developers need a way to selectively hydrate parts of the application as needed to optimize resource usage and improve the user experience.

Solution:

Angular introduces Incremental Hydration as an experimental feature, building on the foundations of defer blocks, deferrable views (introduced in v17), and event replay (v18). This feature hydrates server-rendered sections selectively, depending on predefined triggers. This gives developers control over component activation, enhancing both speed and user experience.

To enable Incremental Hydration, update the application configuration:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideClientHydration(
      withIncrementalHydration()
    )
    // other providers...
  ]
};
```

Incremental Hydration works in tandem with defer blocks. To use it, developers can add new hydration triggers to specific components:

```
@defer (hydrate on hover) {
  <app-hydrated-cmp />
}
```

Supported hydration triggers include:

- **idle:** Hydrate during idle time.
- **interaction:** Trigger hydration upon user interaction.
- **immediate:** Hydrate immediately.
- **timer(ms):** Hydrate after a specified delay.
- **hover:** Trigger hydration on hover.
- **viewport:** Hydrate when the component enters the viewport.
- **never:** Keep the component dehydrated indefinitely.
- **when {{ condition }}:** Conditional hydration based on a specified condition.

These triggers give developers fine-grained control over hydration timing and behavior, optimizing resources based on app needs and user actions.

Benefits:

Incremental Hydration improves load times and interactivity by activating components only when needed, reducing the initial payload and resource consumption. This allows large applications to run more efficiently by hydrating only the necessary components at startup.

In the same way, the flexibility provided by different triggers allows developers to tailor hydration strategies to specific use cases, making applications more efficient and responsive.

Expert Opinion:

Incremental hydration is an amazing technique that can significantly improve application performance while ensuring a smooth user experience (UX). Every feature has strengths and weaknesses, and it's important to use them effectively.

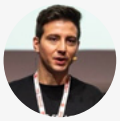
While Server-Side Rendering (SSR) and Incremental Hydration offer significant performance gains, they are not meant to replace Single-Page Applications (SPAs) completely. These techniques benefit client-facing applications, especially when it's essential to minimize the time it takes for users to see and interact with the page for the first time.

Furthermore, incremental hydration can improve performance without negatively impacting the SEO.

~ Fanis Prodromou
Google Developer Expert

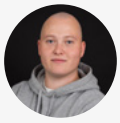
Incremental hydration has been missing for a while. Its predecessor, Server Side Render, was a nightmare to implement, and it has impacted the decisions between React and Angular for many teams in the past. Unfortunately, many chose React because it was already "ready."

But now it is here and straightforward to use.
Not all projects need this feature, so it will probably impact only a tiny part of the projects followed by the Angular community. But it can help open the doors for new meta frameworks based on Angular.



~ Luca Del Puppo
Google Developer Expert

Angular has been shipping essential features and is adding the missing pieces to the SSR story for Angular. For a long time, Angular's support for server-side rendered applications has been lacking and insufficient. But this is changing quickly, and incremental hydration is playing a big role in this story. That said, I believe this feature plays a big role for SSR apps, but most traditional Angular apps that use CSR will not benefit from this particular feature.



~ Stefan Haas
Nx Champion

Server Route Configuration (experimental)

Performance UX

Challenge:

Hybrid rendering in Angular applications requires a high degree of flexibility to optimize performance and user experience. Without a clear way to define route rendering—server-side, pre-rendered, or client-side—developers rely on complex setups and manual work. The absence of a clear API can lead to inefficiencies and higher maintenance efforts.

Solution:

Angular's experimental Server Route Configuration API lets developers set rendering modes for routes flexibly and declaratively. Using this API, developers can optimize the performance of their applications by choosing the most appropriate rendering strategy for each route, such as server-side rendering (SSR), static site generation (SSG), or client-side rendering (CSR).

Example configuration:

```
import { RenderMode, ServerRoute } from '@angular/ssr';
export const serverRouteConfig: ServerRoute[] = [
  { path: '/login', renderMode: RenderMode.Server },
  { path: '/fruits', renderMode: RenderMode.Prerender },
  { path: '/**', renderMode: RenderMode.Client }
];
```

In this configuration:

- **/login:** Uses SSR to ensure that the latest data is rendered on every request.
- **/fruits:** Uses SSG to generate content at build time for faster loading.
- **/*:** Defaults to CSR for all other routes, optimizing for interactivity.

In addition, the API supports dynamic path parameters in pre-render mode by defining functions to resolve path parameters:

```
export const serverRouteConfig2: ServerRoute[] = [
 {
  path: '/fruit/:id',
  renderMode: RenderMode.Prerender,
  async getPrerenderParams() {
   const fruitService = inject(FruitService);
   const fruitIds = await fruitService.getAllFruitIds();
   return fruitIds.map(id => ({ id }));
  },
 },
];
```

This example shows how the /fruit/:id route can dynamically generate static pages for all available fruit IDs, guaranteeing optimized performance for frequently accessed resources.

Benefits:

The Server Route Configuration API makes it easier to manage hybrid rendering in Angular applications. It reduces complexity and maintenance by allowing developers to declaratively define rendering modes for specific routes. The ability to dynamically resolve parameters for pre-rendered paths further increases flexibility, making it easier to build high-performance and scalable applications.

RouterOutlet data input

Dev Experience

Challenge:

In Angular applications, sharing data between parent and child components routed through a RouterOutlet often requires manual implementations, such as input bindings or service-based state management. These methods can be tedious and prone to errors, especially when dynamic updates are needed. Developers are looking for a more efficient way to pass and update data.

Solution:

Angular 19 introduces the routerOutletData input for RouterOutlet, simplifying the process of sending data from parent components to child components routed through the outlet. When routerOutletData is set, the data becomes accessible within the child components using the ROUTER_OUTLET_DATA token. This token takes advantage of Angular's Signal API. Changes in input data are dynamically reflected in child components, eliminating the need for manual assignments or extra code.

Parent Component:

```
<router-outlet [routerOutletData]="routerOutletData()" />
```

Child Component:

```
import { Signal, inject } from '@angular/core';
import { ROUTER_OUTLET_DATA } from '@angular/router';

export class ChildComponent {
  readonly routerOutletData: Signal<MyType> = inject(ROUTER_OUTLET_DATA);
}
```

With this configuration, the parent component can dynamically update routerOutletData, and the changes are automatically reflected in the child component. This increases the flexibility of data sharing within routed components by eliminating the need for static data mappings.

Benefits:

The routerOutletData input streamlines data communication between parent and child components routed through RouterOutlet. Using Angular's reactive signaling type, it makes dynamic updates possible, reducing the need for boilerplate code and manual state synchronization.

RouterLink directive enhancements

Dev Experience

Challenge:

Configuring route navigation in Angular often requires the use of multiple inputs to RouterLink, such as query parameters, fragment, and other options. When it comes to complex navigation scenarios, this approach can become cumbersome and can cause errors.

Solution:
As of Angular version 18.1, the RouterLink directive now accepts anUrlTree object as its input. By encapsulating route options like query parameters and paths into a single UrlTree object, this update streamlines the navigation process.

Example:

```
<a [routerLink]="homeUrlTree">Home</a>
```

Using a UrlTree allows developers to define navigation configurations in one place, which helps to clarify and maintain the code. However, to avoid conflicts, Angular enforces strict rules: if an UrlTree object is passed to RouterLink, additional inputs such as queryParams or fragments cannot be used. If such inputs are provided alongside an UrlTree, Angular will throw an error:

```
Cannot configure queryParams or fragment when using a UrlTree
as the routerLink input value.
```

It guarantees a consistent navigation setup and eliminates any confusion in route handling.

Benefits:
By allowing RouterLink to accept UrlTree objects, Angular gives developers a cleaner and more flexible way to define navigation logic. This improvement simplifies code, enhances readability, and keeps navigation configurations in one place. Additionally, the clear error messages help developers avoid common pitfalls, making applications more robust and easier to maintain.

Default query params handling strategy

Dev Experience | Efficiency

Challenge:

Configuring query parameter handling strategies individually for each navigation can become repetitive and susceptible to errors. Developers often need a consistent way to manage query parameters across the entire application without having to manually configure each route.

Solution:

Angular now allows developers to globally set a default query parameter handling strategy in the provideRouter() configuration. The result is the elimination of repetitive per-route configurations, which streamlines navigation logic and improves maintainability.

Example:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withRouterConfig({ defaultQueryParamsHandling: 'preserve' }))
  ]
};
```

Angular's default strategy is replace, but developers can choose between:

- preserve: It retains the current query parameters during navigation.
- merge: It combines the new query parameters with the existing ones.

This flexibility ensures that query parameter handling meets application-specific requirements, reducing boilerplate and potential inconsistencies.

Benefits:

With a global query parameter handling strategy, Angular reduces repetitive code, simplifies navigation, and ensures consistency across the app. This feature increases the productivity of the developer while maintaining the flexibility for advanced use cases.

Components become standalone by default

Performance | Dev Experience | Efficiency

Challenge:

The legacy approach to Angular components often relies on explicit module declarations, which can add complexity and create barriers for new developers. Standalone components were introduced in v14, but not by default, creating inconsistencies and additional setup for developers wishing to use them.

Solution:

With Angular v19, standalone: true becomes the default for components, directives, and pipes. Angular becomes more intuitive and accessible with this change, as explicit module declarations are no longer needed in most cases.

Example:

```
@Component({
  imports: [],
  selector: 'home',
  template: './home-component.html'
  // standalone in Angular 19!
})
export class HomeComponent { }
```

For existing projects, an automated migration during the ng update will adjust standalone flag settings as needed, ensuring compatibility and facilitating a smooth transition.

Benefits:

Making standalone components the default makes Angular's mental model simpler, especially for new developers. In keeping with Angular's goal of streamlining development workflows, this enhancement better supports features such as lazy loading and component composition, and reduces boilerplate. Automated migration tools make sure that existing projects can seamlessly adopt the new defaults, minimizing disruption while providing a modern development experience.

New useful migrations (injections, standalone API)

Dev Experience | Efficiency

Challenge:

Keeping Angular projects up to date with the latest features and best practices can be a daunting task. Especially for large or legacy codebases, manually refactoring code to take advantage of new APIs, improve performance, or simplify the architecture is a significant maintenance burden.

Solution:

A set of automated migrations in Angular helps developers transition smoothly to modern features. These migrations are designed to address common refactoring needs, reduce manual effort, and ensure a problem-free upgrade process.

Shift from constructor-based injection to inject() function:

```
ng g @angular/core:inject
```

This migration simplifies the code by replacing the constructor syntax with a more streamlined approach:

```
// before
constructor(private productService: ProductService) {}

// after
private productService = inject(ProductService);
```

After migration, you may encounter compilation issues, especially in tests that directly create instances of injectables. The migration utility provides several options to customize the process, such as how to handle abstract classes, how to maintain backward-compatible constructors, and how to manage nullable settings to ensure a smooth transition without breaking existing code.

Lazy loading of standalone components in routing configuration:

```
ng g @angular/core:route-lazy-loading
```

This migration changes direct references to components into dynamic imports (lazy loading):

```
// before
{
  path: 'products',
  component: ProductsComponent
}
// after
{
  path: 'products',
  loadComponent: () => import('./products.component').then(m => m.ProductsComponent)
}
```

The migrations for signal inputs, outputs, and queries make Angular more responsive by converting traditional input properties, event outputs, and query fields to their modern, signal-based counterparts. This improves code efficiency and maintainability.

```
ng generate @angular/core:signal-input-migration
ng generate @angular/core:output-migration
ng generate @angular/core:signal-queries-migration
```

Example:

```
export class MyComponent {
  // before
  @Input() name: string|undefined = undefined;
  @Output() someChange = new EventEmitter<string>();
  @ContentChild('someRef') ref: ElementRef|undefined = undefined;
  // after
  name = input<string>();
  someChange = output<string>();
  ref = contentChild<ElementRef>('someRef');
}
```

Benefits:

These migrations dramatically increase developer productivity by simplifying the adoption of the latest Angular features, such as the inject() function and standalone APIs. By automating refactoring tasks, they reduce errors, ensure code consistency, and allow developers to focus on building modern, maintainable applications. Developers can focus on delivering new features and improving the user experience instead of spending time on manual updates.

Strict standalone flag

Dev Experience **Efficiency**

Challenge:

Using Angular's standalone components, directives, and pipes consistently can be hard in projects migrating from older versions or where developers are unfamiliar with standalone conventions. Without enforcement, non-standalone components can unintentionally proliferate, leading to mixed patterns and increased complexity in maintaining the codebase.

Solution:

Angular introduces the strictStandalone flag in angularCompilerOptions to enforce the exclusive use of standalone components, directives, and pipes. By default, this flag is set to false, to allow for gradual adoption. When enabled, this flag prohibits any component, directive, or pipe from being explicitly marked as non-standalone. This ensures

alignment with Angular's default standalone architecture, which was introduced in version 19.

Configuration Example:

```
{
  "angularCompilerOptions": {
    "strictStandalone": true
  }
}
```

Enabling this flag ensures that only standalone entities are used, reinforcing Angular's evolution toward a modular and simplified framework. Violations will result in a compiler error:

```
[ERROR] TS-992023: Only standalone components/directives are allowed when
'strictStandalone' is enabled. [plugin angular-compiler]
```

Benefits:

The strictStandalone flag simplifies project architecture by enforcing consistent usage of standalone components, directives, and pipes. This reduces developer cognitive load, eliminates legacy patterns, and aligns with Angular's modern design principles. Enforcing standalone usage keeps the codebase cleaner, easier to manage, and ready for advanced features like lazy coding or modular composition.

Initializer provider functions

Dev Experience

Challenge:

Setting up initializers in Angular has traditionally relied on using the APP_INITIALIZER, ENVIRONMENT_INITIALIZER, and PLATFORM_INITIALIZER tokens. While functional, this approach often results in verbose and less readable code, making the configuration process cumbersome, especially for complex projects. Developers need a simpler, more intuitive way to manage application, environment, and platform initialization.

Solution:

Angular v19 introduces new helper functions:

- provideAppInitializer
- provideEnvironmentInitializer
- providePlatformInitializer

These functions serve as syntactic sugar that simplifies the setup process for application, environment, and platform-level initializers. By replacing traditional tokens, they provide a cleaner, more readable, and more intuitive way to configure initialization logic.

Example:

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideAppInitializer(() => {
      console.log('app initialized');
    })
  ]
};
```

In addition, Angular v19 includes a migration tool to help developers convert existing initializers to this new format. This automation minimizes manual effort and facilitates a smooth adoption of the updated approach.

Benefits:

New initializer provider features improve the readability and maintainability of initializer setups by reducing boilerplate and simplifying configuration.

New angular diagnostics

Dev Experience | Efficiency

Challenge:

Maintaining clean, efficient, and bug-free Angular applications is a challenge, especially when subtle issues like unused imports or incorrect event bindings can go unnoticed during development. Traditional diagnostics can miss these nuanced problems, leading to technical debt and inefficient applications over time.

Solution:

Angular's Extended Diagnostics provide real-time code checks that go beyond standard errors and warnings to identify potential problems and enforce best practices. In Angular 19, two new diagnostics extend this capability:

Uninvoked Functions: Flags instances where a function is used in an event binding but isn't called due to missing parentheses in the template. This helps ensure that functions in event bindings are executed correctly, preventing runtime errors.

Example:

```
<!-- Incorrect -->
<button (click)="onClick">Click me</button>
<!-- Correct -->
<button (click)="onClick()">Click me</button>
```

Unused Standalone Imports:

Detects standalone components, directives, or pipes that are imported but not used in the application. This maintains a clean codebase by prompting developers to remove unused imports.

Example:

```
// Incorrect
@Component({
 imports: [UnusedComponent],
 template: '<div>hello</div>'
})
export class MyComponent {}
// Correct
@Component({
 imports: [],
 template: '<div>hello</div>'
})
export class MyComponent {}
```

We encourage you to explore and test other Angular diagnostics documented in the official Angular documentation to further refine your projects and uphold strong code quality.

Benefits:

These new diagnostics improve the quality of your code by catching subtle problems early in the development process, reducing technical debt and run-time errors. They ensure adherence to Angular's best practices, resulting in cleaner, more maintainable applications.

Hot module replacement for ng serve

Dev Experience **Efficiency**

Challenge:

Developers often face workflow disruptions when making changes to styles or templates, because the Angular CLI traditionally requires a full page refresh for updates to take effect. This process not only slows down the development cycle, but also results in the loss of application state, breaking the flow of iterative design and debugging.

Solution:
Angular v19 introduces built-in support for Hot Module Replacement (HMR) for styles and experimental support for template HMR, greatly improving the development expe-

rience. With HMR, any changes made to styles or templates are compiled and patched into the application in real time, without requiring a page refresh. This ensures that updates are applied immediately while preserving the state of the application, resulting in a faster and more seamless development workflow.

The HMR for styles is enabled by default. Simply modify the styles of a component, save the changes, and see them immediately reflected in the browser without reloading the page.

To enable HMR for templates, use the following command:

```
NG_HMR_TEMPLATES=1 ng serve
```

To disable HMR for development servers, use:

```
ng serve --no-hmr
```

Benefits:

Hot module replacement updates styles and templates in real time, cutting development time without losing application state. This feature increases developer productivity by maintaining an uninterrupted workflow, speeding up iterations, and promoting a more efficient debugging process.

New features in Angular Language Service

Dev Experience **Efficiency**

Challenge:

Angular developers rely heavily on the Angular Language Service (ALS) to guarantee productive and bug-free development. However, keeping ALS up to date with the latest features and supporting advanced functionalities has been a challenge. Developers often lack auto-completion for unimported directives, and may struggle with unused imports or migrating to newer APIs like signals.

Solution:

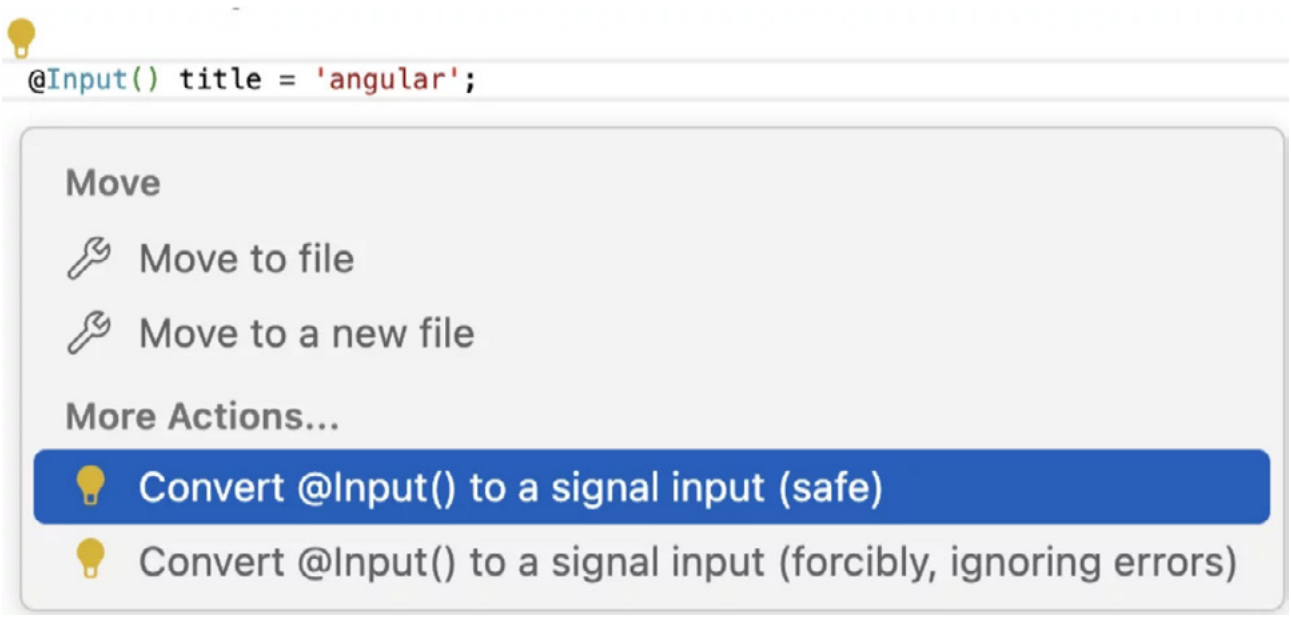
The latest updates to the Angular Language Service bring a host of new features to enhance the development experience:

Diagnostics for Unused Standalone Imports: It automatically flags standalone components, directives, or pipes that are imported but not used, helping developers maintain a clean and efficient codebase.

Support for Signal Migrations: It provides refactoring tools for migrating @Input properties to signal inputs and updating queries to signal-based APIs, reducing the effort required to adopt Angular's modern features.

In-Template Autocompletion: It enables autocompletion for all directives, even those that haven't yet been imported, speeding up template development.

Refactoring Schematics: Integrates useful refactoring schematics directly into supported IDEs, streamlining the transition to updated Angular APIs and features.



Benefits:

The enhanced Angular Language Service significantly boosts developer productivity by providing real-time insights, intelligent diagnostics, and automated refactoring tools. With these improvements, developers can now integrate the latest Angular features more easily while ensuring clean, efficient code.

Auto-completion for non-imported directives further simplifies template authoring, reducing errors and saving time. By integrating powerful tools into IDEs, the updated ALS fosters a seamless and efficient development experience.

Typescript/Node.js support

Angular v18.1 introduces support for TypeScript 5.5, while Angular v19.0 extends compatibility to TypeScript 5.6 and removes support for versions prior to 5.5.

Highlighted below are some notable feature updates.

Inferred Type Predicates (available in 5.5)

TypeScript automatically infers type predicates, narrowing types in places where we previously had to explicitly define predicate.

```
const availableProducts = productIds
  .map(id => productCatalog.get(id))
  .filter(product => product !== undefined);


/* TypeScript now knows availableProducts are no longer considered as possibly undefined */
availableProducts.forEach(product => product.displayDetails());
```

Control Flow Narrowing for Constant Indexed Accesses (available in 5.5)

TypeScript can now narrow expressions such as obj[key] when both obj and key are actually constants.

```
function logUpperCase(key: string, dictionary:Record<string, unknown>): void {
  if(typeof dictionary[key] === 'string') {
      /* valid since ts 5.5 */
      console.log(dictionary[key].toUpperCase());
  }
}
```

Disallowed Nullish and Truthy Checks (available in 5.6)

Typescript will throw an error when truthy or nullish checks always evaluate to true (which is correct in terms of JS syntax, but usually implies some logical error).

The following examples will trigger an error:

```
if(/^[a-z]+$/) {
 /* missing .test(value) call, regex itself is always truthy  */
}


if (x => 0) {
   /* "x => 0" is an arrow function, always truthy */
}
```

Isolated modules

Angular 18.2 introduced support for TypeScript's isolatedModules, which can improve production build times by up to 10% by allowing code transpilation through the bundle. This optimizes TypeScript constructs and reduces Babel-based passes.

To enable isolatedModules support in your Angular project, update your TypeScript configuration (tsconfig.json) as follows:

```
"compilerOptions": {
  "isolatedModules": true
}
```

It applies a few extra restrictions, such as no cross-file type inference, allowing only const enums to be exported, and forcing explicit declarations of type-only exports (using import type syntax).

Without isolatedModules, full type checking is performed for the entire codebase during the compilation process. In contrast, with isolatedModules enabled, each file is compiled independently, and certain cross-file type analysis is skipped.

Template improvements

NgComponentOutlet

httpResource

Angular v20

release date: **05.2025**

Angular 20 marks a more measured release compared to its predecessors. Instead of adding plenty of new features, it aims to improve the developer experience and make the framework more reliable.

This chapter highlights how the update focuses on building on the groundwork laid by previous versions. The update aims to make the framework more consistent and intuitive for developers.

While it may not be packed with headline-grabbing additions, Angular 20 shows maturity by prioritizing long-term maintainability, smoother workflows, and the polishing of existing capabilities. This release is a testament to Angular's commitment to stability and usability as the framework continues to evolve.

Short list of features:	Dev Experience	Efficiency	Performance	UX
New Features of NgComponentOutlet	✅	✅		
SubPath in multilingual applications	✅	✅		
HttpResource (experimental)	✅	✅		
Default value in resource	✅			
Missing structural directives import detection	✅			
Bindings and directives support for dynamic components	✅			
Asynchronous redirect function	✅			
Supporting new features in templates	✅			
Next step towards zoneless	✅		✅	
Keepalive support for fetch requests	✅	✅		
Type checking support for host bindings	✅			
Extended diagnostics for nullish coalescing and uninvoked track function	✅			
Stop producing ng-reflect attributes	✅			

New Features of NgComponentOutlet

Dev Experience | Efficiency

Challenge:

NgComponentOutlet is a dynamic way to instantiate and render components within templates. It works like RouterOutlet, but without needing router configuration, which makes it great for situations involving dynamically loaded components or plugin-like architectures.

While it's been very useful, it has traditionally required manual setup and configuration, which is cumbersome.

```
@Component({
  template: `
    <ng-container #container />
  `
})
export class AppComponent {
  private _cmpRef?: ComponentRef<MyComponent>;
  private readonly _container = viewChild('container', {
    read: ViewContainerRef
  });

  createComponent(title: string): void {
    this.destroyComponent(); // Otherwise it would create second instance

    this._cmpRef = this._container()?.createComponent(MyComponent);
    this._cmpRef?.setInput('title', title);
  }

  destroyComponent(): void {
    this._container()?.clear();
  }
}
```

Solution:

Angular introduced a new API for NgComponentOutlet, making it easier to use by handling manual configuration internally. This upgraded API is controlled through dedicated inputs:

- The ngComponentOutlet is used to specify the component type.
- The ngComponentOutletInputs lets you pass input values straight to the component.
- The ngComponentOutletContent is used to define the content nodes for content projection.
- The ngComponentOutletInjector passes a custom injector to the component that's created dynamically.

This enhancement reduces the need for manual setup by making it easier to create dynamic components.

```
@Component({
  template: `
    <ng-container
      [ngComponentOutlet]="myComponent"
      [ngComponentOutletInputs]="myComponentInput()"
      [ngComponentOutletContent]="contentNodes()"
      [ngComponentOutletInjector]="myInjector"
      #outlet="ngComponentOutlet"
    />

    <ng-template #emptyState>
      <app-empty-state />
    </ng-template>
  `,
})
export class AppComponent {
  private readonly _vcr = inject(ViewContainerRef);
  private readonly _injector = inject(Injector);

  protected myComponent: Type<MyComponent> | null = null;
  protected readonly myComponentInput = signal({ title: 'Title' });

  private readonly _emptyStateTemplate =
    viewChild<TemplateRef<unknown>>('emptyState');

  readonly contentNodes = computed(() => {
    if (!this._emptyStateTemplate()) return [];

    return [this.vcr.createEmbededView(this._emptyStateTemplate()!).rootNodes];
  });

  readonly myInjector = Injector.create({
    providers: [{ provide: MyService, deps: [] }],
    parent: this.injector,
  });

  createComponent(): void {
    this.myComponent = MyComponent;
  }

  destroyComponent(): void {
    this.myComponent = null;
  }
}
```

Benefits:

This approach lets you define dynamic components in a clean, declarative way. Angular takes care of configuration and interactivity for you, removing the need for manual component destruction and recreation. NgComponentOutlet automatically regenerates the dynamic component whenever its inputs are updated.

A simple way to listen to dynamic component outputs would be the icing on the cake. Right now, you can achieve this using the componentInstance property of NgComponentOutlet (which, as expected, returns an instance of the dynamic component) and subscribing to its outputs.

SubPath in multilingual applications

Dev Experience | Efficiency

Challenge:

Since there are separate baseHrefs, a multilingual application must be built separately for each language. This results in multiple sets of files and a number of builds corresponding to the supported languages. Each set needs its own place on the server, which requires more setup and uses more storage space as it results in copies of the files.

The URL doesn't change, and without extra logic, it may not always show the selected language, which could affect SEO. Also, if you change the language while the application is running, you need to reload the application. This is a result of the fact that changing baseHref modifies the entire base path of the URL.

Solution:

Since version 19.1 of Angular, you can use subPath. This feature automatically defines URL segments for different languages, organizes generated files, and improves both the performance of the application and the experience for developers.

```
{
  "my-app": {
    "i18n": {
      "locales": {
        "pl": {
          "translation": "src/locale/pl.json",
          "subPath": "pl"
        },
        "en": {
          "translation": "src/locale/en.json",
          "subPath": "en"
        }
      }
    }
  }
}
```

Benefits:

This way, you only have to build the application once with a single baseHref. The Router handles multilingual support by examining URL subpaths and loading the right resources, making the process more efficient. All application files are stored together in one directory.

The server handles different languages by analyzing the URL and redirecting to a single index.html. Each language is reflected in the URL, allowing search engines to recognize separate pages for each language. The Router also lets you switch languages and load translations without having to reload the application.

HttpResource (experimental)

Dev Experience | Efficiency

Challenge:

The Angular ecosystem is changing: more and more areas are adopting signal-based APIs. Until now, there was no way to make HTTP requests using signals — despite HTTP being a key feature. That's changing with the introduction of HttpResource, a new signal-based API for HTTP requests. The classic method uses HttpClient, which relies on manual calls and an imperative programming style — essentially, you tell the program how to perform each step.

Solution:

The introduction of httpResource makes managing HTTP requests more declarative and reactive. In short, it automatically reloads and makes a new request each time the source signal changes.

```
const userResource = httpResource<User>(() => `/users/${userId()}`);
```

If you need more fine-grained control over the request, you can pass a function that returns an HttpResourceRequest object. In addition to the URL property, it can specify and react to options such as the method, parameters, or headers.

You can define a method when an API uses a POST request to fetch data, as it allows for more detailed queries to be set in the request body. HttpResource is supposed to be used to retrieve data. If you need to mutate data on the server, stick with HttpClient.

```
const usersResource = httpResource<User[]>(() => ({
  url: `/markets/${marketId()}/users`,
  params: {
    sort: sortOption(),
    search: searchTerm(),
  },
  headers: {
    'X-Special': myHeader(),
  },
}));
```

The second, optional parameter is an options object that lets you define:

- injector - use this in case you're using httpResource outside the injection context.
- defaultValue is for use in idle, loading, and error states.
- equal is a function that determines if two values are the same.
- parse is an important function that takes the response and changes it before it's sent to the resource.

By default, httpResource returns and parses the response as JSON, but you can customize it to return other types using:

- httpResource.text() function will return the text value.
- httpResource.blob() will give you a Blob object back.
- httpResource.arrayBuffer() will return an ArrayBuffer.

It's built on the Resource API, which was just introduced recently. So, along with standard features like value, status, error, and isLoading, it also shows dedicated signals for response metadata, like headers, statusCode, and progress.

This solution still uses HttpClient and its underlying architecture, so you can use interceptors and testing utilities without making any changes.

Benefits:

This approach makes the code simpler, improves state management, and does away with having to refresh the data manually. The feature is still experimental, but it has a lot of potential to make asynchronous operations in Angular applications more efficient.

Expert Opinion:

I see httpResource as the built-in replacement of httpClient in the long run. This is exciting because such resources enable developers to get data from a server without using any RxJs code, which means no more subscription and unsubscription.

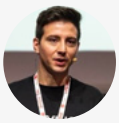
~ Alain Chautard
Google Developer Expert

For a long time, we've tried to move our extended component logic into a Service. This usually makes things easier to read and test. But when it comes to making HTTP calls, that service often just acts as a middleman, passing the request along to another service that actually handles the HTTP call. There's not much benefit there. Plus, we still end up with a bunch of boilerplate code in our components—like having to manually subscribe/unsubscribe or remember to use the async pipe.

Now, with httpResource, that same Service can totally act as a local-state manager without the boilerplate code, and you don't have to worry about memory leaks. On top of that, it really pushes you towards that declarative style by utilizing other awesome Signal APIs.

~ Fanis Prodromou
Google Developer Expert

httpResource simplifies HTTP requests to retrieve data based on different parameters. It improves the code and introduces an easy way to create HTTP requests in Angular. Lastly, in this way, it's easier to react to changes and re-fetch data if needed. Compared to the old RxJS approach with observables, this way is also simpler for junior developers to understand and learn.
I think the Angular team is doing a great job of reducing the cognitive load so that new developers can work with the framework and be productive as soon as possible.



~ Luca Del Puppo
Google Developer Expert

I see httpResource as a step toward simplifying HTTP calls in Angular. Previously, we had to manually handle things like loading states, reloads, and canceling pending requests. With httpResource, much of that comes "for free."
It also serves as an important abstraction that hides the HttpClient's Observable-based API, making the potential transition to a future RxJS-less Angular much smoother. And of course, it exposes a signal-based API that integrates seamlessly with other new reactive signal primitives.

~ Dmytro Mezhenskyi
Google Developer Expert

Default value in resource

Dev Experience

Challenge:

When you use the resource() function, the resource's value is in an unknown state until it's fully loaded. In that case, it defaults to undefined. This means that developers have to deal with potential undefined values in their code, which complicates state management and requires more type checks. If we don't address this kind of ambiguity, it can make the code more complex and even result in runtime errors.

Solution:

The new defaultValue option in resource() and rxResource() lets developers set a fallback value for resources that aren't currently loaded. Instead of returning undefined, .value() now gives you the predefined default, so you don't have to keep checking for undefined manually.

```
const usersResource = resource({
  defaultValue: [],
  request: () => ({ marketId: marketId() }),
  loader: ({ request }) => fetchUsers(request.marketId),
});


const users = usersResource.value();
```

Benefits:

This improvement makes state management easier by ensuring resources always return a predictable value, even during loading. It cuts down on all that boilerplate code that gets used for undefined handling, and improves type safety by guaranteeing non-null returns. With explicit default states, developers can now write cleaner, more reliable code.

Missing structural directives import detection

Dev Experience

Challenge:

In older versions of Angular, the compiler would flag missing imports for built-in structural directives, like ngIf or ngFor. However, it would silently fail if developers missed imports for custom structural directives.

This lack of warning made debugging tricky, especially in standalone component migrations, where missing dependencies were hard to pinpoint. Developers had to go through the hassle of manually verifying imports, which led to a fair amount of frustration and wasted time.

Solution:

This update makes sure that everything works the same way across all types of directives, so it won't lead to any silent failures. The improved error messages help developers quickly find and fix import issues during development.

Benefits:

The updated version solves the silent failures by catching missing imports early on, saving developers time and frustration. It also makes it easier to adopt standalone components because it shows you the dependency issues right away. Best of all, it makes custom structural directives work with Angular's built-in directives, so development is more intuitive and reliable.

Bindings and directives support for dynamic components

Dev Experience

Challenge:

Creating dynamic components is a useful pattern in many situations. A previous chapter introduced the new capabilities of NgComponentOutlet. However, there are situations where an imperative approach is more convenient.

Keep in mind, though, that this approach has a less developer-friendly API and may lead to unexpected behavior.

Solution:

Angular has introduced a new API that makes it easier for developers to define input, output, and two-way bindings.This API not only simplifies how you write bindings but also lets you attach directives to components more clearly and cleanly.

```
@Component({
 ...
})
export class AppWarningComponent {
  readonly canClose = input.required<boolean>();
  readonly isExpanded = model<boolean>();
  readonly close = output<boolean>();
}


@Component({
  template: ` <ng-container #container></ng-container> `,
})
export class AppComponent {
  readonly vcr = viewChild.required('container', { read: ViewContainerRef });
  readonly canClose = signal(true)
  readonly isExpanded = signal(true)


  createWarningComponent(): void {
    this.vcr().createComponent(AppWarningComponent, {
      bindings: [
        inputBinding('canClose', this.canClose),
        twoWayBinding('isExpanded', this.isExpanded),
        outputBinding<boolean>('close', (isConfirmed) => console.log(isConfirmed))
      ],
      directives: [
        FocusTrap,
        {
          type: ThemeDirective,
          bindings: [inputBinding('theme', () => 'warning')]
        }
      ]
    })
  }
}
```

Benefits:

Creating dynamic components has become much easier. The new API introduces powerful features, such as handling custom component events through output bindings, as well as the ability to apply host directives and define bindings for specific directives.

These bindings use the same mechanism as template bindings to ensure consistent behavior. The API is also tree-shakeable, meaning unused code will not be included in the final bundle.

Expert Opinion:

Creating dynamic components in Angular is not a new feature. Lots of companies have already embraced this to support their user flows. Let's, however, take a 10,000-foot view of what we can achieve by manually creating a component:

- *We can encapsulate the styles.*
- *We can apply component communication by providing inputs, outputs, and two-way data binding.*
- *We can host different directives, which enforces separation of concerns and a DRY approach.*

Let's now take a 10,000-foot view of what we could do with dynamic components:

- *We could encapsulate the styles.*
- *We could provide the inputs.*

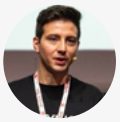
While we could provide the inputs, their values behaved more like attributes—like snapshots. It was "fire and forget," and no matter what changes the parent component had, the dynamic components were unable to get the new input value.

The Angular team knows this difference and understands how imbalanced this is, and they decided to surprise us once more. The fact that we can provide Inputs (and I mean live Inputs that change over time), Outputs, Two-way data binding, and even Directives in ***createComponent()*** *method is close enough to the list of features we have when creating a component manually. Companies can now extend their user flows without that many compromises.*

~ Fanis Prodromou
Google Developer Expert

Dynamically created components can be a powerful feature. Have you had a chance to utilize them in an interesting way? What do you think about the new possibilities of the NgComponentOutlet directive and the new API of the createComponent() method supporting inputs and outputs bindings and applying host directives to dynamic components?

I remember when, in one of my previous jobs, I had the chance to build a complex system that built a dynamic UI based on a configuration. It was Angular 7/8, and it hasn't been easy, almost a nightmare! With the new API `createComponent()`, doing these things is pretty straightforward. The API is very structured and guides you in smoothly building dynamic components. Building a dynamic component is always tricky, but the games have been making almost a joke with this new API!
If I considered my job with Angular 7/8 and the possibilities available now, I would like to go back and rewrite my code using these new ways to simplify everything!



~ Luca Del Puppo
Google Developer Expert

Yes, I've had many use cases where I needed to create components dynamically. One common scenario is dynamic forms, where the form is built from a JSON config.
I was really excited to see the improvements in Angular 20 that make dynamic component creation much simpler - it's actually one of my favorite features of the Angular 20 release :)

Unfortunately, NgComponentOutlet itself didn't receive any updates in Angular 20, but I believe the new binding capabilities will soon unlock long-awaited features for it, such as handling component outputs and two-way bindings.

~ Dmytro Mezhensky
Google Developer Expert

Asynchronous redirect function

Dev Experience

Challenge:

In most cases,the static redirection defined in the route configuration is sufficient to guide users to the correct path. However, a more dynamic approach is necessary in situations where the redirect destination depends on data that must be fetched asynchronously.

This could involve making a call to a remote server or retrieving configuration details before determining where the user should be redirected.

Solution:

A RedirectFunction now returns a value of type MaybeAsync. This means that, in addition to returning a path string or UrlTree, it can also return an Observable or Promise that resolves to one of these types. As a result, handling asynchronous redirection scenarios is more flexible.

```
export const ROUTES: Routes = [
 ...,
 {
  path: '**',
  redirectTo: () => {
   const router = inject(Router);
   const authService = inject(AuthService);


   return authService.isAuthorized$.pipe(
    map((isAuthorized) =>
     router.createUrlTree([`/${isAuthorized ? 'home' : 'login'}`]),
    ),
   );
  },
 },
];
```

Benefits:

The redirect function can now send you to a different route, where the target route is created based on asynchronous data. This makes it possible to use more advanced routing scenarios that require the destination to be found at runtime.

Supporting new features in templates

Dev Experience

Challenge:

Templates in Angular have always had limited support for JavaScript language features. Developers often had problems when they tried to use basic expressions directly in the template. Because of this, they had to move logic into the component class. This made the code less readable and more fragmented.

Solution:

Angular keeps adding features to templates that are more convenient and powerful. This makes templates more expressive and better matches modern JavaScript capabilities.

Template literals

At the moment, to put a variable into text, you have to use the regular + symbol to join strings together. This approach makes the code harder to read and write. Supporting template literals lets you use of modern string interpolation to create more readable and concise expressions.

```
<p>{{ `${user().name} is ${user().age} years old` }}</p>
```

Tagged template literals

Angular also supports tagged template literals within expressions. This feature makes it ossible to add custom processing functions, such as for localization or formatting. As a result, Angular templates become more expressive and flexible. The addition aligns Angular more closely with modern JavaScript syntax and developer expectations.

```
<p>{{ greet`Hello ${name()}!` }}</p>


greet(strings: TemplateStringsArray, name: string) {
  // strings contains ['Hello', '!']
  return `${strings[0]} ${name}${strings[1]}`;
 }
```

Exponential operator

This improvement lets developers do exponentiation calculations directly in the template, eliminating the need for complicated workarounds or moving logic to the component class. It simplifies template logic, so you don't need to use extra functions or computations.

```
<p>This cube is {{ sideLength() ** 3 }} cubic meters</p>
```

In keyword

The "in" keyword in binary expressions checks if an object has certain properties before using those properties in expressions. It's an easy way to do this instead of creating methods in the component class to perform this check and calling them in the template.

```
@if ('foo' in bar) {
<p>Property foo exists in object bar</p>
} @else {
<p>Property foo doesn't exist in object bar</p>
}


<div [class.foo]="'foo' in bar"></div>
```

Void operator

You can use the void operator to ignore the result of an expression. In Angular it might be useful if a bounded listener may return false and unintentionally prevent the default event behavior.

```
@Directive({
  host: {
    '(mousedown)': 'void handleMouseDown()'
  }
})
```

Benefits:

As Angular supports more and more features like these in templates, it is making big steps toward the goal of letting template expressions be plain TypeScript expressions. This reduces the mental effort for developers and brings greater consistency between the logic of the components and the syntax of the template.

Next step towards zoneless

Performance Dev Experience

Challenge:

Zone.js is a package that helps Angular track asynchronous operations to trigger change detection. It's been with Angular since the very beginning but it can slow down an application and might trigger change detection more often than necessary. Also, the initial bundle has more code, which affects how fast the application loads.

Solution:

The Angular Team is working on a modern, zoneless change detection mechanism to replace Zone.js. Angular 18 introduced the first experimental version, which was described in a previous chapter. It could be enabled using the provideExperimentalZonelessChangeDetection function.

Version 20 upgrades its status from experimental to developer preview which means we are one step closer to the final version. This version's function to enable it is named provideZonelessChangeDetection. Additionally, when creating an application using Angular CLI, a new step asks if you want to create one without Zone.js.

Benefits:

Eliminating Zone.js offers several benefits, including improved performance, reduced overhead, and simplified debugging. The upgrade to developer preview status means that the work is going well, and a stable version will be available sooner rather than later. If your application uses ChangeDetection.OnPush, it's zoneless-compatible, so you can try it out.

Keepalive support for fetch requests

Dev Experience | Efficiency

Challenge:

In web applications, it's common to perform asynchronous operations during page unload events, such as sending analytics data or logging. However, browsers typically terminate in-flight network requests when a user navigates away from a page. This behavior can result in data loss or incomplete operations, which is particularly problematic in SPA applications like those built with Angular.

Solution:

To address this issue, Angular has added support for the keepalive flag in fetch-based HTTP requests. This enhancement uses the keepalive option in the Fetch API, which is accessed using the withFetch() function when setting up an HttpClient. This option allows certain requests to continue executing even after the page has begun unloading.

```
@Injectable({ providedIn: 'root' })
export class AnalyticsService {
  private readonly _http = inject(HttpClient);


  sendAnalyticsData(data: AnalyticsData): Observable<unknown> {
    return this._http.post('/api/analytics', data, { keepalive: true });
  }
}
```

Benefits:

This feature makes Angular applications more reliable by making sure important background requests aren't interrupted when pages change or when they're being unloaded. This guarantees that important operations, like logging or saving data, are completed successfully even if the user closes the page before the operation is finished.

Type checking support for host bindings

Dev Experience

Challenge:

Host bindings are commonly used to bind component or directive class properties to attributes, properties, or event listeners on the host element. In the past, Angular only checked types for templates. However, host bindings can also contain expressions, which can cause type checking problems, like referencing properties that don't exist.

These issues would only show up at runtime, potentially causing subtle bugs or failures that were harder to detect and debug during development.

Solution:

Angular has added type checking support for host bindings, ensuring that expressions used in the @HostBinding and @HostListener decorators - as well as in the host metadata field - are validated at compile time.

Benefits:

Now the compiler makes sure you're using the right properties and methods in host bindings, based on their types. It also unlocks features like rename and hover in the language service.

Extended diagnostics for nullish coalescing and uninvoked track function

Dev Experience

Challenge:

Angular templates often include dynamic expressions and complex bindings that can fail without warning or cause performance and correctness issues. Without clear, compile-time diagnostics, developers may overlook incorrect logic or usage patterns.

Solution:

Angular is adding more and more tools to help identify and fix these problems.

Nullish coalescing

This diagnostic detects cases where the nullish coalescing operator ("??") is mixed with the logical OR ("||") or logical AND ("&&") operators without parentheses to disambiguate precedence. Without it, it's hard to tell which operator is evaluated first. This is considered an error in TypeScript and JavaScript, but it has historically been allowed in Angular templates.

Uninvoked track function

Simply passing the function name to the track function of the @for block does not work because it is not a property binding, as it is with *ngFor. A new diagnostic warns developers that they need to call the function to achieve the desired performance outcome.

Benefits:

These diagnostics make Angular applications more reliable and easier to maintain by helping developers find subtle issues during development. By providing clearer feedback during the build process, Angular continues to strengthen its tooling and support for scalable development.

Stop producing ng-reflect attributes

Dev Experience

Challenge:

Angular has always included ng-reflect-* attributes in DOM elements in development modeto help developers inspect bound values using browser dev tools. While useful for debugging, these attributes come with trade-offs. If you use this tool, it can increase the size of the rendered DOM. It can also expose sensitive data without your intention or lead to confusion or security concerns when you use it without being aware of the risks.

Solution:

To improve the experience for developers and make markup safer and cleaner, Angular has changed its behavior to no longer generate ng-reflect attributes by default. Developers who still need this feature for debugging turn it back on using theprovideNgReflectAttributes() function in the list of providers.

Benefits:

By turning off how ng-reflect automatically makes attributes, Angular reduces the visual noise in the DOM, avoids accidentally showing the internal state, and performs better during development.

This change makes it easier to use security practices and improve the quality of the code, while still allowing teams that need reflective debugging to enable it with a special step.

What Experts Say About Angular 20

As already defined by fellow content creators, Angular 20 is the 'Stabilizer'.
Signals effect, linkedSignal, to Signal, toObservable, and after* were already used in applications, even if still in preview, as quite robust and really useful to fill some missing pieces in Angular. So, nothing really new about their usage.

If the Resource API and Vitest are quite amazing, they are experimental, so you might want to give them a try, but you're fine waiting for some fine-tuning.

If they are not promoted that much, there are a lot of DX enhancements, from better error reporting to some edge case fixes on migration scripts.

For most people, this release can be summed up as: You can use more stable Signal APIs!

~ Gérôme Grignon
Angular Devs France Founder

The Angular team made a few decisions in Angular 20:

- *Deprecated NgIf, NgFor, and NgSwitch*
- *Stabilized the Signal API, toSignal, toObservable, outputFromObservable, outputToObservable, and afterRenderEffect functions.*
- *New naming convention proposal*

Regarding the deprecated structure directives, developers should apply the schematic to migrate legacy applications to use the control flow syntax. Structure directives will not be eliminated, but they will be replaced by something better and easier to understand.

Developers who maintain production legacy applications should convince their managers to use signals to maintain states after the Signal API reaches the stable status. The Signal API is simple to use, manages states synchronously, and can interoperate with RxJS when needing to retrieve values from asynchronous events. If teams are worried about the efforts, they can apply various signal schematics to make the changes incrementally.

The naming convention proposal sparked debates in the Angular community. Some welcome the idea of dropping the suffix, while others want to retain it. The debates will continue for a while, which is a good thing. Constructive discussions have led to the development of good features and tooling in Angular in the past. In terms of enterprise development, the naming must be consistent. Either all files have a suffix or without it, not a mix of both.

Pro tip:
When a value depends on reactive values (signal, computed,etc) and is writable, use the ***linkedSignal*** *function.*
When a value depends on reactive values and is read-only, use the ***computed*** *function.*
When a DOM element is read and written reactively, apply ***afterRenderEffect*** *in the constructor.*
When none of the above applies, use ***effect*** *to update signal values.*

~ Connie Leung
Google Developer Expert

I feel like Angular 20 was more about making things stable. A bunch of features that developers were just trying out are now stable. It still has some cool stuff like improvements to SSR, createComponent, the template syntax, better diagnostics, and the style guide, but it's not as huge as version 19 was. This also gives developers some much-needed time to really get a handle on all the new features.

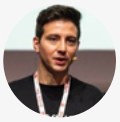
Here's a quick pro tip: really try to focus on a declarative coding style instead of an imperative one. Even though this advice has been around forever, a lot of developers used to really struggle trying to code declaratively. But now, with so many Signal APIs available, going the declarative route is way easier than it's ever been!

~ Fanis Prodromou
Google Developer Expert

Don't forget to upgrade to Angular 20 and always stay on the current version as much as possible. This will make future upgrades a breeze, rather than a struggle.

~ Alain Chautard
Google Developer Expert

Like all the last Angular releases, this one introduced new key features and deprecated others to improve the framework. The team listens to the community and tries to accommodate their and the community's needs. Ivy first, signals later, and all these new features make Angular easier to use. The team's commitment is incredible; every six months, a new version full of improvements is shipped. My pro tip is a simple one: keep your codebase always up to date with the new versions, plan the upgrade, and with a small effort every six months, you will always have a modern and fancy codebase that is easy to maintain. Hiring new developers should also be easier for you.



~ Luca Del Puppo
Google Developer Expert

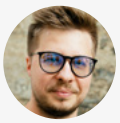
I think that Angular 20 in terms of community reception and scope of delivered features was not so exciting, but that does not discredit it because it provided us with a lot of important stabilization fixes. Without stabilization of what has been done so far, taking the next step would be difficult. That's why it's worth dedicating time to updating and making sure you benefit from these stability improvements.

~ Mateusz Stefańczyk
Google Developer Expert,
Angular Team Leader at House of Angular

I see Angular 20 as a stabilization and refinement release. In addition to introducing new features, it also marks many "developer preview" APIs as stable.

I was especially glad to see zoneless Angular promoted from experimental to developer preview, as well as continued improvements to SSR — including the stabilization of incremental hydration and the new route-level rendering mode configuration.

Overall, I'm really happy to see this kind of "stabilization" release. After the intense transformations Angular has gone through recently, I think it was much needed.



~ Dmytro Mezhensky
Google Developer Expert,

Signal-based Forms

Vitest

Animations API

Angular v21

release date: **11.2025**

Angular v21 continues evolution toward a fully signal-driven architecture. The headline feature is experimental signal-based forms — a complete reimagining of form handling that treats forms as natural reflections of your data model rather than separate state managers. This is experimental territory, but it offers a clear preview of how forms will work in the signal-first Angular of tomorrow.

Alongside this, the new animations API introduces lightweight animate.enter and animate.leave bindings that reduce bundle size, improve performance, and make animations feel like natural template declarations rather than module-level configurations.

Beyond these technical features, v21 marks a watershed moment for Angular's developer experience: Vitest becomes the default testing framework. After more than two years of uncertainty about which testing framework to use, the Angular team has made a clear decision.

Beyond these marquee features, v21 release represents Angular's ongoing commitment to refinement: making the everyday development experience cleaner and more predictable with smaller but meaningful improvements that smooth out rough edges across the framework.

Short list of features:	Dev Experience	Efficiency	Performance	UX
New expressions supported in templates	✅			
More configuration options for HttpClient	✅			
New properties in HttpResponse	✅			
Routes lazy loading runs in injection context	✅			
Support for decoding in NgOptimizedImage			✅	
Support bindings in TestBed	✅			
New animations API	✅		✅	
Improved ARIA property binding	✅			
First signal-based API in Router	✅			
Improved server bootstrapping				✅
Signal forms (experimental)	✅			
Vitest as default testing framework	✅			
Angular Aria	✅			✅
MCP Server	✅			

New expressions supported in templates

Dev Experience

Challenge:

Angular templates have come a long way, but they have always had limited support for plain JavaScript expressions. Angular 20 introduced features such as template literals, the exponential operator, the void operator, and the *in* keyword.

But there were still especially when it came to more advanced or expressive syntax used in everyday JavaScript.

Solution:

Angular keeps adding to its template capabilities.

Now, we canuse a whole new set of assignment operators directly inside templates:

- += - addition assignment
- -= - subtraction assignment
- *= - multiplication assignment
- /= - division assignment
- %= - remainder assignment
- **= - exponentiation assignment
- &&= - logical AND assignment
- ||= - logical OR assignment
- ??= - nullish coalescing assignment

along with regular expressions:

```
Matches: {{/\\d+/.test(value)}}
```

Benefit:

This improvement is another big step toward making template expressions act like plain TypeScript expressions.

More configuration options for HttpClient

Dev Experience

Challenge:

Effective communication between clients and servers is one of the most critical aspects of any web application. Angular's HttpClient is the tool of choice for making it happen.

Although the default configuration usually works well, there are times when you need more precise control. Up until recently, customizing the request was somewhat limited.

Solution:

Angular's HttpClient has just got a big upgrade with a new set of configuration options.

Most of these options (except for timeout) are only available when using the withFetch configuration of provideHttpClient (this setting switches Angular from the legacy XMLHttpRequest to the modern Fetch API) or making a request through httpResource.

Here's what's new:

- **Timeout** - the maximum amount of time (in milliseconds) to wait for a response. If it takes longer, the request is aborted.
- **Cache** - it controls browser caching behavior. Options: default, no-store, reload, no-cache, force-cache, only-if-cached (experimental).
- **Priority** - it sets request priority: auto, high, or low. This is useful for distinguishing between high-priority requests (like ones impacting LCP) and low-priority background tasks.
- **Mode** - It determines if the cross-origin requests lead to valid responses and which response properties are readable. Options: same-origin, no-cors, cors and navigate
- **Redirect** - it defines redirect handling: follow, error, or manual.
- **Credentials** - Specifies whether cookies and HTTP authentication are included. Options: omit, same-origin, or include.
- **Referrer** - it sets the referrer of the request.
- **ReferrerPolicy** - it controls how much referrer information should be included with requests.
- **Integrity** - it verifies the response integrity using a hash (Subresource Integrity), ensuring the response hasn't been tampered with.

Benefits:

These enhancements makeHttpClient more powerful and future-proof than ever. Fine-tuning settings such as the cache strategy, credentials, and request priority helps optimize performance, strengthen security and improve reliability.

With Angular, you get the best of both worlds: the convenience of a high-level API and the precision of low-level network control.

New properties in HttpResponse

Dev Experience

Challenge:

As Angular evolves toward tighter alignment with the Fetch API, its HttpClient has been getting some behind-the-scenes updates. Please note that to use its fetch-based implementation, you need to use the withFetch configuration of provideHttpClient.

Solution:

The Angular's latest update fills the gaps by introducing two small yet powerful properties to HttpResponse: responseType and redirected.

The responseType property indicates how the browser actually treated the response — whether it's "basic", "cors", "opaque", or "error".

This insight is especially useful for CORS debugging, security analysis, and request validation. When working with APIs that operate across domains or use various request modes, responseType gives you a clear indication about what's been going on under the hood.

The redirected property simply tells you whether the response came from a redirect. But behind that simplicity lies significant value. It lets developers track redirect flows, enforce security policies, or log redirect behavior for analytics and performance monitoring — all without adding custom fetch logic.

Benefits:

Though these two properties may seem minor, together they bring Angular's HttpClient even closer to the modern Fetch API. They improve transparency, observability, and debugging, which developers will immediately appreciate.

This is yet another clear sign that Angular is steadily modernizing, bridging the gap between its abstractions and the native web platform underneath.

Routes lazy loading runs in injection context

Dev Experience

Challenge:

Lazy loading is one of Angular's superpowers. It helps improve app performance by only loading code when it's needed. Yet, in real-world projects, it's not always as straightforward as defining a static route; sometimes it depends on runtime conditions.

Solution:

The router functions loadChildren and loadComponent now run within the injection context of the route, This means that you can inject services directly inside these functions.

Here's what that looks like in practice:

```
{
  path: 'product-configuration',
  loadComponent: async () => {
   const featureFlagService = inject(FeatureFlagService);
   const isEnabled = featureFlagService.isEnabled(
    'ai_product_configuration',
   );


   return isEnabled
    ? (await import('@my-app/ai-product-configuration'))
      .AiProductConfigurationComponent
    : (await import('@my-app/product-configuration'))
      .ProductConfigurationComponent;
  },
}
```

Now, you can base your lazy loading logic on the real-time state of your application without using workarounds or global dependencies.

Benefits:

This improvement makes Angular routing far more flexible and expressive.

Now that Angular supports dependency injection in loadComponent and loadChildren, you can get dynamic, context-aware route loading right out of the box. It makes the architecture simpler and the code cleaner.

Support for decoding in NgOptimizedImage

Performance

Challenge:

Images play a major role in modern web applications, but the way and when they're decoded can make or break how fast your content shows up on the screen.

Most browsers automatically handle image decoding, which works well in most situations. But there are times when you might want more control.

Solution:

With the NgOptimizedImage directive, you can now specify how the browser should decode images, giving you fine-grained control over rendering performance.

You can set the decoding behavior using the decoding attribute:

- **Async** - decodes an image asynchronously to avoid delaying the presentation of other content (non-blocking)
- **Sync** - decodes an image synchronously which prevents the presentation of other content until it's finished (blocking)
- **Auto (default)** - indicates no preference

If an image is marked as priority, Angular automatically sets its decoding behavior to sync, ensuring it's painted as early as possible.

Benefits:

This update helps developers strike the perfect balance between performance and visual stability. If you define how images are decoded, you can optimize page load, improve the user experience and simplify fine-tuning for metrics like LCP and CLS.

Support bindings in TestBed

Dev Experience

Challenge:

When writing unit tests in Angular, it is common practice to wrap the component under test in a host (wrapper) component. This approach allows you to simulate bindings, inputs, and outputs, basically making it act like it would in a real template.

Although this technique is effective, it requires additional setup, imports, and template code for each wrapper component, which distracts from the actual behavior you're trying to verify.

Solution:

The TestBed.createComponent() function now supports the new binding helpers - inputBinding(), outputBinding(), and twoWayBinding() - allowing you to bind inputs and outputs directly when creating the component.

```
it('old approach', () => {
  @Component({
    imports: [MyCheckbox],
    template: '<my-checkbox [isChecked]="isChecked"/>',
  })
  class Wrapper {
    isChecked = false;
  }


  const fixture = TestBed.createComponent(Wrapper);
  const checkbox = fixture.nativeElement.querySelector('my-checkbox');


  fixture.componentInstance.isChecked = true;
  fixture.detectChanges();
  expect(checkbox.classList).toContain('checked');
});


it('new approach', () => {
  const isChecked = signal(false);
  const fixture = TestBed.createComponent(MyCheckbox, {
    bindings: [inputBinding('isChecked', isChecked)]
  });


  const checkbox = fixture.nativeElement.querySelector('my-checkbox');
  isChecked.set(true);
  fixture.detectChanges();
  expect(checkbox.classList).toContain('checked');
});
```

Benefits:

This improvement makes writing unit tests with TestBed easier. By removing the need for a wrapper component, your test focuses solely on the logic of the component itself. This approach keeps the behavior consistent between tests and the actual app, makes tests more declarative and reduces boilerplate code.

New animations API

Dev Experience | Performance

Challenge:

Although animations are a key part of delivering rich user interfaces, the built-in animation tooling in many Angular applications has begun to feel heavy and outdated. The legacy package @angular/animations was created years ago - before the modern CSS animation capabilities and native browser APIs had matured.

As a result, developers face several real drawbacks: large bundle sizes, animations that are entirely managed in JavaScript rather than leveraging native performance optimizations, and limited interoperability with third-party libraries.

The mismatch between Angular's animation module and the modern web platform means animations can become a maintenance burden, slow down performance, or get in the way when trying to integrate new tools or patterns.

Solution:

Angular has introduced two new built-in animation bindings - animate.enter and animate.leave - that make it easier than ever to animate elements as they appear or disappear in your application.

These bindings apply CSS classes or call custom animation functions at appropriate times during an element's lifecycle. Unlike traditional directives, animate.enter and animate.leave are special compiler-level features that are recognized directly by Angular. You can use them on elements in your templates or even as host bindings inside components.

animate.enter

The animate.enter binding runs whenever an element enters the DOM. This makes it perfect for defining entrance animations using either CSS transitions or keyframes.

Once the animation is complete, Angular automatically removes the specified animation class or classes from the element, ensuring that they are active only for the duration of the animation.

```
@Component({
 selector: 'animate-enter-example',
 template: `
  <button (click)="toggleVisibililty()">Toggle element</button>


  @if (isVisible()) {
   <div class="container" animate.enter="enter-animation">
    <p>animate.enter example</p>
   </div>
  }
 `,
 styles: [
  `
   .container {
    border: solid 1px black;
    padding: 1rem;
   }


   .enter-animation {
    animation: slide-fade 1s;
   }


   @keyframes slide-fade {
    from {
     opacity: 0;
     transform: translateY(20px);
    }


    to {
     opacity: 1;
     transform: translateY(0);
    }
   }
  `,
 ],
})
export class EnterExample {
 readonly isVisible = signal(false);


 toggleVisibility(): void {
  this.isVisible.update((isVisible) => !isVisible);
 }
}
```

animate.leave

The animate.leave binding works similarly, but is used for elements being removed from the DOM. It allows you to define exit animations that run just before Angular removes the element.

Angular waits until the animation finishes before detaching the element.

```
@Component({
 selector: 'animate-leave-example',
 template: `
  <button (click)="toggleVisibililty()">Toggle element</button>


  @if (isVisible()) {
   <div class="container" animate.leave="leave-animation">
    <p>animate.leave example</p>
   </div>
  }
 `,
 styles: [
  `
   .container {
    border: solid 1px black;
    padding: 1rem;


    @starting-style {
     opacity: 0;
    }
   }


   .leave-animation {
    opacity: 0;
    transform: translateY(20px);
    transition:
     opacity 500ms ease-out,
     transform 500ms ease-out;
   }
  `,
 ],
})
export class LeaveExample {
 readonly isVisible = signal(false);


 toggleVisibility(): void {
  this.isVisible.update((isVisible) => !isVisible);
 }
}
```

Event bindings

Both the animate.enter and animate.leave support event binding syntax. This allows you to to call component functions or integrate third-party animation libraries.

The $event object has the type AnimationCallbackEvent. It includes the element as the target and provides an animationComplete() function that notifies the framework when the animation finishes.

```
@Component({
 selector: 'event-binding-example',
 template: `
  <button (click)="toggleVisibililty()">Toggle element</button>


  @if (isVisible()) {
   <div class="container" (animate.leave)="animateLeaving($event)">
    <p>event binding example</p>
   </div>
  }
 `,
 styles: [
  `
   .container {
    border: solid 1px black;
    padding: 1rem;
   }
  `,
 ],
})
export class EventBindingExample {
 readonly isVisible = signal(false);


 toggleVisibility(): void {
  this.isVisible.update((isVisible) => !isVisible);
 }


 animateLeaving(event: AnimationCallbackEvent): void {
  // Example of calling GSAP
  gsap.to(event.target, {
   duration: 1,
   x: 100,
   onComplete: () => event.animationComplete(),
  });
 }
}
```

Benefits:

With this new API, Angular developers get a more streamlined, performant, and flexible way to handle animations.

Reliance on the heavyweight animation module is reduced, resulting in smaller bundles and better runtime performance. Native CSS transitions and third-party animation tools can now be used seamlessly, giving you the freedom to choose the right animation strategy for your project.

The declarative animate.enter and animate.leave bindings make templates more readable and expressive by aligning animations with how elements actually enter and leave the view. Additionally, animate.leave's built-in delayed removal capability ensures smooth transitions without hacks or excessive boilerplate.

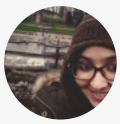
Ultimately, your animations feel more integrated, your code becomes cleaner, and your app runs better.

Expert Opinion:

Moving from `@angular/animations` to native CSS with `animate.in`/`animate.out` is absolutely the right direction, as it simplifies the mental model by putting animations back where they belong—directly in CSS—while improving performance and reducing framework-specific overhead. This shift aligns with Angular's broader move toward standard Web APIs, smaller bundles, and a cleaner, less magical architecture that's easier for developers from other ecosystems to understand.

The trade-off, however, is that complex orchestrations might require custom coordination layers or specialized libraries such as GSAP, since Angular's abstractions will no longer handle this automatically. Additionally, existing code using Angular's trigger/state machine patterns won't map directly, making migration non-trivial. Technical leads will need to set clear guidelines to ensure CSS animations remain consistent, maintainable, and decoupled from component logic.

As a forward-looking step, Angular should fully embrace platform capabilities such as the View Transitions API for route- and page-level animations, offering architectural guidance without reintroducing heavy abstractions. Overall, this move is philosophically and technically sound, but complex animation scenarios still require careful handling and thoughtful migration planning.



~ Fatima Amzil
Google Developer Expert

Is moving from @angular/animations to native CSS animations with `animate.in` and `animate.out` the right direction for Angular? Yes, it is. Browsers are becoming more powerful with built-in CSS functionality. It is much easier to achieve CSS animations with native CSS, without the need for libraries. The syntax of @angular/animations is not intuitive, and native CSS and other CSS libraries can easily do what it can which is animating DOM elements.

We are trying to bring non-Angular developers into the Angular community. The last thing they need is to learn another Angular package to do animation. They have enough to pick up which signals, resources, httpResources and finally signal form.

~ Connie Leung
Google Developer Exper

Moving code outside of the main thread is always a win! We can free up CPU and memory for other tasks and let the browser handle animations more efficiently than JavaScript. On the web, in the last few years, we have seen many features that used to be in JavaScript move to CSS. With the devices' improvements, delegating tasks to the browser and using the MainThread only for significant tasks is fantastic. Doing so enables the application to deliver fluent, high-quality UI and achieve better performance when handling data or managing components.
Ideally, everything should be fantastic as I described it before!
But we will also need to consider how much memory and GPU we will use if we have heavy animation, as this can create issues on small devices.

~ Luca del Puppo
Google Developer Expert

Animations are offloaded to the GPU, freeing up the main thread and keeping the UI responsive. This makes it much easier to maintain smooth 60fps, even when heavy JavaScript logic is running in parallel. On mobile devices, this can also translate into better battery efficiency.
In Angular specifically, the biggest difference is visible in reusable UI-heavy areas such as long lists, overlays, modals, virtual scrolling, micro-interactions and animated route transitions — all common performance bottlenecks in real-world apps.

GPU layers consume memory, so animating large numbers of elements or stacking multiple layers can backfire and actually reduce performance. Only a very limited set of properties (transform, opacity, filter) are truly GPU-accelerated — animating layout-dependent properties like width, height, top or left still triggers layout and paint cycles.
In practice, many applications overuse animations where subtle state changes or even no animations would scale better and produce a more consistent user experience.

~ Mateusz Stefańczyk
Google Developer Expert

Improved ARIA property binding

Dev Experience

Challenge:

Angular required developers to use the attr. prefix when binding to ARIA attributes, like [attr.aria-label]="label". Although this method was functional, it was less intuitive than other bindings.

This also created inconsistencies across environments. During server-side rendering (SSR), ARIA properties were not always properly reflected properly as attributes in the output HTML. It could lead to accessibility gaps or inaccurate results in automated accessibility testing tools.

Solution:

Angular now lets you bind to ARIA attributes directly, without using the attr. prefix.

For example, [aria-label]="label" and [ariaLabel]="label" are both valid and equivalent.

Angular automatically identifies the correct target - whether it's an input property or an ARIA HTML attribute - ensuring that accessibility data is always applied correctly.

```
@Component({
  selector: 'aria-binding-example',
  template: `
    <!-- old way -->
    <button [attr.aria-label]="label"></button>


    <!-- new way -->
    <button [aria-label]="label"></button>
  `
})
export class AriaBindingExample {
  readonly label = 'My button label'
}
```

Benefits:

This change significantly improves the developer experience while it also makes Angular applications more accessible. Treating ARIA attributes as first-class bindings allows developers to write cleaner templates.

More importantly, it ensures that ARIA attributes are rendered correctly on the server, where the emulated DOM may not accurately reflect ARIA properties as attributes.

First signal-based API in Router

Dev Experience

Challenge:

Signal-based APIs are becoming the standard for more and more Angular features.

However, the Router has been lagging behind, as it still relies on older, non-reactive patterns.

Solution:

Starting with this update, Angular brings a little signal-based reactivity directly to the Router. The old getCurrentNavigation() method is now deprecated, so you should use the new currentNavigation signal instead. And lastSuccessfulNavigation has been converted into a signal, too.

This shift means you can now reactively track navigation state and easily derive other stateful information.

```
export class CurrentNavigationExample {
  private readonly _router = inject(Router);

  readonly isNavigating = computed(
    () => this._router.currentNavigation() !== null
  );

  readonly url = computed(
    () => this._router.lastSuccessfulNavigation()?.finalUrl
  );
}
```

Benefits:

It's a small change, but a meaningful one. This update is the first real step toward bringing full signal-based reactivity to the Angular Router.

Improved server bootstrapping

UX

Challenge:

Server-side rendering (SSR) has become a key part of Angular's story, helping developers create fast, SEO-friendly, and accessible web experiences. Yet, behind the scenes, the server bootstrapping process had a subtle problem that could turn serious in concurrent environments.

Historically, Angular's SSR relied on a module-level global platform injector — a single dependency injection (DI) container shared across all server renders. That approach worked fine for single-request scenarios, but became problematic when multiple requests were processed at once.

In high-traffic environments, concurrent requests could accidentally share or overwrite the global injector state, leading to unpredictable behavior and, in the worst case, leaking request-specific data between users. This was not just a stability issue, but also a potential security risk. Under certain conditions, sensitive tokens or user-specific data could appear in another user's rendered response.

Solution:

The fix introduces a major improvement to how Angular handles server bootstrapping: the introduction of the BootstrapContext.

Rather than depending on a single global platform injector, each server-side render now receives its own scoped platform reference through this new context. This reference is passed directly to the bootstrapApplication() function, ensuring that every request runs in its own isolated environment.

```
const bootstrap = (context: BootstrapContext) =>
  bootstrapApplication(AppComponent, config, context);
```

This shift makes the server bootstrapping process safer and more predictable. It also clarifies ownership — now, each SSR request is fully encapsulated and manages its own platform lifecycle.

To reinforce this design, a few related APIs were updated:

- getPlatform() will always return null on the server to prevent accidental cross-request platform access
- destroyPlatform() has become a no-op during server rendering, which eliminates the risk of destroying a platform instance used by another request

Benefits:

The new bootstrapping model fundamentally strengthens Angular's SSR foundation. Now, each request now runs in a self-contained environment, ensuring that no sensitive data leaks across concurrent renders.

Signal forms (experimental)

Dev Experience

Challenge:

Angular's form APIs have long existed in two distinct categories: template-driven forms, which prioritize simplicity, and reactive forms, which emphasize structure and control. Both approaches have their merits, but each comes with trade-offs. Reactive forms can feel verbose, while template-driven ones often hide too much logic in the template.

As Angular evolves around signals, the question naturally arises: What would forms look like if signals were at the core? How could we handle forms more declaratively, intuitively, and reactively – without juggling subscriptions, emitters, or duplicated state?

Solution:

Version 21 introduces an experimental version of the highly anticipated signal-based forms feature — signal-based forms.

Let's take a look at how the new API works in practice.

At the heart of every form is its data model. In this new approach, that model is represented by a signal that serves as the single source of truth for the form's value.

Instead of Angular's traditional FormControl and FormGroup, you define a writable signal and pass it into the form() function. Then, it can be bound to controls using the Field directive.

This creates a two-way connection: typing in an input updates the signal, and updating the signal automatically updates the input. The form itself no longer owns the data; it simply reacts to it.

```
@Component({
 selector: 'app-logint',
 imports: [Field],
 template: `
  <form>
   <input placeholder="Email" [field]="loginForm.email" />
   <input
    type="password"
    placeholder="Password"
    [field]="loginForm.password"
   />


   <button [disabled]="loginForm().invalid()" (click)="login()">
    Login
   </button>
  </form>
 `,
})
export class LoginComponent {
 private readonly _loginModel = signal<LoginFormModel>({
  email: '',
  password: '',
 });


 readonly loginForm = form(this._loginModel);


 login(): void {
  console.log(this._loginModel());
 }
}
```

The second argument of the form() function introduces a schema that defines the behavior and validation logic of each field. Validators, read-only states, and conditional hiding (this type of control doesn't contribute to form state and validation) or disabling are all declared right next to the data model making the logic more explicit, typed, and testable.

```
readonly loginForm = form(this._loginModel, (login) => {
   required(login.email, { message: 'Email is required' });
   email(login.email, { message: 'Provide valid email address' });
   required(login.password, { message: 'Password is required' });
  });
```

Since schemas can be reused and composed, you can extract common logic into shareable utilities:

```
export const emailSchema = schema<string>((field) => {
  required(field, { message: 'Email is required' });
  email(field, { message: 'Provide valid email address' });
});


export const loginSchema = schema<LoginFormModel>((login) => {
  apply(login.email, emailSchema);
  required(login.password, { message: 'Password is required' });
});
```

Another welcome simplification is that a ControlValueAccessor is no longer required for custom controls. Instead, components integrate with the Field directive by implementing the lightweight FormValueControl interface. This interface has only one required property, "value," which is a model that keeps the component in sync with the control's value.

```
@Component({
  selector: 'app-rating-control',
  ...
})
export class RatingControl implements FormValueControl<number> {
  readonly value = model(0);
}


@Component({
  selector: 'app-review-component',
  template: `
    <form>
      <input [field]="reviewForm.name" />
      <app-rating-control [field]="reviewForm.rating" />
      <textarea [field]="reviewForm.comment"></textarea>
    </form>
  `,
  imports: [Field, RatingControl],
})
export class ReviewComponent {
  private readonly _reviewModel = signal<ReviewFormModel>({
    name: '',
    rating: 0,
    comment: '',
  });


  readonly reviewForm = form(this._reviewModel, (review) => {
    required(review.name);
    required(review.rating);
    min(review.rating, 0);
    max(review.rating, 5);
  });
}
```

Benefits:

This signal-based approach reimagines forms as a natural extension of the reactive Angular ecosystem. Forms no longer duplicate or manage state; they simply reflect your data model in real time.

Validation and conditional logic move out of the template and into clean, declarative TypeScript schemas. This makes forms easier to reason about, test, and compose. The entire experience feels lighter, more predictable, and deeply reactive from the ground up.

This experimental API is still in its early stages, but it offers a clear glimpse into the future of Angular forms — a future, in which form handling is simpler, more expressive, and perfectly aligned with the framework's signal-driven architecture.

Expert Opinion:

Regarding the advantages of signal-based forms: If you are already using Signals, you are in some weird situation where you have 2 'reactive' APIs not working nicely together.
We won't have to rely on Signal effect() anymore to update the value of a form.
The new Signal Forms API also simplifies custom form control creation, moving away from the old ControlValueAccessor, such as Wonder!

~ Gérôme Grignon
Angular Devs France Founder

Signal Forms biggest advantage is Type safety. Reactive Forms constantly fight TypeScript - `form.get('email')` returns `AbstractControl | null`, refactoring field names is error-prone. Signal Forms give real type inference: `loginForm.email` is `Field<string>`, rename a property → instant compiler errors. Bugs caught at compile time, not runtime.

~ Mateusz Stefańczyk
Google Developer Expert

First and foremost, the most obvious reason to choose signal-based forms: no more observables to react to changes in the form.

Furthermore, it feels like the best of both approaches: template-driven forms and reactive forms. It offers the simplicity of declaration like template-driven forms and the possibilities that reactive forms provide. This combination results in Signal Forms.

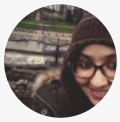
~ David Muellerchen
Google Developer Expert

The introduction of signal-based forms in Angular brings meaningful architectural challenges. Among them, the temporary coexistence of three distinct form paradigms risks fragmentation, inconsistency, and increased cognitive load across teams. Large reactive-form-driven applications will face significant migration friction, especially where complex custom controls, dynamic form builders, or deep validation logic are involved.

This transition also requires a shift from a push-based to a pull-based reactive mental model, demanding new team conventions to prevent accidental reactivity chains or hybrid anti-patterns. With the APIs still maturing and ecosystem libraries, tooling, and testing utilities not yet fully aligned, technical leads should cautiously introduce signal-based

forms in production, starting with controlled scope and monitoring. Simultaneously, investing time in experimentation, prototypes, and internal exploration will help teams build expertise early and ensure a smoother transition.

Despite these short-term hurdles, the long-term payoff—simpler mental models, more predictable change detection, and a cohesive signal-first Angular architecture—promises a cleaner, more maintainable future once migration becomes viable.



~ Fatima Amzil
Google Developer Expert

*The interoperability between Angular's **Signal Forms** and the existing **Reactive Forms** is very well-designed. The core philosophy seems to be to enable a smooth, incremental adoption rather than a big migration all at once. This is built upon three key pillars: **Component Level**, Model Level, and **Dependency Level**.*

Component Level Interoperability

*At the **component level**, the new Signal Forms integrate really well with the existing ecosystem. This is crucial because it ensures **backwards compatibility**.*

- *Any existing components based on the ControlValueAccessor (CVA)—which is how custom form inputs are typically built—will **continue to work perfectly** with Signal Forms.*
- *This means your application's custom input components, as well as components from third-party UI libraries (like Angular Material or PrimeNG), require **no changes** to be used within a Signal Form.*

Model Level Interoperability

*This is perhaps the most impressive and important pillar, especially for **large and complex applications**.*

- *Many enterprise applications rely heavily on Reactive Forms, often having **complex validation logic** or reusable form services that expose FormGroups, FormControls, or FormArrays.*
- *The interoperability here allows developers to **mix and match** Signal Forms with the existing AbstractControl types (the base of Reactive Forms).*
- *This means you **don't have to rewrite** all your established validation or form logic. You can integrate a new Signal Form and point it to an existing, complex Reactive*

Forms validator, or embed an existing FormGroup within a Signal Form structure.

This feature is an important step forward, allowing teams to preserve their business logic while transitioning toward the new style.

Dependency Level Interoperability

Similar to the Component Level, this ensures that components that rely on NgControl will keep functioning.

- *Many components rely on the NgControl token to automatically handle things like marking an input as **required(*)** or displaying **automatic error messages**.*
- *Some custom form components rely on the NgControl to use the ValueAccessor in order to bridge with the Reactive Forms—this will **continue to function as expected**.*

Steps for Adoption

*As of today, Signal Forms are still experimental and should **not be used in production**. However, assuming they become stable and ready for use:*

*For **New Features**: Use Signal Forms by default. Start building team expertise and use the new structure where there's no existing technical debt.*

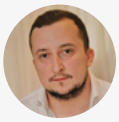
*For **New Features** with **Complex Existing Logic:** Create a "compatibility" Signal Form that allows you to mix Signal Forms with Reactive Forms. You can use the existing custom validations or logic that is written in services.*

***Existing Code Migration:** Do not attempt a full, immediate migration. You can take advantage of the strong interoperability to let the existing code run while you focus on new development. Migrate existing features only when necessary.*

The interoperability is designed to allow applications to use third-party libraries and existing code while the team builds expertise in Signal Forms. It is a low-risk path to adoption.

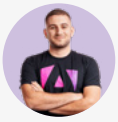
~ Fanis Prodromou
Google Developer Expert

I love that signal-based forms are designed to coexist with existing reactive forms, so teams don't need a big bang migration. However, I'd still wait until Signal Forms move beyond experimental status before using them in critical production paths, because the team explicitly reserves the right to introduce breaking changes even in patch releases. In the meantime, I'd recommend spiking them in smaller, isolated features, aligning on internal patterns, and preparing shared utilities so that when they're stable, migration is mostly incremental refactoring rather than a full rewrite.



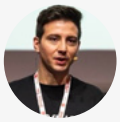
~ Marko Stanimirović
Google Developer Expert

The compatForm function that signal forms provide helps a lot here. You can put FormControls inside a signal and that signal can be used inside compatForm and it should just work. Best part is: You can do the same with FormGroups. If you have a FormGroup, you can put it inside a signal and that's ready to be used inside compatForm. And it just works with the new [field] directive too.
Signal forms are experimental, so I'd recommend start playing with them so you become familiar with this new forms system, and when they reach stable start using them.



~ Enea Jahollari
Google Developer Expert

I love signal-based forms, but I also love Reactive forms! Regarding signal-based forms, I see real benefits for small-to-medium complex forms, where validations are straightforward and we have only a few fields to handle.
But when the forms start growing and validations start to become tricky and async, I'd still prefer the Reactive one. Async validation with signals creates a more verbose codebase and makes the flow harder to follow.
But maybe it's a bias because I love React Forms too much.
Said so: signal-based forms are really cool for aligning the framework's ecosystem with signals, and especially for junior developers, they are easier to learn and use at the beginning.



~ Luca del Puppo
Google Developer Expert

Vitest as default testing framework

Dev Experience

Challenge:

For more than two years, Angular developers faced an uncomfortable uncertainty about testing: which framework should they use, especially for new projects? The official Jasmine/Karma combination was aging. Karma was explicitly not going to be supported long-term, with the Modern Web Test Runner mentioned as a potential replacement. Jest emerged as a community-supported option, but then Vitest appeared as another strong candidate.

This created analysis paralysis. Should you stick with the official but outdated Jasmine? Invest in Jest, knowing Vitest might become the official choice? Or adopt Vitest early and hope the Angular team would follow?

Solution:

Starting with Angular 21, Vitest becomes the default testing framework. The Angular team chose Vitest for several strategic reasons. Most importantly, Vitest offers a browser mode that executes tests in a real browser environment, just like Jasmine and Karma. This makes migration from the old stack significantly smoother – your existing browser-dependent tests can transition without fundamental rewrites. Jest, by contrast, runs on Node.js, which would have created more migration friction.

Vitest 4 recently stabilized its browser mode, likely the final piece that made this decision possible. Beyond migration compatibility, Vitest brings modern tooling advantages: first-class TypeScript support, seamless ESM handling (unlike Jest's ongoing struggles), and integration with Vite – the build tool that's become the standard across the JavaScript ecosystem.

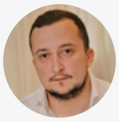
Benefits

The biggest benefit isn't Vitest itself – it's finally having a clear, official answer. The uncertainty is over. New projects have an obvious starting point. Existing projects have a clear migration path.

Vitest brings Angular closer to the broader JavaScript ecosystem. It's used across nearly all major frontend frameworks, which means better community support, more plugins, shared knowledge, and a larger pool of developers who already know the tool.

Expert Opinion:

While Signal Forms get most of the spotlight, my personal "hidden gem" in Angular 21 is the first-class, stable Vitest support. It significantly improves the developer experience around testing (fast runs, great watch mode, Vite ecosystem), and the migration tooling lowers the barrier for teams stuck on Karma/Jasmine. For some codebases, this change will have a bigger day-to-day impact than any single feature.



~ Marko Stanimirović
Google Developer Expert

Angular Aria

Dev Experience **UX**

Challenge:

Building accessible, interactive UI components from scratch is a significant undertaking for development teams.

Implementing proper ARIA attributes, keyboard navigation, focus management, and screen reader support requires deep expertise and extensive testing across a range of assistive technologies. Creating robust, production-ready interactive components such as accordions, menus, and comboboxes requires considerable development effort that could be spent on core business logic.

Existing component libraries often come with opinionated styling that conflicts with custom design systems, making it difficult to achieve the exact look and feel teams need.

These challenges force teams to choose between spending precious development time on component infrastructure or compromising on accessibility and custom styling requirements.

Solution:

Angular Aria provides a modern library of headless, accessible UI components that solve these challenges through a thoughtful, developer-focused approach. Available now in developer preview, this library delivers 8 essential UI patterns across 13 components, all unstyled and ready for customization.

The library launches with comprehensive coverage of common interactive patterns, including Accordion, Combobox, Grid, Listbox, Menu, Tabs, Toolbar, and Tree. Each component is built with accessibility as the top priority, automatically handling complex requirements such as keyboard navigation, ARIA attributes, and focus management.

Angular Aria embraces modern Angular architecture with signal-based reactivity and contemporary directive patterns, ensuring your components are performant and maintainable.

```
@Component({
 selector: 'app-tab',
 template: `
  <div ngTabs>
   <div ngTabList selectionMode="follow" selectedTab="movie">
    <div ngTab value="movie">Movie</div>
    <div ngTab value="theatres">Cast</div>
    <div ngTab value="showtimes">Reviews</div>
   </div>


   <div class="sliding-window">
    <div ngTabPanel [preserveContent]="true" value="movie">
     <ng-template ngTabContent>Panel 1</ng-template>
    </div>


    <div ngTabPanel [preserveContent]="true" value="theatres">
     <ng-template ngTabContent>Panel 2</ng-template>
    </div>


    <div ngTabPanel [preserveContent]="true" value="showtimes">
     <ng-template ngTabContent>Panel 3</ng-template>
    </div>
   </div>
  </div>
 `,
 imports: [TabList, Tab, Tabs, TabPanel, TabContent],
})
export class TabComponent {}
```

This new offering complements the Angular team's existing component solutions, creating a complete toolkit where you can use Angular Aria for accessible, headless components with complete styling control, leverage the CDK for behavior primitives like Drag and Drop in custom components, or choose Angular Material for fully styled components following Material Design principles with theming customization.

Benefits:

Angular Aria delivers significant advantages that accelerate development while maintaining the highest standards of accessibility and flexibility. Every component ships with built-in ARIA attributes, keyboard navigation, and screen reader support, eliminating months of specialized development and testing.

The headless architecture means you can apply any design system, CSS framework, or custom styles without fighting against pre-existing opinions, making it perfect for teams with established brand guidelines.

MCP Server

Dev Experience

Challenge:

Modern development workflows increasingly rely on AI assistants to accelerate coding tasks, answer technical questions, and guide developers through complex framework features. However, AI tools often lack direct integration with development environments and build tooling, limiting their ability to perform concrete actions like generating code, adding dependencies, or executing framework-specific commands.

Developers must frequently switch context between their AI assistant and command-line tools, manually translating AI suggestions into CLI commands. Furthermore, AI assistants without access to current, authoritative framework documentation may provide outdated advice or recommend deprecated patterns, particularly problematic in rapidly evolving frameworks like Angular, where best practices around signals, standalone components, and zoneless applications are still emerging.

Solution:

The Angular CLI now includes an experimental Model Context Protocol (MCP) server that bridges the gap between AI assistants and your development environment. This integration enables AI tools to interact directly with the Angular CLI, performing actions such as code generation and package management without requiring manual command execution.

The server comes with several powerful tools enabled by default:

- **ai_tutor:** Launches an interactive AI-powered Angular tutor, recommended for new Angular projects using version 20 or later
- **find_examples:** Searches a curated database of official, best-practice examples focusing on modern and recently updated Angular features
- **get_best_practices:** Retrieves the Angular Best Practices Guide covering modern standards, including standalone components, typed forms, and modern control flow
- **list_projects:** Reads your workspace's angular.json configuration to identify all applications and libraries
- **onpush_zoneless_migration:** Analyzes your code and provides step-by-step migration plans to OnPush change detection, a prerequisite for zoneless applications
- **search_documentation:** Queries the official Angular documentation at https://angular.dev for APIs, tutorials, and best practices

Benefits:

The Angular MCP server transforms how developers interact with AI assistants by creating a seamless bridge between conversation and action. AI tools can now execute Angular CLI commands directly rather than just suggesting them, reducing context switching and the need to translate recommendations into terminal commands manually. This integration ensures developers spend less time copying and pasting commands and more time building features.

The curated toolset guarantees that AI assistance is grounded in authoritative, up-to-date Angular knowledge. By connecting assistants directly to official documentation, best practices guides, and vetted code examples, the MCP server helps prevent the common problem of AI tools recommending outdated patterns or deprecated APIs. This is particularly valuable as Angular continues evolving with modern features like signals and zoneless change detection, where current guidance is essential.

The future

Angular continues to evolve with exciting new features and enhancements aimed at improving performance, developer experience, and accessibility. The framework is focusing on modernizing tools, streamlining workflows, and introducing innovative capabilities to make building dynamic, scalable applications more efficient than ever.

We have a lot of features that are currently in the experimental phase and will sooner or later become stable. Let's take a look at some extra areas that seem very promising:

Selectorless

Angular is considering ways to simplify the usage of standalone components by making selectors optional. This would enable developers to use components or directives in templates directly after importing them, removing the need to define selectors. The aim is to reduce code complexity and make component integration more seamless. This feature is still under early development, with plans to gather feedback from the community through a formal request for comments (RFC) once preliminary designs are ready..

Streamed server-side rendering

Streamed server-side rendering (SSR) is an advanced approach to delivering web applications where rendered content is sent to the client incrementally as it becomes available. Unlike traditional SSR, which waits for the entire page to render before sending it, streamed SSR allows users to start interacting with the application sooner by progressively loading visible parts of the page. This method reduces initial load times, enhances interactivity, and is particularly beneficial for complex applications. Angular is actively exploring streamed SSR to improve performance, especially in applications that do not use Zone.js.

Expert Opinion:

In the future Angular SSR technique will be easier to implement, it will not require much configuration and will provide hydration so that Angular apps will be able to resume at some point.

~ Aristeidis Bampakos
Google Developer Expert

Signal integrations

Angular's signal primitives have fundamentally changed how we think about reactivity in the framework. Components written with signals feel cleaner, more predictable, and naturally reactive. But step outside component logic make an HTTP request, work with forms, read route parameters and you're back in Observable territory. This dual-reactive world isn't a bug; it's a reflection of Angular's evolution. The core packages were built before signals existed.

The Angular team is working to close this gap. The roadmap calls for improving signal integration across fundamental packages like forms, HTTP, and router. We've already seen the first steps: experimental signal-based forms in v21, resource API and httpResource, signal-based APIs starting to appear in the router. These aren't just convenience wrappers around existing Observable APIs they're rethinking how these packages should work in a signal-first world.

The transition won't happen overnight. Angular's commitment to stability means these changes will be incremental, well-documented, and backward-compatible. But the direction is clear: Angular is moving toward a world where signals aren't just for components they're the native language of the entire framework. When that vision is fully realized, the mental model becomes simpler, the code becomes cleaner, and the experience of building Angular applications feels cohesive from top to bottom.

Overall

The future of the Angular framework is focused on embracing modern web development paradigms, prioritizing performance, and simplifying developer workflows. Key efforts include advancing reactive state management with signals, enhancing server-side rendering capabilities through streaming and hydration, and reducing boilerplate with features like selectorless components. Angular is committed to evolving as a versatile and developer-friendly framework, integrating community feedback and adapting to emerging trends in web technology.

What the experts are saying

How do the overall changes affect the learning curve? Is it easier to learn Angular in its latest form?

I think that in the future it will definitely become easier. Before, everyone who came to try Angular was hit immediately by complex concepts like RxJS or NgModules. Now, the Now, the learning curve has become more gentle and you have to learn less to build the simplest Angular app. Nowadays, we experience a transition period which is hard for newcomers because they have to learn the old stuff as well as new ones but I think in a year or two things will settle down.

~ Dmytro Mezhenskyi
Google Developer Expert

This question is somewhat complex to answer. If you're just starting to learn Angular now, the process is considerably easier than it was a few years ago. This is because most of the popular web frameworks have embraced TypeScript, and Angular is not the only framework advocating for a typed approach to web development. Notably, features like standalone components, simplified routing APIs, the introduction of new control flow syntax, and improved error reporting have made it more accessible for newcomers to dive into the Angular ecosystem.

In addition, the Angular documentation has grown significantly, offering a wealth of tutorials and advanced concepts. There are official cheat sheets, resources to address common errors and their solutions, an open Discord channel for developers to connect, and a range of Google-sponsored and community-driven events that enable developers to network and learn from one another.

However, it's worth noting that many experienced developers have expressed concerns about the rapid pace of change and the introduction of new features. This sometimes requires them to revisit and potentially rewrite their existing projects, as well as continually learn and adapt to stay current in the Angular landscape.

~ Balram Chavan
Google Developer Expert

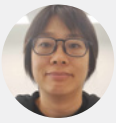
I have been able to collect a lot of feedback from my students over the last few years. RxJs and the app structure with the modules were often described as too complicated. Little by little, we are eliminating these pain points

.

~ David Muellerchen
Google Developer Expert

The overall learning curve increases again after the signal form. History may repeat itself, with some people leaving and using other frameworks that have simpler form handling. To use the signal form in modern Angular, they must understand reactivity and signals. If they have not tried signals now, they will not have any incentive to learn signal form that is built on top of signal.

For people who are new to the framework or have recently returned to it, they may not know how to start their journey into modern Angular. There are signals that we should adopt, yet the official Angular documentation continues to cover ngModule and decorators. It is a tough task for them to figure out without expert guidance.



~ Connie Leung
Google Developer Expert

Wow, I feel bad for new Angular developers. They must feel like there's quite a dichotomy. I think the difficulty in learning changes from the learner's past experience. If they are already familiar with the web frameworks out there, they may prefer the new changes, and if they are .NET devs, they may prefer the prior form of Angular. New devs who have to learn how to maintain legacy Angular projects while simultaneously learning new changes for new work are the ones I feel the most for. That said, while learning can cause discomfort for all of us, I'm looking forward to these changes, how the framework grows, and seeing all the cool things we can do with these features.

~ J. Alisa Duncan
Google Developer Expert

Angular's learning curve used to be steep. Newcomers had to learn how to handle the modules, how to work with the structural directives, how to extract the required info from the error stack trace, and learn how to master RxJS.

The latest features and improvements made these hurdles easy to overcome.
The modules are no longer needed as an application can be standalone.
The structural directives have been replaced with the new control flow and as a result the developer experience has been improved a lot!
The error stack trace provides useful information and it's easier to understand where an error is coming from.
Last but not least, the presence of signals means that mastering RxJS is no longer a necessity.

~ Fanis Prodromou
Google Developer Expert

Stepping back is quite a difficult task to evaluate the learning curve, as we often forget how hard it was to learn some fundamentals we now see as 'easy learnings'.

In my opinion, it's easier if you have someone to guide you. Otherwise, there is now so much outdated content due to the latest changes, it might be harder to find yourself valuable to land a job with modern practices.

~ Gérôme Grignon
Angular Devs France Founder

As always in the programming world - it depends. Signals are definitely easier to grasp than RxJS, and for brand-new projects the learning curve is much smoother than it used to be. However, in the real world there are many large corporate applications that have been developed over numerous versions. These apps still rely on older concepts as it's simply not possible to rewrite everything at once. Therefore, developers should still understand how these "old" concepts worked and how to maintain such code.

~ Kasia Biernat - Kluba
NgKato co-organiser

With the latest changes, Angular is still a long way from being easy to master, but it's easier to start moving the first steps with it. The team has removed tons of noise around the framework, making it easy to use and learn.
It is full of fantastic features to use, but for junior developers joining its learning, it's no longer confusing and being productive is easier than a few years ago.

~ Luca del Puppo
Google Developer Expert

What are your expectations for the direction and pace of change in Angular in the future?

In the next few versions, I expect that changes in the framework will reduce a lot of boilerplate code, improve the developer experience and apply ergonomic approaches when it comes to building Angular apps.

~ Aristeidis Bampakos
Google Developer Expert

The Angular team is highly responsive to community feedback, and their recent additions reflect their commitment to improving the framework.

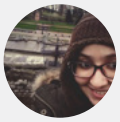
Furthermore, the Angular team is investing more effort into areas like Server-Side Rendering, Hydration, SEO optimization, integrating ESBuild, engaging in discussions around Microfrontends, and more. These initiatives clearly indicate that there are exciting developments on the horizon for Angular.

~ Balram Chavan
Google Developer Expert

Angular's trajectory in 2025 is no longer theoretical—it's aggressively practical, with a clear architectural north star: signals-first reactivity, production-ready zoneless change detection, and enterprise-grade SSR/hydration capabilities that rival any meta-framework. Angular is evolving at an unprecedented yet sustainable pace. v21, landing November 20, 2025, introduces experimental Signal Forms and zoneless change detection by default. Angular's adoption of AI integrations signals that the modern Angular stack is crystallizing faster than anticipated.

The cognitive distance between "legacy Angular" (v14–16, NgModules, Zone.js) and "modern Angular" (v19–21, standalone components, signals, zoneless) is widening exponentially every six months. Organizations still on Angular 14–16 aren't just a few versions behind—they are operating in a fundamentally different paradigm, for example, moving from NgModule-centric architecture to standalone components with signal-based reactivity.

The framework team continues to honor backwards compatibility (NgModules, Zone.js, and decorators still work in v19–21), but by v25, Angular is likely to look dramatically different from its 2020-era self. The real challenge for organizations isn't whether Angular is evolving too fast—it's whether updates are treated as annual technical debt paydown or continuous strategic investment.



~ Fatima Amzil
Google Developer Expert

While the Angular team was busy adding signal form, Tailwind CSS integration, and Vitest to Angular 21, efforts were also allocated to AI and MCP. The roadmap for next year is intriguing. Will the team be 100% committed to AI? If not, how will the team allocate resources between AI and improving code, documentation, and tooling?

After delivering the signal form feature, the team may look into "Resource" which is a vital concept in Angular 20. The status is experimental, but I hope it gets promoted to stable soon so that it can be applied to the production system

The community has many questions and feedback about the signal form after the v21 release, and the Angular team will be busy reviewing them and continuing to improve the signal form.

~ Connie Leung
Google Developer Expert

I'd enjoy seeing the documentation more shaped to build real-world applications rather than being more focused on explaining the API: recipes, tips, app examples...

~ Gérôme Grignon
Angular Devs France Founder

The next big challenge for Angular is improving the SSR and hydration developer experience.
Compared to frameworks like Next.js, Angular's SSR setup is still more complex, hydration mismatches are difficult to debug, and edge runtime support lags behind. This leads to real friction: many teams attempt SSR, hit complexity, then abandon it because the debugging cost outweighs the SEO and performance gains.
With Core Web Vitals being a ranking factor and SSR becoming table stakes in modern web apps, Angular needs a more deterministic and zero-config SSR experience, similar in clarity to Next.js' App Router: better defaults, a clearer mental model, and more predictable hydration behavior.
Solving this would have the biggest impact on Angular's competitiveness in product-focused companies.

~ Mateusz Stefańczyk
Google Developer Expert

What is your most anticipated feature in the Angular Roadmap?

Moving the form and router APIs to signal-based would remove the need for RxJS entirely, making Angular much easier to learn. As a person who teaches Angular regularly, this will make my life easier!

~ Alain Chautard
Google Developer Expert

The biggest remaining challenge the Angular team should address is Local and Feature State Management.

Right now, there are many different ways to handle State Management and developers often have to choose between:
Using complex RxJS Services.
Using third-party libraries for global state management (like NGXS or NgRx).
Using community-made Signal stores (like SignalStore).

This mix of solutions is confusing. It means different teams and different applications end up following different patterns. This adds a lot of mental work for developers and makes it harder to jump between projects.

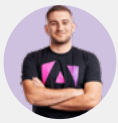
It would be amazing if the Angular team steps in and gives a built-in way to handle global/local state that is built completely on top of Signals.

This built-in solution would be a huge help because it would offer a simple-standard way to handle the state, and it will also Unify the Community.

By adding a standard state management solution, Angular would make the developer experience much smoother and easier for everyone.

~ Fanis Prodromou
Google Developer Expert

New authoring format. I don't know what to expect there, so I'd like to be surprised.



~ Enea Jahollari
Google Developer Expert

Since I'm currently working on a corporate design system based on Angular Material, I'm primarily anticipating the announced improvements to the Material and CDK libraries - especially the new CDK primitives and accessibility enhancements. I'm also looking forward to the Angular ARIA package.

~ Kasia Biernat - Kluba
NgKato co-organiser

Business perspective of upgrading from Angular 14 - 21

As discussed in previous chapters, each new version of Angular brings substantial advancements in performance, efficiency, user experience (UX), and developer experience (DX).

However, from a business standpoint, one of the most impactful benefits of upgrading from Angular 14 through 21 **is the significant reduction in technical debt**. Each version not only adds new features but often deprecates or automates away old patterns. By keeping with these updates, your code remains clean, updated, and easier to work with. You avoid the scenario of accumulating years of outdated code that becomes increasingly fragile and **expensive to modify**. Instead, you have a codebase that follows current best practices, which is easier for any new team member or external expert to understand and contribute to.

Crucially, Angular's commitment to *backward compatibility and automated migrations* means that upgrading incrementally is far less expensive than it used to be. The framework provides tools to handle many breaking changes for you. For example, as we saw with v19, tasks like refactoring dependency injection or adjusting router syntax can be handled by running a command, not by manual code changes. This significantly **cuts down the time and risk involved in upgrades**. In other words, Angular upgrades are **predictable and manageable projects** rather than massive "start from scratch" efforts. By taking advantage of these tools and possibly engaging Angular upgrade experts, you can keep your app modern **with minimal downtime or regression risk**.

All these improvements directly impact **time-to-market for new features**. When your developers have a faster build/test cycle, they **iterate quicker**. When the framework handles more for them (like default behaviors or structured guidance), they write less boilerplate and **can focus on the features that build your business**. And when the app is performant and stable, you spend less time fixing production issues or optimizing code, and more time delivering value to users. Teams can **deliver new capabilities to customers sooner**. In a competitive market, that agility can be the difference between leading and catching up.

Staying current with Angular can be a boost for **developer morale and hiring**. Top engineering talent enjoys working with the latest technology. By showing that your company invests in keeping its tech stack modern, you make your project more attractive to current and prospective developers. This can help retain your best people and recruit skilled Angular developers more easily (since they won't be stepping into a legacy). **A motivated, modernized team is a more productive team**.

Finally, a modern Angular app ensures **support and compatibility**. Angular has an LTS (long-term support) policy for older versions, but that support eventually ends. By using Angular 21, you ensure you're within the official support window, meaning any critical bugs or security issues will be addressed by the Angular team. **You also gain compatibility with the ecosystem**: third-party libraries, tools, and browsers will all target modern Angular versions. You won't be stuck with outdated dependencies (which can happen if you let your app age too long without updates). In short, regular upgrades **de-risk your project** by keeping it aligned with the active Angular ecosystem and community.

The business case for upgrading Angular is clear. It's an opportunity to deliver a faster, better product to your users and to enable your development team to do their best work. While upgrades do require effort, Angular has made them more seamless than ever, and the payoff in performance, productivity, and peace of mind is well worth it.

If your organization doesn't have the bandwidth or in-house expertise to manage the upgrade, consider partnering with an expert Angular team. Experienced Angular consultants can execute upgrades efficiently using proven tools and strategies based on experience from plenty of similar projects.

At House of Angular we can help you **leverage the new features to their fullest potential, also by training your team**. As Angular continues to evolve, having experienced guides can turn your framework upgrade into a smooth ride and let you focus on the business.

To better understand the real-world impact of these changes, let's turn to insights from leading Angular experts working at the forefront of the framework.

How important is it to stay up-to-date with all the latest changes in Angular, given the current pace of changes?

Keeping Angular up to date isn't just about security—it's a foundation for your entire digital strategy. Modern updates bring performance boosts, better tooling, and support for the latest browser features. They also reduce technical debt and ensure compatibility with third-party libraries. This in turn improves developer satisfaction, helping you attract and retain talent. Falling behind creates hidden costs: longer upgrade cycles, slower delivery, and missed innovation opportunities. In today's fast-moving digital world, staying current with Angular is key to delivering a fast, secure, and competitive product that users and developers love.

~ Mateusz Stefańczyk
Google Developer Expert,
Angular Team Leader at House of Angular

Nowadays, Angular is making rapid changes, by introducing standalone APIs, signals, new control flow, etc. Although it is completely possible to not adapt the new changes and for example, keep using NgModules because of the backwards compatibility, it is still recommended to adapt to the changes, because there are, and will be even more, features only available for newer APIs. A great example is host directives, which are only possible with standalone APIs. But don't worry about refactoring, because Angular usually delivers migration schematics to migrate to standalone APIs, or to the new control flow.

~ Stefan Haas
Nx Champion

In this context, a rational approach is derived from the observed behavior of the majority of clients I have encountered. Within the organizations I have collaborated with, those employing Angular-based applications exhibit a certain reluctance when it comes to embracing the latest Angular versions. Their hesitancy arises from their search for compelling reasons to justify such upgrades. At the core of this deliberation lies the end consumer, the user, and their user experience.

For instance, take the case of Angular 15's Standalone API. Its implementation may not manifest any discernible impact on the end user; it could be argued that its primary effects are confined to enhancing the developer experience, and even this judgment is inherently subjective. It's important to acknowledge the formidable challenge posed by the task of updating huge applications that lack comprehensive test coverage. The absence of robust testing engenders substantial risks, rendering the application more susceptible to vulnerability in the face of such alterations.

~ Artur Androsovych
Angular Expert

In my opinion, it is very important. Especially in times of dynamic changes, I think the strategy of frequent and small updates is much more beneficial than waiting until everything settles down. I always invest time to keep my Angular projects updated to benefit from new Angular features and improvements. New features bring new patterns and unleash better architectural seditions. Also, a simple update can often bring you performance, security, and bundle size improvements for free.

~ Dmytro Mezhenskyi
Google Developer Expert

I know with all the exciting new changes coming out, you may be excited to embrace the latest and greatest. And, pragmatically, I think it's a great way to learn for personal projects or small isolated projects, but when it comes to larger projects involving a lot of developers, I'd urge you to hold back for 2 things: For your team to feel comfortable discussing the change and taking on the work, and for good documentation and best practices to be established. Some of the new concepts are just the beginning of bigger things and we may find reasons to tweak, iterate, or pivot. No need to chase shiny as the goal. On the other hand, Angular has also released security updates, enhancements, and updating their dependencies to ensure the Angular projects we create are secure in recent versions. It just doesn't get all the attention some of the bigger changes get. It's vital to update Angular and dependencies following security guidelines and when enhanced security features are released.

~ J. Alisa Duncan
Google Developer Expert

If your current project is based on Angular 10 or an earlier version, it may start to feel outdated within the rapidly evolving framework landscape. The community is continually striving to stay up-to-date, and you'll often find that answers on platforms like StackOverflow and other websites revolve around the latest Angular frameworks.

Therefore, it is crucial to consider regularly upgrading your enterprise application to a version close to the latest release. From a practical standpoint, I recommend maintaining a gap of -1 or -2 Angular versions from the current one. This approach allows ample time for many enterprise projects with dependencies on various Angular libraries, both open-source and commercial, to catch up and align their libraries with the most recent developments.

~ Balram Chavan
Google Developer Expert

I think it's more important than ever to stay up-to-date and try to adopt every new major version of the framework as it is released. That way, you won't feel overwhelmed in a year or two, making it even more difficult to catch up.

~ Alain Chautard
Google Developer Expert

The innovation phase in Angular is still ongoing. It will end at some point and we will have another stable phase, but I expect that the innovation trend will continue this year and in 2025. Significant new features are likely to include selector-less and signal components, a new signal-based form feature, zoneless partial hydration, and probably a HttpClient without Observables.

Upgrading is about gaining access to new features and productivity improvements (e.g. the new API (input, output)). With Signals and the new upcoming APIs, a codebase pre-dating Angular 14 is currently outdated.

Tooling is also essential. Built tools are becoming faster and better, and often tend to depend on a modern codebase.

Another issue with an outdated codebase is the difficulty of onboarding new developers. They might not have heard about NgModules, have been trained to work with Signals, and struggle with OnChanges, @Input decorators, etc.

Staying up to date with the latest version of Angular is crucial.

However, there are also other criteria to consider. The main one is available resources. One may not be able to modify/rewrite huge code bases with every new release.

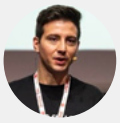
Most companies will find themselves in the middle. This means trying to make use of Angular's new features when they implement a new feature themselves or when refactoring parts of their app.

That would be an incremental migration with different layers of "Angular generations." It's a compromise, but it's always better than sticking to the old waysor doing a big-bang migration.

~ Rainer Hahnekamp
Google Developer Expert

You have to be up-to-date with all the frameworks that you want to choose. Sometimes it's difficult to explain this to the business, but with a few small steps you can make it happen.

What we can do in my team is basically use the Dependabot. Within a minor release or a patch, we have a CI, and the Dependabot will try to update the library. And if the CI passes and it's green, it's an auto-merge in the codebase. This is fantastic. When you have a major update, you need to schedule it, as it's normal to spend some time upgrading. However, if you spend three days upgrading to the major version, you'll benefit from all the code-based updates and can utilize the latest features. This not only helps your current team members but also makes it easier to integrate new developers. Conversely, if you lose the ability to upgrade to the latest version, you risk losing valuable talent from your team.



~ Luca del Puppo
Google Developer Expert

I'd say you need to stay reasonably up-to-date to understand general direction of the framework. I think most attention should be paid to breaking changes and security patches. For those, migrating as soon as possible makes sense. I'd put less priority on migrations because of new paradigms, e.g, standalone components, or minor productivity gains. In general, it's falling too far behind Angular releases can limit the options when deciding which package you can install. Also, the further you're from breaking changes, the harder it is to migrate.

~ Max Koretskyi
Founder of Angular In Depth
(now part of angular.love)

I value new changes and how the framework evolves. But sometimes it can be indeed hard to keep up with all these changes, especially if you are struggling to update your project on each new release. That's why I created Angular Can I Use, just to discover I was not the only one failing at reminding me when a given feature is now stable.

The good thing is that the Angular 2 projects still work! You obviously want to update because that's not just about new framework changes, but fixing some bugs or security vulnerabilities from the dependencies galaxy that Angular depends on.

But don't feel shame at updating your projects while delaying updating your codebase. The Angular team and the community got you covered by providing migration scripts you can run when you are ready!

~ Gérôme Grignon
Angular Devs France Founder

Thank You

We hope you found this ebook informative and that its contents will help you **improve your app's performance, user experience and overall project efficiency.**

If you'd like to upgrade the Angular version in your project but don't know where to start, or if you have any questions about Angular or its features, **we would be happy to help.**

Book a 30-minute meeting with an ebook author to discuss your challenges or just contact us by e-mail: **ebook@houseofangular.io**

Consulting and audit

Identifying bottlenecks within an Angular project can be challenging. Why can't your product scale, and new team members don't increase the project's velocity?

This is where **expert advice** can make a significant difference.

Our team at House of Angular is passionate about Angular. We have worked with the framework since its release and will happily share our knowledge and experience with you through **audits** or **consulting**.

How the audit can benefit your project

An audit of the code can help you **identify vulnerabilities** within the code and figure out how to deal with them. It will also help you determine how to **leverage Angular's new features in your project effectively**. Our experienced Angular Developers will give you recommendations in four key areas:

- Code quality
- Functionality
- Architecture
- Performance

You will receive a **detailed report** and get access to:

- A list of places that should be improved, sorted by priority and complexity
- Identified risks when it comes to long-term development
- Recommendations about tools and libraries your team should use
- Best Angular practices that will benefit the project

If you'd like to find out whether the audit or consulting will address the challenges in your Angular project, just fill out the contact form. We will schedule the first 30-minute talk just to discuss your project.

Contact us

Media partners

NG-DE Conference

The **NG-DE** Conference, Germany's leading Angular event, returns in 2026! It brings together Angular developers, architects, and enthusiasts for three days of high-quality talks, practical workshops, and networking opportunities. With speakers including Angular team members and international experts, NG-DE focuses on cutting-edge Angular development, best practices, and real-world case studies, making it a must-attend event for anyone serious about modern web development.

WeAreDevelopers

The **WeAreDevelopers** World Congress is the world's leading event for developers, taking place from July 8–10, 2026, in Berlin. Bringing together 15,000+ developers and 500+ high-level speakers, the congress is designed to exchange knowledge, challenge assumptions, and push technology forward. Use the exclusive discount code "Community_Angular" for 10% off. Secure your spot at worldcongress.dev.

NGRome

NGRome invites developers to experience the 'Dolce Vita' of the Angular world. As Italy's biggest Angular gathering, it pairs top-tier technical content—including workshops and conference talks—with the unforgettable backdrop of Rome. Attendees can look forward to building connections, exploring cutting-edge topics, and enjoying world-class Italian hospitality. Join the experience at **ngrome.io**.

DevDays

DevDays Europe (returning to Vilnius, May 19-22, 2026) brings together internationally recognized speakers and developers to encourage excellence and innovation. The conference covers emerging technologies and best practices regardless of technological platform or language—all without commercial hype. Attendees learn about the latest tech advances from experts specifically flown in for the event, ensuring high-quality, unbiased knowledge transfer directly from industry leaders and peers.

Bibliography

Official Typescript Documentation: https://www.typescriptlang.org/
Official Angular Documentation (old): https://angular.io/
Official Angular Documentation (new): https://angular.dev/
Official Angular blog: https://blog.angular.dev/
Angular.love blog: https://angular.love/

About authors

Main Authors

Mateusz Dobrowolski
Angular Developer at House of Angular. Key Contributor to Angular.love. He takes an active part in the angular.love community, writing expert articles, and sharing his knowledge at Angular meetups.

If you found this e-book useful (or not) please, let us know. Send your feedack to the author: @m-dobrowolski

Mateusz Stefańczyk
Google Developer Expert. Angular Team Leader at House of Angular. Key Contributor to Angular.love. For 7 years, he has been developing web applications with Angular. He has performed dozens of audits for Angular projects worldwide. Mateusz actively participates in the angular.love community, writing expert articles, and sharing his knowledge at Angular meetups in Poland, Norway, Germany, and the UK.

If you found this e-book useful (or not) please, let us know. Send your feedack to the author: @m-stefanczyk

Miłosz Rutkowski
Angular Developer at House of Angular, passionate about clean code, best practices, and well-structured architecture. Miłosz actively contributes to the angular.love community by writing blog posts and developing new initiatives for the blog website.

If you spotted something that needs fixing, felt like something was missing, or enjoyed the e-book, please don't hesitate to reach out. Send your feedback to the author: @miłosz-rutkowski

Special thanks to

Krzysztof Skorupka
Angular Team Leader & Architect at House of Angular. Key Contributor at Angular.love community, blogger, and speaker.

Przemysław Łęczycki
Angular Team Leader at House of Angular. Expert at Angular.love community, blogger, and speaker.

Dominik Donoch
Angular Developer at House of Angular. Key Contributor Angular.love.

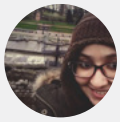
Dawid Kostka
Angular Developer at House of Angular. Angular.love Contributor.

Piotr Wiórek
Blogger and member of Angular.love community.

Mateusz Basiński
Angular Developer at House of Angular. Angular.love blogger.

Maja Hendzel
Angular Developer at House of Angular. Angular.love blogger.

Invited experts



Fatima Amzil
Fatima is a cross-functional frontend technical leader and an expert in Angular. Outside of her professional life, she contributes as a technical writer on Medium, mentor at MyJobGlasses, and an Angular GDE.

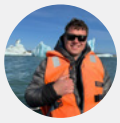
Artur Androsovych
Artur is an OSS core developer of various projects as NGXS, NG-ZORRO, single-spa, ng-neat (until-destroy code owner), and RxAngular. He was a Google Developer Expert in Angular in 2023. He focuses on runtime performance and has taught teams about Angular internals for the past few years.

Aristeidis Bampakos
Aristeidis is an Angular GDE who works as Web Development Team Lead at Plex-Earth. He is an award-winning author of Learning Angular and Angular Projects books. Currently, he is leading the effort of making Angular accessible to the Greek development community by maintaining the Greek translation of the official Angular documentation.

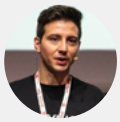
Alain Chautard
Alain is a Google Developer Expert in Angular, and Google Maps. His daily mission is to help development teams adopt Angular and build at scale with the framework. He has taught Angular on all six continents!

Balram Chavan
Balram is an Architect specializing in designing and creating scalable, secure solutions for various domains. His expertise spans the cloud, AI and web development landscape. As the author of an Angular framework book, he excels as a team player, an effective communicator, and a mentor.

Gérôme Grignon
Gérôme is a Frontend Software Engineer at Lucca and the creator of Angular Caniuse. He shares expertise with the community as a moderator on the official Angular Discord server, an open-source project maintainer, and a content creator. He co-organizes Angular Devs France, providing video content about Angular for the French-speaking community.

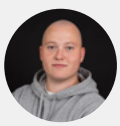


Max Koretskyi
Founder of Angular in Depth blog (now part of angular.love). Max is an experienced full-stack engineer with a strong focus on frontend architecture, performance optimization, and infrastructure automation.



Luca Del Puppo
Luca is a Senior Software Developer, Microsoft MVP, Google Developer Expert and GitKraken Ambassador. He loves JavaScript and TypeScript. In his free time, I loves studying new technologies, improving himself, creating YouTube content or writing technical articles.

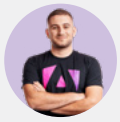
J. Alisa Duncan
Alisa is a Senior Developer Advocate at Okta, full-stack developer, content creator, conference speaker, Pluralsight author, and community builder who loves the thrill of learning new things. She is a Google Developer Expert in Angular, a Women Techmaker Ambassador, a ngGirls core team member, and a volunteer at community events supporting underrepresented groups entering tech.



Stefan Haas
Stefan is a freelancer and trainer focusing on Angular from Austria. He is the author of NG Journal - The Place Beyond Fundamentals - and an Nx Champion.

Rainer Hahnekamp

Rainer is a Google Developer Expert, working as a trainer and consultant in the expert network of Angular Architects. In addition, he offers a weekly brief overview of relevant events in the Angular ecosystem on YouTube through **ng-news**.



Enea Jahollari

Enea is a Google Developer Expert for Angular, Consultant, Trainer & Senior Software Engineer who builds, audits and optimizes web applications @ Push-Based.io. He loves open source and contributes to it by writing code, content, and tweets! Loves hyping Angular in his free time.

Connie Leung

Google Developer Expert for Angular. Software Architect at Diginex, blogger and youtube content creator.

Dmytro Mezhenskyi

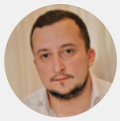
Dmytro Mezhenskyi is the founder of the **Decoded Frontend** YouTube channel, where he shares his knowledge about the Angular framework. As an author, he has created a series of advanced video courses focusing on Angular and GraphQL. With over 10 years of experience in frontend development, Dmytro has been recognized as a Google Developer Expert in Angular and a Microsoft MVP in Web Development.

David Muellerchen

Better known as WebDave, David has been an integral part of the Angular community since 2014. He is a Google Developer Expert, an Angular consultant and trainer, and a frequent speaker at meetups and conferences. He also organizes weekly Angular livestreams and the Hamburg Angular Meetup.

Fanis Prodromou

Fanis is a full-stack web developer with a passion for Angular and NodeJs. He is a Google Developer Expert, co-organizer of Angular Athens meetups, who also creates content for YouTube channe **Code Shots With Profanis**. During 14 years of coding, he has developed vast experience in code quality, application architecture, and application performance.



Marko Stanimirović

Marko is a Principal Frontend Engineer at Swiss Marketplace Group. He is also a core member of the NgRx and AnalogJS teams, a Google Developer Expert in Angular, and an organizer of the Angular Belgrade group.

Kasia Biernat - Kluba

Kasia is a software engineer and a web developer. Her main area of expertise is Angular, but she also has experience with backend technologies, such as Node.js and NET. She is an occasional speaker at tech meetups and conferences. You can also find her at NgKato meetups, which are regularly hosted in Katowice, where she is a co-organizer.

HOUSE OF ANGULAR

House of Angular is a software agency with a focus on developing and supporting Angular applications. They offer a variety of services, including:

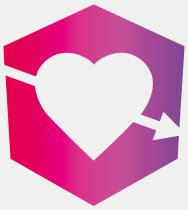
- application development,
- team extension,
- product and team audits,
- and business consulting.

Having worked with Angular since its beginning and with over 10 years in app development, they have become a trusted business partner for companies of different sizes and sectors.

Their goal is to craft the finest frontend solutions using Angular, while also imparting their knowledge and expertise to other users of this framework.

The company is actively involved in nurturing the Angular community. They support key Angular libraries like ngrx, NGXS, and ngneat. Additionally, their team members contribute to open-source Angular projects and spread their expertise through writing articles and giving talks at European meetups.

House of Angular has been recognized for their contributions with the Angular Hero of Community 2021 award.



angular.love

Angular.Love is a community platform for Angular enthusiasts, supported by House of Angular to facilitate the growth of Angular developers through knowledge-sharing initiatives. It started as a blog where experts published articles about Angular news, features, and best practices. Now angular.love also organizes in-person and online meetups, which frequently feature Google Developer Experts.

Angular developers can find expert insights and expand their Angular knowledge by reading the angular.love blog, attending meetups with talks from experts, and following the community platform's social media pages.

Angular.love community is a recipient of the Angular Hero of Education 2022 award.

Blog | Meetups | LinkedIn | YouTube | X

HOUSE OF ANGULAR

created in cooperation with

angular.love